

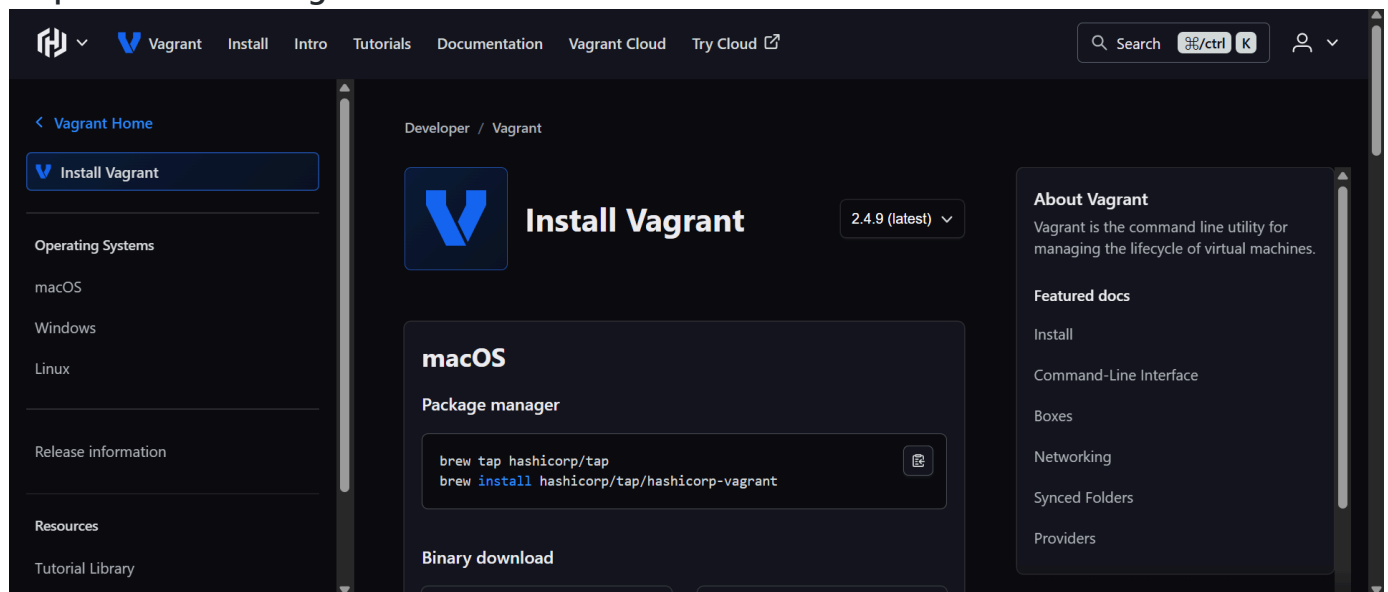
Containerization-and-DevOps

Experiment-1 : VM vs Container

PART-A:- Virtual Machine (Windows)

Step 1: Download Virtual Box from [here](#)  Download VM

Step 2: Download Vagrant from [here](#)



Step 3: To verify the installation we will check the version via following command

```
vagrant --version
```

```
PS C:\Users\srist> vagrant --version
Vagrant 2.4.9
```

Step 4: Initialize Vagrant with Ubuntu box:

```
vagrant init hashicorp/bionic64
```

```
PS C:\Users\srist> vagrant --version
Vagrant 2.4.9
```

Step 5: Start the VM:

```
vagrant up
```

```
PS C:\Users\srist> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'hashicorp/bionic64' could not be found. Attempting to find and install...
    default: Box Provider: virtualbox
    default: Box Version: >= 0
==> default: Loading metadata for box 'hashicorp/bionic64'
    default: URL: https://vagrantcloud.com/api/v2/vagrant/hashicorp/bionic64
==> default: Adding box 'hashicorp/bionic64' (v1.0.282) for provider: virtualbox
    default: Downloading: https://vagrantcloud.com/hashicorp/boxes/bionic64/versions/1.0.282/providers/virtualbox/unknown/vagrant.box
Progress: 24% (Rate: 142k/s, Estimated time remaining: 0:02:13)
```

Step 6: Access the VM:

```
vagrant ssh
```

```
PS C:\Users\srist> vagrant ssh
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-58-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information disabled due to load higher than 1.0

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

0 packages can be updated.
0 updates are security updates.

vagrant@vagrant:~$ |
```

Step 7: Install Nginx inside VM

```
sudo apt update
sudo apt install -y nginx
sudo systemctl start nginx
```

```
vagrant@vagrant:~$ sudo apt update
Get:1 http://security.ubuntu.com/ubuntu bionic-security InRelease [102 kB]
Hit:2 http://archive.ubuntu.com/ubuntu bionic InRelease
Get:3 http://archive.ubuntu.com/ubuntu bionic-updates InRelease [102 kB]
Get:4 http://security.ubuntu.com/ubuntu bionic-security/main i386 Packages [1,380 kB]
Get:5 http://archive.ubuntu.com/ubuntu bionic-backports InRelease [102 kB]
Get:6 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 Packages [3,045 kB]
Get:7 http://archive.ubuntu.com/ubuntu bionic-updates/main i386 Packages [1,666 kB]
Get:8 http://security.ubuntu.com/ubuntu bionic-security/main amd64 Packages [2,717 kB]
Get:9 http://archive.ubuntu.com/ubuntu bionic-updates/main Translation-en [554 kB]
Get:10 http://archive.ubuntu.com/ubuntu bionic-updates/restricted i386 Packages [39.7 kB]
Get:11 http://archive.ubuntu.com/ubuntu bionic-updates/restricted amd64 Packages [1,347 kB]
Get:12 http://security.ubuntu.com/ubuntu bionic-security/main Translation-en [468 kB]
Get:13 http://security.ubuntu.com/ubuntu bionic-security/restricted amd64 Packages [1,317 kB]
Get:14 http://archive.ubuntu.com/ubuntu bionic-updates/restricted Translation-en [187 kB]
Get:15 http://security.ubuntu.com/ubuntu bionic-security/restricted i386 Packages [33.0 kB]
Get:16 http://archive.ubuntu.com/ubuntu bionic-updates/universe i386 Packages [1,663 kB]
Get:17 http://security.ubuntu.com/ubuntu bionic-security/restricted Translation-en [182 kB]
```

```
vagrant@vagrant:~$ sudo apt install -y nginx
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  fontconfig-config fonts-dejavu-core libfontconfig1 libgd3 libjpeg-turbo8 libjpeg8 libnginx-mod-http-geoip
  libnginx-mod-http-image-filter libnginx-mod-http-xslt-filter libnginx-mod-mail libnginx-mod-stream libtiff5 libwebp6 libx11-6
  libx11-data libxcb1 libxpm4 nginx-common nginx-core
Suggested packages:
  libgd-tools fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  fontconfig-config fonts-dejavu-core libfontconfig1 libgd3 libjpeg-turbo8 libjpeg8 libnginx-mod-http-geoip
  libnginx-mod-http-image-filter libnginx-mod-http-xslt-filter libnginx-mod-mail libnginx-mod-stream libtiff5 libwebp6 libx11-6
  libx11-data libxcb1 libxpm4 nginx nginx-common nginx-core
0 upgraded, 21 newly installed, 0 to remove and 294 not upgraded.
Need to get 3,195 kB of archives.
After this operation, 11.4 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 libjpeg-turbo8 amd64 1.5.2-0ubuntu5.18.04.6 [111 kB]
Get:2 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 libxcb1 amd64 1.13-2~ubuntu18.04 [45.5 kB]
Get:3 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 libx11-data all 2:1.6.4-3ubuntu0.4 [114 kB]
Get:4 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 libx11-6 amd64 2:1.6.4-3ubuntu0.4 [572 kB]
Get:5 http://archive.ubuntu.com/ubuntu bionic/main amd64 fonts-dejavu-core all 2.37-1 [1,041 kB]
Get:6 http://archive.ubuntu.com/ubuntu bionic/main amd64 fontconfig-config all 2.12.6-0ubuntu2 [55.8 kB]
Get:7 http://archive.ubuntu.com/ubuntu bionic/main amd64 libfontconfig1 amd64 2.12.6-0ubuntu2 [137 kB]
Get:8 http://archive.ubuntu.com/ubuntu bionic/main amd64 libjpeg8 amd64 8c-2ubuntu8 [2,194 B]
Get:9 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 libjpeg-turbo8 amd64 2.1-3.1ubuntu0.18.04.1 [27.0 kB]
```

Step 8: Verify Nginx

```
curl localhost
```

```

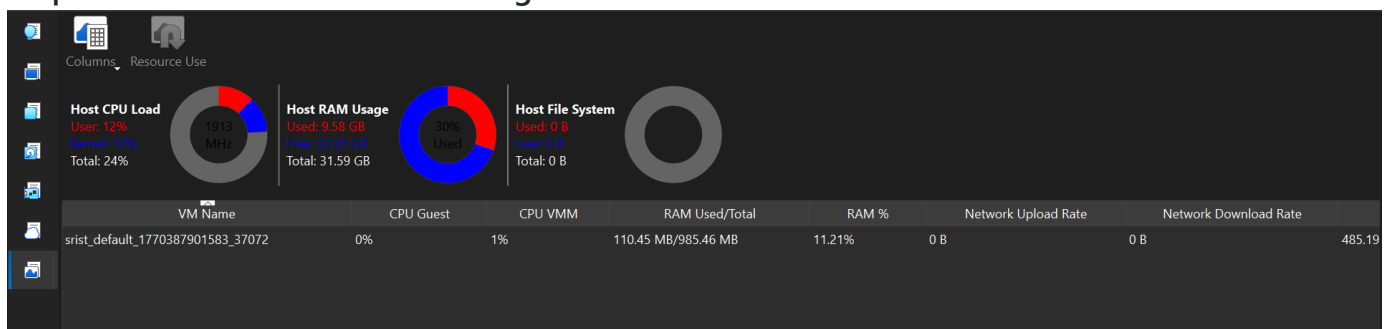
vagrant@vagrant:~$ sudo systemctl start nginx
vagrant@vagrant:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>

```

Step 9: Utilization Matrix In Running State



Step 10: Stop VM

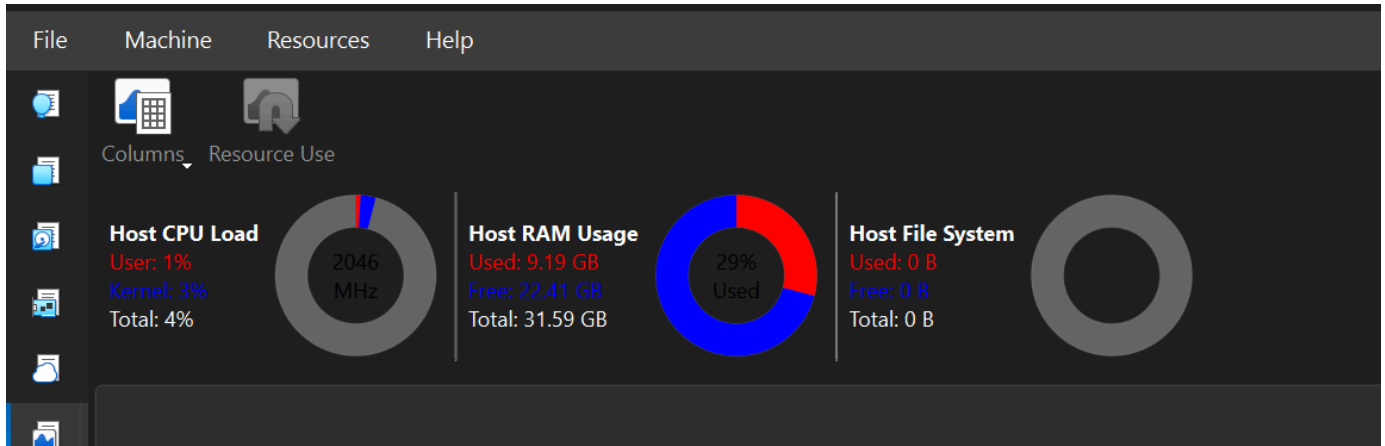
```
vagrant halt
```

```

PS C:\Users\srist> vagrant halt
==> default: Attempting graceful shut down of VM
PS C:\Users\srist>

```

Step 11: Utilization Matrix In Stop State



Step 12: Remove VM

```
vagrant destroy
```

```
vagrant@vagrant:~$ exit
logout
PS C:\Users\srist> vagrant halt
==> default: Attempting graceful shutdown of VM...
PS C:\Users\srist> |
```

PART-B:- Containers using WSL (Windows)

Step 1: Install WSL

```
wsl --install
```

Reboot the system after installation.

Step 2: Install Ubuntu on WSL

```
wsl --install -d Ubuntu
```

Step 3: Install Docker Engine inside WSL

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl start docker
sudo usermod -aG docker $USER
```

Logout and login again to apply group changes.

Step 4: Verify Docker Installation

```
docker --version
```

```
sristee@Sristee:/mnt/c/Users/srist$ sudo docker --version
[sudo] password for sristee:
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

Step 5: Pull Ubuntu Image

```
docker pull ubuntu
```

```
sristee@Sristee:/mnt/c/Users/srist$ sudo docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881737d24f200828b2b
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest
```

Step 6: Run Ubuntu Container with Nginx

```
docker run -d -p 8080:80 --name nginx-container nginx
```

```
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Waiting
10b68cfefee1: Download complete
57f0dd1befe2: Download complete
eaf8753feae0: Download complete
```

Step 7: Verify Nginx in Container

```
curl localhost:8080
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Step 8: Container Observation Commands

```
docker stats
free -h
```

```
sristee@Sristee: /mnt/c/Users  ×  +  ▾
CONTAINER ID   NAME          CPU %       MEM USAGE / LIMIT   MEM %       NET I/O       BLOCK I/O      PIDS
5b1a1944aa35   nginx-container  0.00%       14.42MiB / 15.42GiB  0.09%       2.02kB / 1.4kB  14.1MB / 12.3kB  13

sristee@Sristee: /mnt/c/Users/srist$ free -h
              total        used        free      shared  buff/cache   available
Mem:           15Gi         621Mi        14Gi        3.6Mi       703Mi        14Gi
Swap:           4.0Gi           0B         4.0Gi
```

Parameters to Compare

Parameter	Virtual Machine	Container
Boot Time	High	Very Low
RAM Usage	High	Low
CPU Overhead	Higher	Minimal
Disk Usage	Larger	Smaller
Isolation	Strong	Moderate

Containerization-and-DevOps

Experiment 2:- Docker Installation, Configuration, and Running Images

Step 1: Pull Image

```
docker pull nginx
```

```
This message is shown once a day. To disable it please create the
/home/sristee/.hushlogin file.
sristee@Sristee:/mnt/c/Users/srist$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
10b68cfefee1: Pull complete
57f0dd1befe2: Pull complete
eaf8753feae0: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Step 2: Run Container with Port Mapping

```
docker run -d -p 8080:80 nginx
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker run -d -p 8080:80 nginx
667f92412486fbf885b1eb1d6c8b82a4cee2866a6a22deaf821c6cda8cfa9264
```

Step 3: Verify Running Containers

```
docker ps
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
667f92412486	nginx	"/docker-entrypoint..."	About a minute ago	Up About a minute	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

Step 4: Stop Container

```
docker stop <container_id>
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker stop 667f92412486
667f92412486
```

Step 5: Remove Container

```
docker rm <container_id>
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker rm 667f92412486
667f92412486
```

Step 5: Remove Image

```
docker rmi nginx
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker rmi nginx
Untagged: nginx:latest
Untagged: nginx@sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Deleted: sha256:4af177a024eb8a1e43f4fb6c66735bb8260115cb5925a64f51673219bd97c144
Deleted: sha256:74821a6b71dfac14eaeb80c7c9e48130391306557f75228278054afe105cde75
Deleted: sha256:ae46c5037a8cc17f658f78b0e6523c9363ca800bf51f91f58cbc3aad8ffe53b5
Deleted: sha256:ec81248cdb6b02bb24b1c32d4f59194a4a42aab3c81a42631efa1af7cba4f69
Deleted: sha256:774cc13f29089d7d0ebaaa291c5ac2d12841f2d70ab07a21c470d3cbdc11c059
Deleted: sha256:f1155703f3479bf6b295f5e17684156f82f815ca0798af8a2d08cf994411a055
Deleted: sha256:a455653294af11c547d0b3d2d769fd184f357cd5c0decabd1f89ffe9b380985aa
Deleted: sha256:e50a58335e1366e2581fe61794c1651afe2fe04e881e795aa166f24f4fc78d92
```

Result Docker images were successfully pulled, containers executed, and lifecycle commands performed.

Containerization-and-DevOps

Experiment 3:- Docker Deploy NGINX Using Different Base Images and Comparing Image Layers

Part 1: Deploy NGINX Using Official Image (Recommended Approach)

Step 1: Pull the Image

```
docker pull nginx:latest
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker pull nginx:latest
latest: Pulling from library/nginx
0c8d55a45c0d: Already exists
46bf3a120c8e: Pull complete
4f4efe02d542: Pull complete
7b6cb8ccac7b: Pull complete
f73400a233fd: Pull complete
47cd406a84ef: Pull complete
bae5a1799a80: Pull complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Step 2: Run the Container

```
docker run -d --name nginx-official -p 8080:80 nginx
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker run -d --name nginx-official -p 8080:80 nginx
ea4aff8a2945dda2c19ac0d6f9be01dd27142e9adf6b6570cf8f1825d4d78257
```

Step 3: Verify

```
curl http://localhost:8080
```

```
sristee@Sristee:/mnt/c/Users/srist$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

You should see the NGINX welcome page.

Key Observations

docker images nginx

```
sristee@Sristee:/mnt/c/Users/srist$ docker images nginx
REPOSITORY      TAG          IMAGE ID      CREATED        SIZE
nginx           latest       5cdef4ac3335  2 weeks ago   161MB
nginx           <none>       248d2326f351  2 weeks ago   161MB
```

- Image is pre-optimized
- Minimal configuration required
- Uses Debian-based OS internally

Part 2: Custom NGINX Using Ubuntu Base Image

Step 1: Create Dockerfile

```
FROM ubuntu:22.04
```

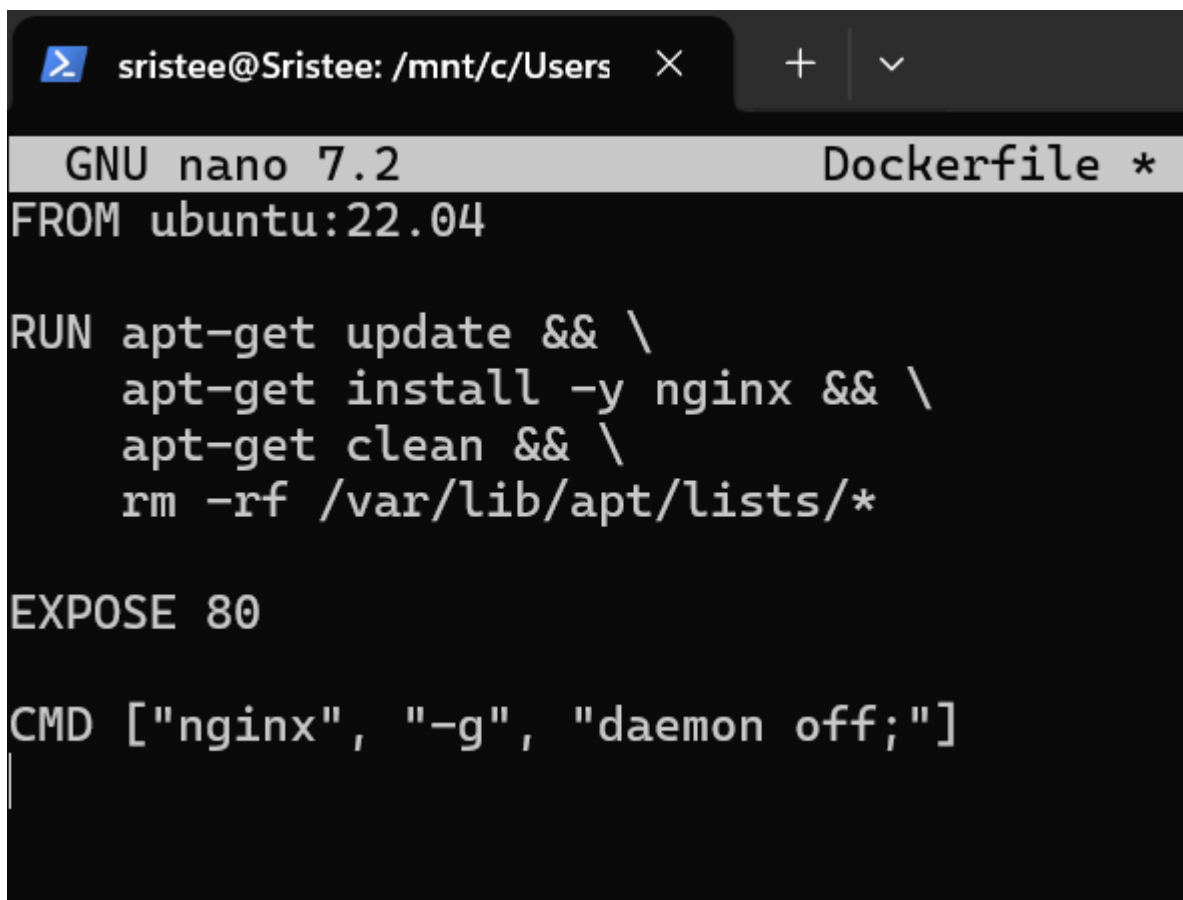
```
RUN apt-get update && \
    apt-get install -y nginx && \
```

```
apt-get clean && \  
rm -rf /var/lib/apt/lists/*
```

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

```
sristee@Sristee:/mnt/c/Users/srist$ mkdir nginx-ubuntu  
cd nginx-ubuntu  
sristee@Sristee:/mnt/c/Users/srist/nginx-ubuntu$ pwd  
/mnt/c/Users/srist/nginx-ubuntu  
sristee@Sristee:/mnt/c/Users/srist/nginx-ubuntu$ nano Dockerfile  
sristee@Sristee:/mnt/c/Users/srist/nginx-ubuntu$ ls  
Dockerfile
```



The screenshot shows a terminal window with a tab titled 'sristee@Sristee: /mnt/c/Users'. The active application is 'GNU nano 7.2' editing a file named 'Dockerfile *'. The content of the Dockerfile is as follows:

```
FROM ubuntu:22.04  
  
RUN apt-get update && \  
    apt-get install -y nginx && \  
    apt-get clean && \  
    rm -rf /var/lib/apt/lists/*  
  
EXPOSE 80  
  
CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

```
docker build -t nginx-ubuntu .
```

```

sristee@Sristee:/mnt/c/Users/srist/nginx-ubuntu$ docker build -t nginx-ubuntu .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/4 : FROM ubuntu:22.04
----> 65c77cbc27c2
Step 2/4 : RUN apt-get update &&          apt-get install -y nginx &&          apt-get clean &&          rm -rf /var/lib/apt/lists/*
----> Running in 4ecb37c7c3d5
Get:1 http://archive.ubuntu.com/ubuntu jammy InRelease [270 kB]
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:3 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1301 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy/multiverse amd64 Packages [266 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy/main amd64 Packages [1792 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [6538 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy/universe amd64 Packages [17.5 MB]
Get:10 http://archive.ubuntu.com/ubuntu jammy/restricted amd64 Packages [164 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [6749 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 Packages [70.9 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [4070 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 Packages [62.6 kB]
Get:15 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3737 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1613 kB]
Get:17 http://archive.ubuntu.com/ubuntu jammy-backports/universe amd64 Packages [37.2 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-backports/main amd64 Packages [83.9 kB]
Fetched 44.6 MB in 22s (2019 kB/s)
Reading package lists...
Reading package lists...
Building dependency tree...
Reading state information...

```

Step 3: Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

```

sristee@Sristee:/mnt/c/Users/srist/nginx-ubuntu$ docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
eb521404c058b6d7c277900f7589abd3b07737f0e57ec87ddd91d182b2ae424b

```

Observations

```
docker images nginx-ubuntu
```

```

sristee@Sristee:/mnt/c/Users/srist/nginx-ubuntu$ docker images nginx-ubuntu
REPOSITORY          TAG             IMAGE ID          CREATED           SIZE
nginx-ubuntu        latest         b80f01724123     About a minute ago 134MB

```

- Much larger image size
- More layers
- Full OS utilities available

Part 3: Custom NGINX Using Alpine Base Image

Step 1: Create Dockerfile

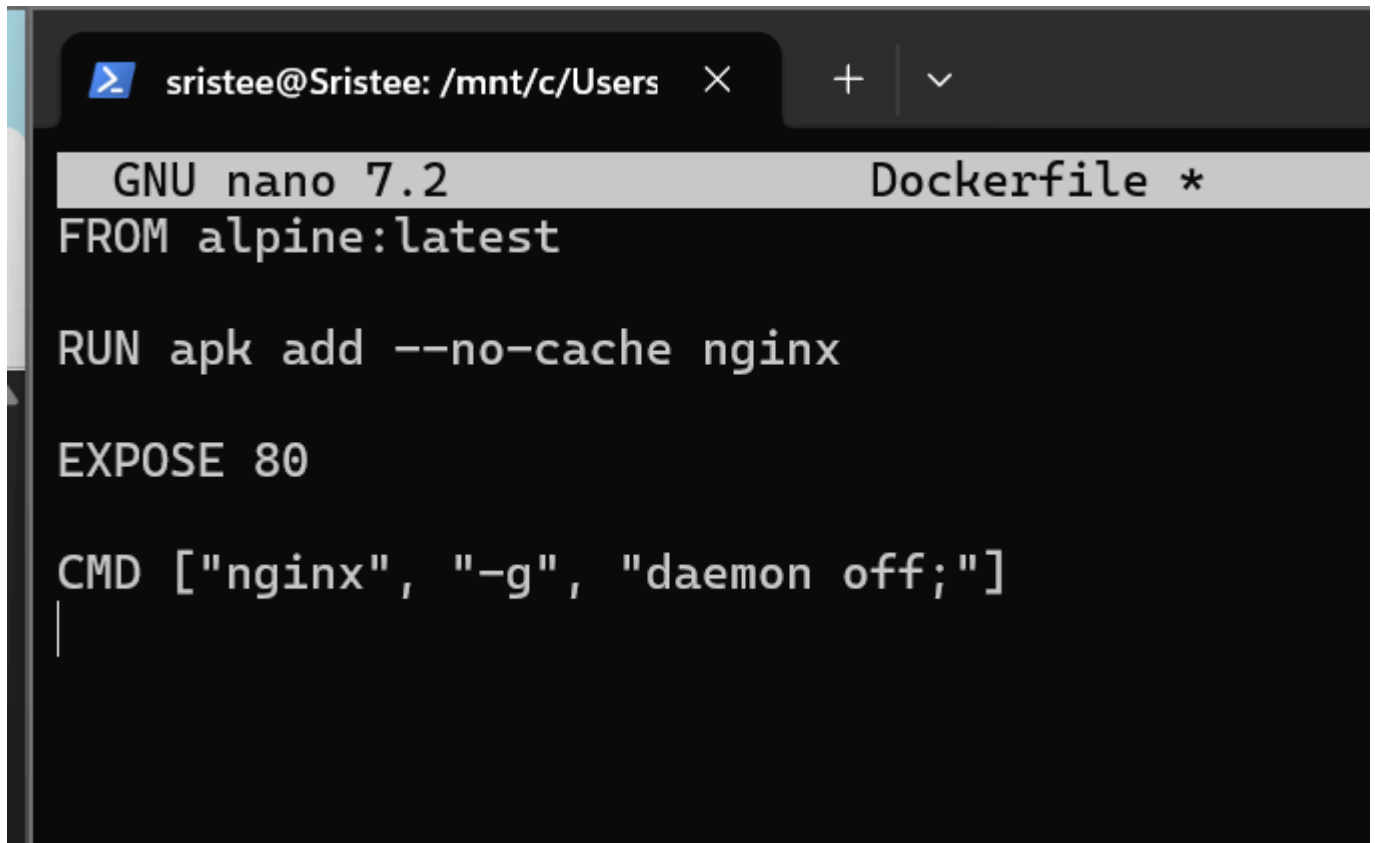
```
FROM alpine:latest
```

```
RUN apk add --no-cache nginx
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

```
sristee@Sristee:/mnt/c/Users/srist/nginx-ubuntu$ cd ..  
mkdir nginx-alpine  
cd nginx-alpine  
sristee@Sristee:/mnt/c/Users/srist/nginx-alpine$ nano Dockerfile
```



The screenshot shows a terminal window with a tab titled 'sristee@Sristee: /mnt/c/Users'. The active window is 'GNU nano 7.2' editing 'Dockerfile *'. The file content is as follows:

```
FROM alpine:latest  
  
RUN apk add --no-cache nginx  
  
EXPOSE 80  
  
CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

```
docker build -t nginx-alpine .
```

```

sristee@Sristee:/mnt/c/Users/srist/nginx-alpine$ docker build -t nginx-alpine .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/4 : FROM alpine:latest
latest: Pulling from library/alpine
589002ba0eae: Pulling fs layer
589002ba0eae: Download complete
589002ba0eae: Pull complete
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
--> a40c03cbb81c
Step 2/4 : RUN apk add --no-cache nginx
--> Running in a45bbddcaa3f
(1/2) Installing pcre2 (10.47-r0)
(2/2) Installing nginx (1.28.2-r0)
     Executing nginx-1.28.2-r0.pre-install
     Executing nginx-1.28.2-r0.post-install
Executing busybox-1.37.0-r30.trigger
OK: 9.9 MiB in 18 packages
--> Removed intermediate container a45bbddcaa3f
--> f7343d53537a
Step 3/4 : EXPOSE 80
--> Running in 11667d93471d
--> Removed intermediate container 11667d93471d
--> 462640db3de7
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]

```

Step 3: Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

```

sristee@Sristee:/mnt/c/Users/srist/nginx-alpine$ docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
735781f95690d84e1d9c9197f8d0143dd0f2231a8f68c7a501fbf07b890a2d1d

```

Observations

```
docker images nginx-alpine
```

```

sristee@Sristee:/mnt/c/Users/srist/nginx-alpine$ docker images nginx-alpine
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
nginx-alpine        latest     2adf2192b96a  45 seconds ago  10.4MB

```

- Extremely small image
- Fewer packages
- Faster pull and startup time

Part 4: Image Size and Layer Comparison

Compare Sizes

```
docker images | grep nginx
```

```
sristee@Sristee:/mnt/c/Users/srist/nginx-alpine$ docker images | grep nginx
nginx-alpine      latest      2adf2192b96a   53 seconds ago   10.4MB
nginx-ubuntu      latest      b80f01724123   3 minutes ago    134MB
nginx             latest      5cdef4ac3335   2 weeks ago      161MB
nginx             <none>      248d2326f351   2 weeks ago      161MB
```

Typical result (approx):

Image Type	Size
nginx:latest	161 MB
nginx-ubuntu	134 MB
nginx-alpine	10.4 MB

Inspect Layers

```
docker history nginx
```

```
docker history nginx-ubuntu
```

```
docker history nginx-alpine
```

```
sristee@Sristee:/mnt/c/Users/srist/nginx-alpine$ docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
5cdef4ac3335    2 weeks ago     CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>       2 weeks ago     STOPIGNAL SIGQUIT                             0B        buildkit.dockerfile.v0
<missing>       2 weeks ago     EXPOSE map[80/tcp:{}]                          0B        buildkit.dockerfile.v0
<missing>       2 weeks ago     ENTRYPOINT ["/docker-entrypoint.sh"]           0B        buildkit.dockerfile.v0
<missing>       2 weeks ago     COPY 30-tune-worker-processes.sh /docker-ent... 4.62kB    buildkit.dockerfile.v0
<missing>       2 weeks ago     COPY 20-envsubst-on-templates.sh /docker-ent... 3.02kB    buildkit.dockerfile.v0
<missing>       2 weeks ago     COPY 15-local-resolvers.envsh /docker-entryp... 389B      buildkit.dockerfile.v0
<missing>       2 weeks ago     COPY 10-listen-on-ipv6-by-default.sh /docker... 2.12kB    buildkit.dockerfile.v0
<missing>       2 weeks ago     COPY docker-entrypoint.sh / # buildkit         1.62kB    buildkit.dockerfile.v0
<missing>       2 weeks ago     RUN /bin/sh -c set -x && groupadd --sys...     82.2MB    buildkit.dockerfile.v0
<missing>       2 weeks ago     ENV DYNPKG_RELEASE=1~trixie                     0B        buildkit.dockerfile.v0
<missing>       2 weeks ago     ENV PKG_RELEASE=1~trixie                        0B        buildkit.dockerfile.v0
<missing>       2 weeks ago     ENV ACME_VERSION=0.3.1                         0B        buildkit.dockerfile.v0
<missing>       2 weeks ago     ENV NJS_RELEASE=1~trixie                       0B        buildkit.dockerfile.v0
<missing>       2 weeks ago     ENV NJS_VERSION=0.9.5                          0B        buildkit.dockerfile.v0
<missing>       2 weeks ago     ENV NGINX_VERSION=1.29.5                       0B        buildkit.dockerfile.v0
<missing>       2 weeks ago     LABEL maintainer=NGINX Docker Maintainers <d... 0B        buildkit.dockerfile.v0
<missing>       2 weeks ago     # debian.sh --arch 'amd64' out/ 'trixie' '1... 78.6MB    debuerreotype 0.17
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
b80f01724123    3 minutes ago   /bin/sh -c #(nop) CMD ["nginx" "-g" "daemon... 0B
76716be45291    4 minutes ago   /bin/sh -c #(nop) EXPOSE 80                     0B
efafd8e84355    4 minutes ago   /bin/sh -c apt-get update && apt-get ins...   55.7MB
65c77cbc27c2    5 weeks ago     /bin/sh -c #(nop) CMD ["/bin/bash"]            0B
<missing>       5 weeks ago     /bin/sh -c #(nop) ADD file:b499000226bd9a7c5... 77.9MB
<missing>       5 weeks ago     /bin/sh -c #(nop) LABEL org.opencontainers.... 0B
<missing>       5 weeks ago     /bin/sh -c #(nop) LABEL org.opencontainers.... 0B
<missing>       5 weeks ago     /bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH     0B
<missing>       5 weeks ago     /bin/sh -c #(nop) ARG RELEASE                   0B
IMAGE          CREATED          CREATED BY                                      SIZE      COMMENT
```

Observations:

- Ubuntu has many filesystem layers
- Alpine has minimal layers

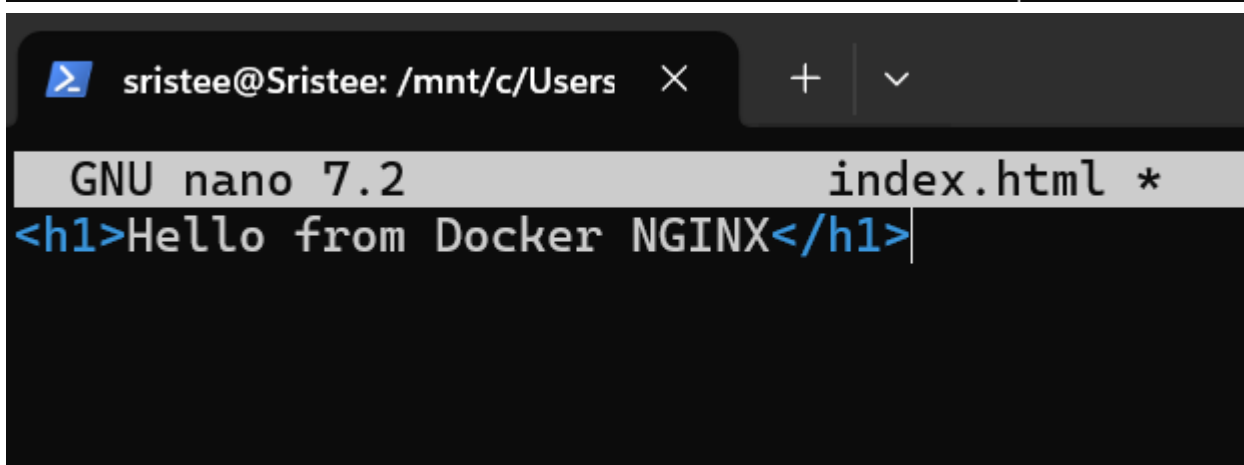
- Official NGINX image is optimized but heavier than Alpine

Part 5: Functional Tasks Using NGINX

Task 1: Serve Custom HTML Page

```
mkdir html
echo "<h1>Hello from Docker NGINX</h1>" > html/index.html
```

```
sristee@Sristee:/mnt/c/Users/srist/nginx-alpine$ mkdir html
cd html
sristee@Sristee:/mnt/c/Users/srist/nginx-alpine/html$ nano index.html
```



Run:

```
docker run -d \
  -p 8083:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx
```

```
sristee@Sristee:/mnt/c/Users/srist/nginx-alpine/html$ cd
sristee@Sristee:~$ docker run -d \
  -p 8083:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx
344384ffe88724f1f794e53e22e7979ceb51884ddab062a7de1ff178b9cc2c0d
sristee@Sristee:~$ |
```

Task 2: Reverse Proxy (Conceptual)

NGINX can:

- Forward traffic to backend services

- Load balance multiple containers
- Terminate SSL

Example use cases:

- Frontend for microservices
- API gateway
- Static file server

Part 6: Comparison Summary

Image Comparison Table

Feature	Official NGINX	Ubuntu + NGINX	Alpine + NGINX
Image Size	Medium	Large	Very Small
Ease of Use	Very Easy	Medium	Medium
Startup Time	Fast	Slow	Very Fast
Debugging Tools	Limited	Excellent	Minimal
Security Surface	Medium	Large	Small
Production Ready	Yes	Rarely	Yes

Part 7: When to Use What

Use Official NGINX Image When:

- Production deployment
- Standard web hosting
- Reverse proxy / load balancer

Use Ubuntu-Based Image When:

- Learning Linux + NGINX internals
- Heavy debugging needed
- Custom system-level dependencies

Use Alpine-Based Image When:

- Microservices
- CI/CD pipelines
- Cloud and Kubernetes workloads

Containerization-and-DevOps

Experiment 4: Docker Essentials

Dockerfile .dockerignore tagging publishing

Containerizing Applications with Dockerfile

Step-1: Create a Directory for Python application

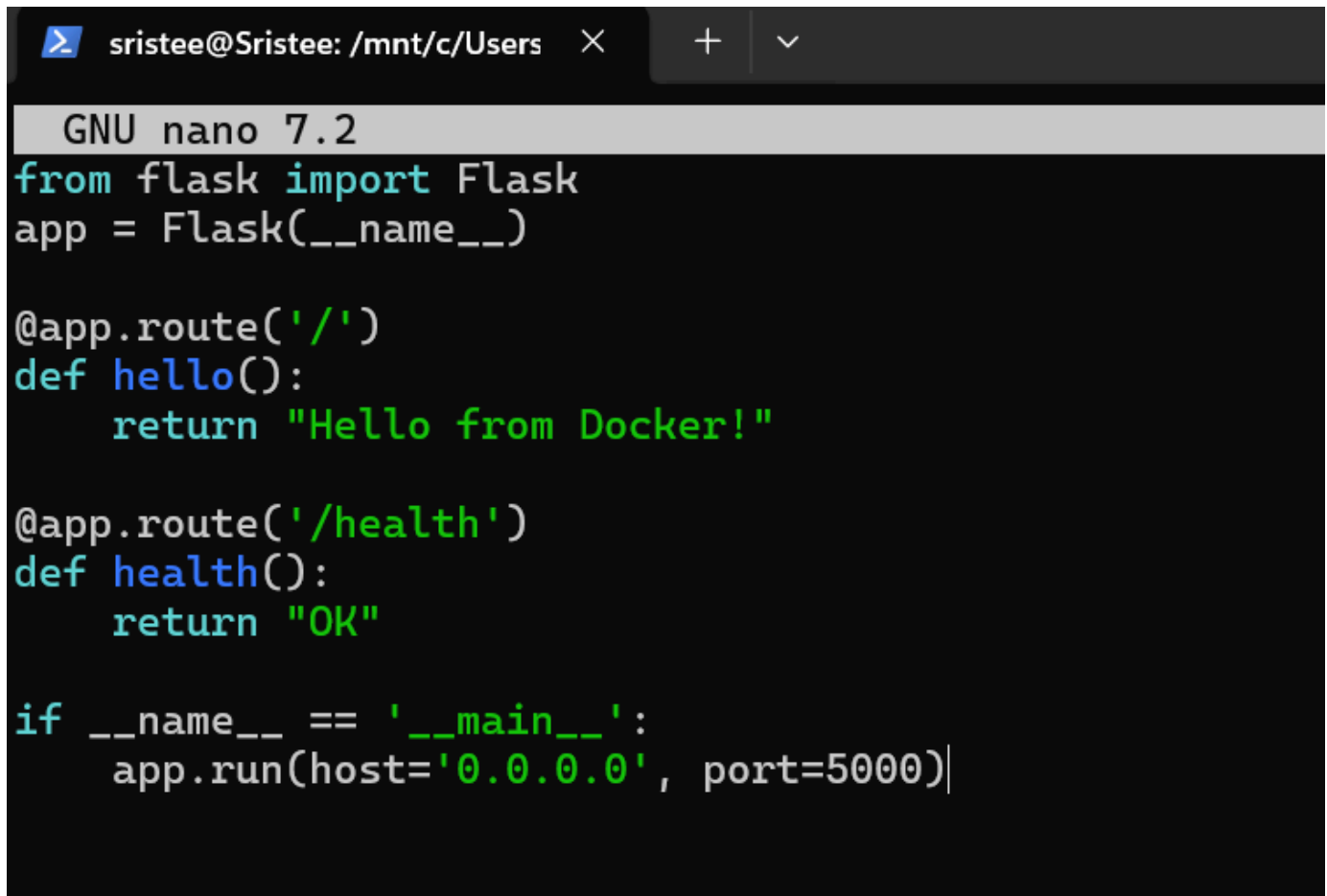
```
mkdir my-flask-app  
cd my-flask-app
```

```
sristee@Sristee:/mnt/c/Users/srist$ cd my-flask-app  
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ ls  
Dockerfile  app.py  requirements.txt
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ nano app.py  
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ nano requirements.txt  
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ nano Dockerfile  
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ nano .dockerignore
```

Step-2:- Create an `app.py`

```
from flask import Flask  
app = Flask(__name__)  
  
@app.route('/')  
def hello():  
    return "Hello from Docker!"  
  
@app.route('/health')  
def health():  
    return "OK"  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```

A terminal window with a dark background and light-colored text. The title bar shows the user 'sristee@Sristee' and the directory '/mnt/c/Users'. The terminal is running 'GNU nano 7.2'. The code being written is a Flask application. It imports Flask, creates an app, and defines two routes: a root route that returns 'Hello from Docker!' and a health route that returns 'OK'. The app is configured to run on host '0.0.0.0' at port 5000.

```
> sristee@Sristee: /mnt/c/Users × + v
GNU nano 7.2
from flask import Flask
app = Flask(__name__)

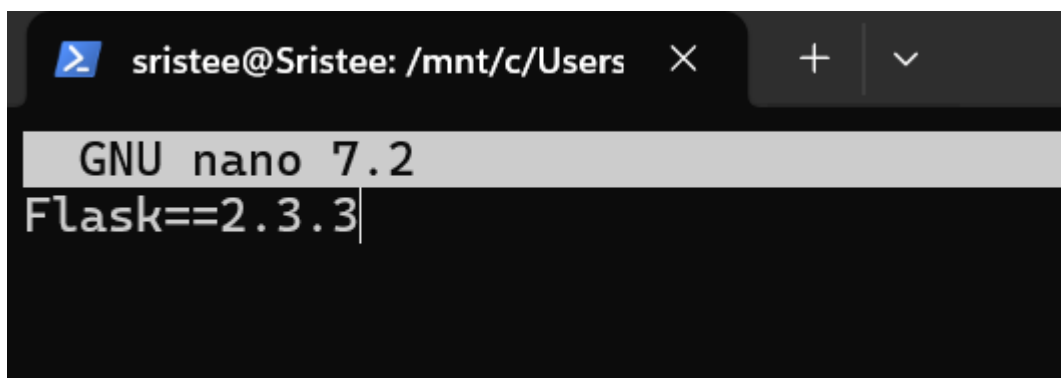
@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)|
```

Step-3:- Create **** requirements.txt ****

```
Flask==2.3.3
```

A terminal window with a dark background and light-colored text. The title bar shows the user 'sristee@Sristee' and the directory '/mnt/c/Users'. The terminal is running 'GNU nano 7.2'. The text 'Flask==2.3.3' is being entered into the file.

```
> sristee@Sristee: /mnt/c/Users × + v
GNU nano 7.2
Flask==2.3.3|
```

Step-4: Create ****Dockerfile****

```
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

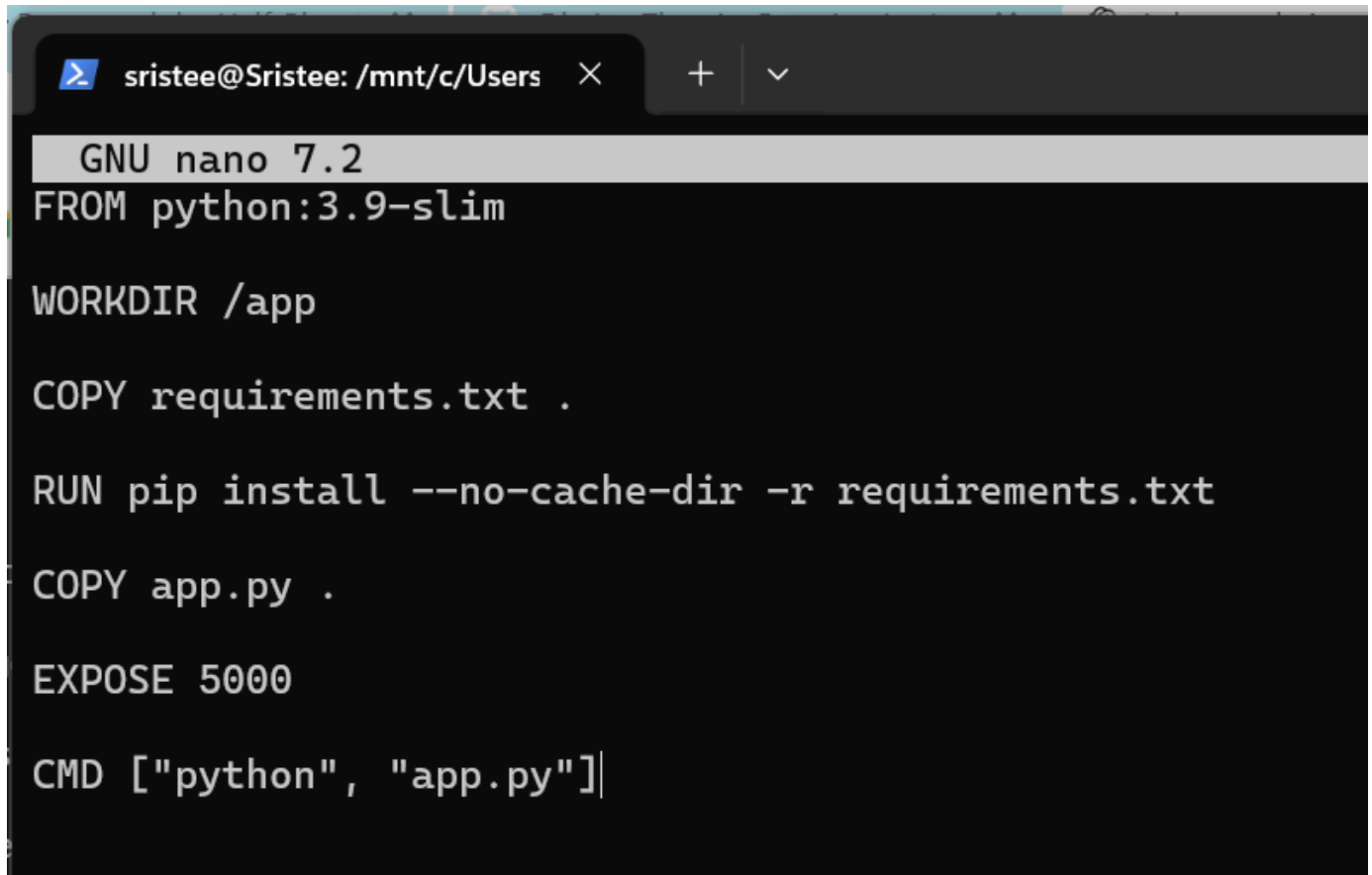
# Copy requirements file
COPY requirements.txt .
```

```
# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```

A screenshot of a terminal window with a dark background. The window title bar shows 'sristee@Sristee: /mnt/c/Users' and standard window controls. The terminal content shows the Dockerfile instructions: 'FROM python:3.9-slim', 'WORKDIR /app', 'COPY requirements.txt .', 'RUN pip install --no-cache-dir -r requirements.txt', 'COPY app.py .', 'EXPOSE 5000', and 'CMD ["python", "app.py"]'. The text is in a light-colored monospace font.

```
GNU nano 7.2
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

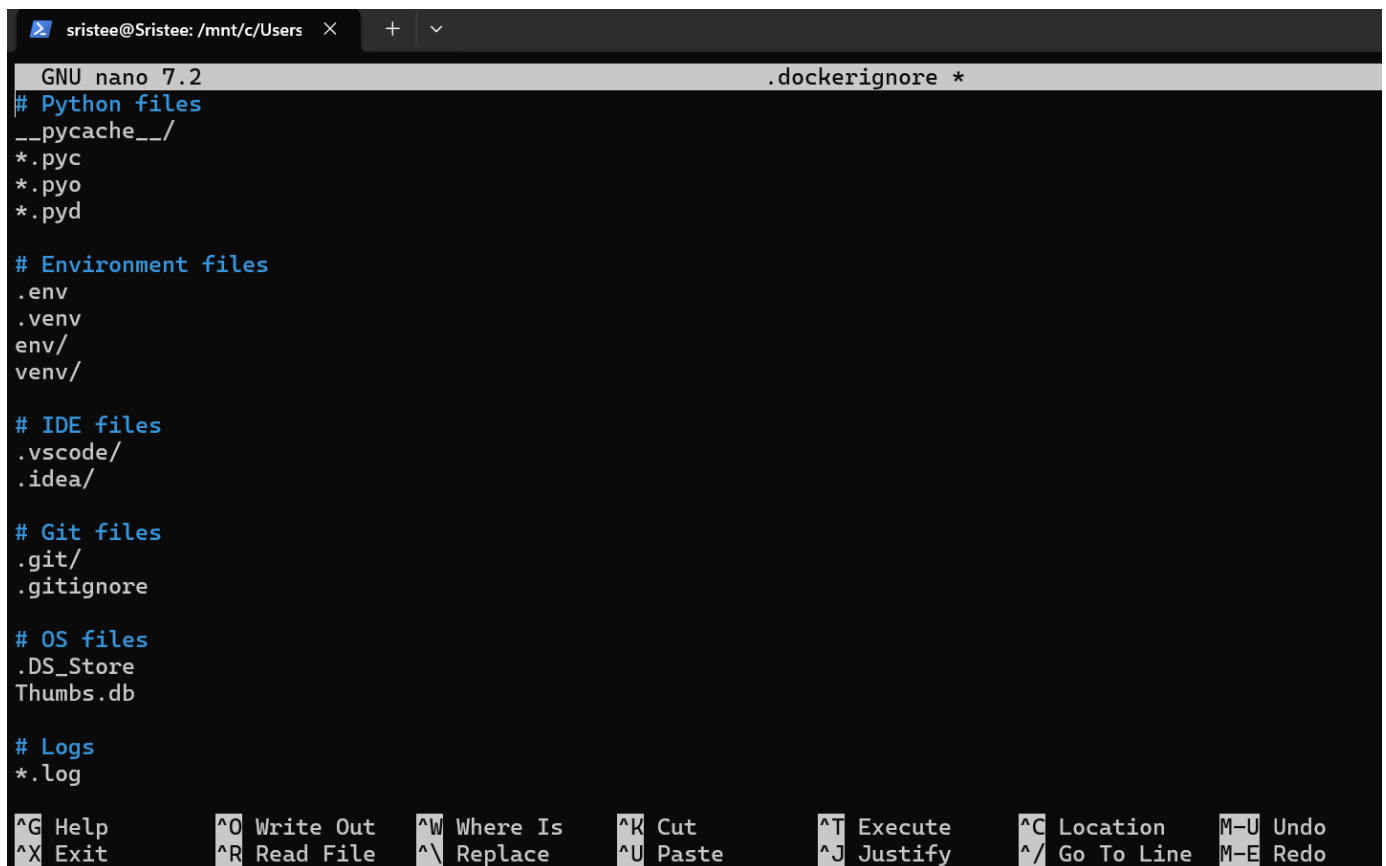
Step-5:- Create a **** .dockerignore ****

```
# Python files
__pycache__/
*.pyc
*.pyo
*.pyd

# Environment files
.env
.venv
env/
venv/

# IDE files
.vscode/
```

```
.idea/  
  
# Git files  
.git/  
.gitignore  
  
# OS files  
.DS_Store  
Thumbs.db  
  
# Logs  
*.log  
logs/  
  
# Test files  
tests/  
test_*.py
```



```
sristee@Sristee: /mnt/c/Users  GNU nano 7.2 .dockerignore *  
# Python files  
__pycache__/  
*.pyc  
*.pyo  
*.pyd  
  
# Environment files  
.env  
.venv  
env/  
venv/  
  
# IDE files  
.vscode/  
.idea/  
  
# Git files  
.git/  
.gitignore  
  
# OS files  
.DS_Store  
Thumbs.db  
  
# Logs  
*.log  
  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

Why `.dockerignore` is Important

- Prevents unnecessary files from being copied
- Reduces image size
- Improves build speed
- Increases security

Step-6: Build image from Dockerfile

```
docker build -t my-flask-app .
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker build -t my-flask-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   5.12kB
Step 1/7 : FROM python:3.9-slim
3.9-slim: Pulling from library/python
38513bd72563: Pulling fs layer
b3ec39b36ae8: Pulling fs layer
fc7443084902: Pulling fs layer
ea56f685404a: Pulling fs layer
ea56f685404a: Waiting
b3ec39b36ae8: Verifying Checksum
b3ec39b36ae8: Download complete
38513bd72563: Verifying Checksum
38513bd72563: Download complete
ea56f685404a: Download complete
fc7443084902: Verifying Checksum
fc7443084902: Download complete
38513bd72563: Pull complete
b3ec39b36ae8: Pull complete
fc7443084902: Pull complete
ea56f685404a: Pull complete
Digest: sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acd1e13cf1731b1b
Status: Downloaded newer image for python:3.9-slim
----> 085da638e1b8
Step 2/7 : WORKDIR /app
----> Running in f10833b0e561
----> Removed intermediate container f10833b0e561
----> 34dbeab1b1c3
Step 3/7 : COPY requirements.txt .
----> ed6570328474
```

Step-7:- Verify Image

```
docker images
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-flask-app	latest	a73aca0361f2	4 seconds ago	132MB
container-app_backend	latest	4f1284b14e38	10 days ago	136MB
<none>	<none>	0010e9735c33	10 days ago	136MB
container-app_database	latest	69a7cf92ffe4	10 days ago	274MB
node	20-alpine	6757e50d3b6b	2 weeks ago	136MB
postgres	15-alpine	cd848ee12e8e	4 weeks ago	274MB
nginx-alpine	latest	2adf2192b96a	5 weeks ago	10.4MB
nginx-ubuntu	latest	b80f01724123	5 weeks ago	134MB
nginx	latest	5cdef4ac3335	7 weeks ago	161MB
nginx	<none>	248d2326f351	7 weeks ago	161MB
java-app	1.0	11efb5c08422	7 weeks ago	773MB
sristee/java-app	1.0	11efb5c08422	7 weeks ago	773MB
<none>	<none>	8971fdfc5261	8 weeks ago	773MB
alpine	latest	a40c03cbb81c	8 weeks ago	8.44MB
myrepo/java-app	1.0	5d7e078d2f15	8 weeks ago	933MB
ubuntu	latest	493218ed0f40	2 months ago	78.1MB
ubuntu	22.04	65c77cbc27c2	2 months ago	77.9MB
python	3.9-slim	085da638e1b8	4 months ago	122MB
hello-world	latest	1b44b5a3e06a	7 months ago	10.1kB
ghcr.io/k3d-io/k3d-proxy	5.8.3	49c793b9faf6	13 months ago	63.1MB
ghcr.io/k3d-io/k3d-tools	5.8.3	8622faa0d552	13 months ago	20.7MB
rancher/k3s	v1.31.5-k3s1	efe65d76faac	14 months ago	222MB
busybox	latest	af3f0f48a24e	18 months ago	4.43MB

Step-8: Add Tag in Image

```
docker build -t my-flask-app:1.0 .
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker build -t my-flask-app:1.0 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   5.12kB
Step 1/7 : FROM python:3.9-slim
----> 085da638e1b8
Step 2/7 : WORKDIR /app
----> Using cache
----> 34dbeab1b1c3
Step 3/7 : COPY requirements.txt .
----> Using cache
----> ed6570328474
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
----> Using cache
----> 73d4b8e55f17
Step 5/7 : COPY app.py .
----> Using cache
----> 5b079ab9ea34
Step 6/7 : EXPOSE 5000
----> Using cache
----> 305ee7a62253
Step 7/7 : CMD ["python", "app.py"]
----> Using cache
----> a73aca0361f2
Successfully built a73aca0361f2
Successfully tagged my-flask-app:1.0
```

Step-9:- Add multiple tags

```
docker build -t my-flask-app:latest -t my-flask-app:1.0 .
```

```
sristee@sristee:/mnt/c/Users/srist/my-flask-app$ docker build -t my-flask-app:latest -t my-flask-app:1.0 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   5.12kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> 34dbeab1b1c3
Step 3/7 : COPY requirements.txt .
--> Using cache
--> ed6570328474
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> 73d4b8e55f17
Step 5/7 : COPY app.py .
--> Using cache
--> 5b079ab9ea34
Step 6/7 : EXPOSE 5000
--> Using cache
--> 305ee7a62253
Step 7/7 : CMD ["python", "app.py"]
--> Using cache
--> a73aca0361f2
Successfully built a73aca0361f2
Successfully tagged my-flask-app:latest
Successfully tagged my-flask-app:1.0
```

Step-10:- Add Tag with custom registry

```
docker build -t username/my-flask-app:1.0 .
```

```
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   5.12kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> 34dbeab1b1c3
Step 3/7 : COPY requirements.txt .
--> Using cache
--> ed6570328474
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> 73d4b8e55f17
Step 5/7 : COPY app.py .
--> Using cache
--> 5b079ab9ea34
Step 6/7 : EXPOSE 5000
--> Using cache
--> 305ee7a62253
Step 7/7 : CMD ["python", "app.py"]
--> Using cache
--> a73aca0361f2
Successfully built a73aca0361f2
Successfully tagged username/my-flask-app:1.0
```

Step-11:- Tag existing image

```
docker tag my-flask-app:latest my-flask-app:v1.0
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker tag my-flask-app:latest my-flask-app:v1.0
```

Step-12: View Image Details

```
docker images
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-flask-app	1.0	a73aca0361f2	About a minute ago	132MB
my-flask-app	latest	a73aca0361f2	About a minute ago	132MB
my-flask-app	v1.0	a73aca0361f2	About a minute ago	132MB
username/my-flask-app	1.0	a73aca0361f2	About a minute ago	132MB
container-app_backend	latest	4f1284b14e38	10 days ago	136MB
<none>	<none>	0010e9735c33	10 days ago	136MB
container-app_database	latest	69a7cf92ffe4	10 days ago	274MB
node	20-alpine	6757e50d3b6b	2 weeks ago	136MB
postgres	15-alpine	cd848ee12e8e	4 weeks ago	274MB
nginx-alpine	latest	2adf2192b96a	5 weeks ago	10.4MB
nginx-ubuntu	latest	b80f01724123	5 weeks ago	134MB
nginx	latest	5cdef4ac3335	7 weeks ago	161MB
nginx	<none>	248d2326f351	7 weeks ago	161MB
java-app	1.0	11efb5c08422	7 weeks ago	773MB
sristee/java-app	1.0	11efb5c08422	7 weeks ago	773MB
<none>	<none>	8971fd5c5261	8 weeks ago	773MB
alpine	latest	a40c03cbb81c	8 weeks ago	8.44MB
myrepo/java-app	1.0	5d7e078d2f15	8 weeks ago	933MB
ubuntu	latest	493218ed0f40	2 months ago	78.1MB
ubuntu	22.04	65c77cbc27c2	2 months ago	77.9MB
python	3.9-slim	085da638e1b8	4 months ago	122MB
hello-world	latest	1b44b5a3e06a	7 months ago	10.1kB
ghcr.io/k3d-io/k3d-proxy	5.8.3	49c793b9faf6	13 months ago	63.1MB
ghcr.io/k3d-io/k3d-tools	5.8.3	8622faa0d552	13 months ago	20.7MB
rancher/k3s	v1.31.5-k3s1	efe65d76faac	14 months ago	222MB
busybox	latest	af3f0f48a24e	18 months ago	4.43MB

Step-13:- Show image history

```
docker history my-flask-app
```

```

sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker history my-flask-app
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
a73aca0361f2   About a minute ago /bin/sh -c #(nop)  CMD ["python" "app.py"] 0B
305ee7a62253   About a minute ago /bin/sh -c #(nop)  EXPOSE 5000             0B
5b079ab9ea34   About a minute ago /bin/sh -c #(nop)  COPY file:444a3904a5c3d221... 228B
73d4b8e55f17   About a minute ago /bin/sh -c pip install --no-cache-dir -r req... 10.5MB
ed6570328474   2 minutes ago    /bin/sh -c #(nop)  COPY file:f095db6d544b2123... 13B
34dbeab1b1c3   2 minutes ago    /bin/sh -c #(nop)  WORKDIR /app            0B
085da638e1b8   4 months ago     CMD ["python3"]      0B      buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux; for src in idle3 p... 36B      buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux; savedAptMark="$(a... 39.5MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8... 0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PYTHON_VERSION=3.9.25 0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3... 0B        buildkit.dockerfile.v0
<missing>      4 months ago     RUN /bin/sh -c set -eux; apt-get update; a... 3.81MB    buildkit.dockerfile.v0
<missing>      4 months ago     ENV LANG=C.UTF-8      0B        buildkit.dockerfile.v0
<missing>      4 months ago     ENV PATH=/usr/local/bin:/usr/local/sbin:/usr... 0B        buildkit.dockerfile.v0
<missing>      5 months ago     # debian.sh --arch 'amd64' out/ 'trixie' '@1... 78.6MB    debuerreotype 0.16

```

Step-14:- Inspect image details

```
docker inspect my-flask-app
```

```

sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker inspect my-flask-app
[
  {
    "Id": "sha256:a73aca0361f2c5909bd2c3207ab6fe15a0271587f104765c031cc33a27532e87",
    "RepoTags": [
      "my-flask-app:1.0",
      "my-flask-app:latest",
      "my-flask-app:v1.0",
      "username/my-flask-app:1.0"
    ],
    "RepoDigests": [],
    "Parent": "sha256:305ee7a62253af0e5c05ba96fc4296ac72d86860f3992f1e3f0ae2611a1c19b6",
    "Comment": "",
    "Created": "2026-03-27T05:56:56.760150673Z",
    "DockerVersion": "28.2.2",
    "Author": "",
    "Config": {
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Env": [
        "PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568",
        "PYTHON_VERSION=3.9.25",
        "PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834"
      ],
      "Cmd": [
        "python",
        "app.py"
      ],
      "WorkingDir": "/app"
    }
  }
]

```

Step-15: Run Container

```
docker run -d -p 5000:5000 --name flask-container my-flask-app
```

```

sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker run -d -p 5000:5000 --name flask-container my-flask-app
24cf8139c3fe48bbdf90c05dfd39bca6b48f2c31da9f3b998d96bbf68085f3ec

```

Step-16:- Test the application

```
curl http://localhost:5000
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ curl http://localhost:5000
Hello from Docker!sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker ps
```

Step-17:- View running containers

```
docker ps
```

```
Hello from Docker!sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
24cf8139c3fe	my-flask-app		"python app.py"	13 seconds ago	Up 12 seconds	0.0.0.0:5000
0->5000/tcp, [::]:5000->5000/tcp		flask-container				
30562c803713	ghcr.io/k3d-io/k3d-proxy:5.8.3		"/bin/sh -c nginx-pr..."	9 days ago	Up 31 minutes	80/tcp, 0.0
.0.0:41349->6443/tcp		k3d-mycluster-serverlb				
6a25d6d965c4	rancher/k3s:v1.31.5-k3s1		"/bin/k3d-entrypoint..."	9 days ago	Up 31 minutes	
		k3d-mycluster-server-0				
61c715c02ee7	container-app_backend		"docker-entrypoint.s..."	10 days ago	Restarting (1) 27 seconds ago	
		node_backend				
fe002d7b644e	container-app_database		"docker-entrypoint.s..."	10 days ago	Up 31 minutes (healthy)	
		postgres_db				

Step-18:- View container logs

```
docker logs flask-container
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker logs flask-container
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [27/Mar/2026 05:59:10] "GET / HTTP/1.1" 200 -
```

Step-19:- Stop container

```
docker stop flask-container
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker stop flask-container
flask-container
```

Step-20:- Start stopped container

```
docker start flask-container
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker start flask-container
flask-container
```

Step-21:- Remove container

```
docker rm flask-container
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker rm flask-container
```

Step-22:- Remove container forcefully

Since Container is in Running State, so we will remove it forcefully...

```
docker rm -f flask-container
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker rm -f flask-container
flask-container
```

Multi-stage Builds

Why Multi-stage Builds?

- Smaller final image size
- Better security (remove build tools)
- Separate build and runtime environments

Step-23: Create a Simple **** Dockerfile.multistage **:***

```
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser
```

```
# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ nano Dockerfile.multistage
```

```
GNU nano 7.2 Dockerfile.multistage
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser
```

[Read 35 lines]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location	M
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^/ Go To Line	M

Step-24:- Build regular image

```
docker build -t flask-regular .
```

```

sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker build -t flask-regular .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  6.656kB
Step 1/7 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/7 : WORKDIR /app
--> Using cache
--> 34dbeab1b1c3
Step 3/7 : COPY requirements.txt .
--> Using cache
--> ed6570328474
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> 73d4b8e55f17
Step 5/7 : COPY app.py .
--> Using cache
--> 5b079ab9ea34
Step 6/7 : EXPOSE 5000
--> Using cache
--> 305ee7a62253
Step 7/7 : CMD ["python", "app.py"]
--> Using cache
--> a73aca0361f2
Successfully built a73aca0361f2
Successfully tagged flask-regular:latest

```

Step-25:- Build multi-stage image

```
docker build -f Dockerfile.multistage -t flask-multistage .
```

```

sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker build -f Dockerfile.multistage -t flask-multistage .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  6.656kB
Step 1/15 : FROM python:3.9-slim AS builder
--> 085da638e1b8
Step 2/15 : WORKDIR /app
--> Using cache
--> 34dbeab1b1c3
Step 3/15 : COPY requirements.txt .
--> Using cache
--> ed6570328474
Step 4/15 : RUN python -m venv /opt/venv
--> Running in 9e40f33c169f
--> Removed intermediate container 9e40f33c169f
--> 46571c38bb82
Step 5/15 : ENV PATH="/opt/venv/bin:$PATH"
--> Running in 0e77e7200f3b
--> Removed intermediate container 0e77e7200f3b
--> c3041c467389
Step 6/15 : RUN pip install --no-cache-dir -r requirements.txt
--> Running in c1a1f7618671
Collecting Flask==2.3.3
  Downloading flask-2.3.3-py3-none-any.whl (96 kB)
    _____ 96.1/96.1 kB 700.0 kB/s eta 0:00:00
Collecting click>=8.1.3
  Downloading click-8.1.8-py3-none-any.whl (98 kB)
    _____ 98.2/98.2 kB 1.7 MB/s eta 0:00:00
Collecting itsdangerous>=2.1.2
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting importlib-metadata>=3.6.0

```

Step-26:- Compare sizes

```
docker images | grep flask-
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker images | grep flask-
flask-multistage      latest      ef1b969daa37  16 seconds ago  145MB
my-flask-app          1.0        a73aca0361f2  10 minutes ago  132MB
my-flask-app          latest     a73aca0361f2  10 minutes ago  132MB
my-flask-app          v1.0      a73aca0361f2  10 minutes ago  132MB
username/my-flask-app 1.0        a73aca0361f2  10 minutes ago  132MB
flask-regular         latest     a73aca0361f2  10 minutes ago  132MB
```

Publishing to Docker Hub

Step-27:- Login to Docker Hub

```
docker login
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker login
Authenticating with existing credentials... [Username: sristee]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
```

Step-28:- Tag image for Docker Hub

```
docker tag my-flask-app:latest username/my-flask-app:1.0
docker tag my-flask-app:latest username/my-flask-app:latest
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker tag my-flask-app:latest username/my-flask-app:1.0
docker tag my-flask-app:latest username/my-flask-app:latest
```

Step-29:- Push to Docker Hub

```
docker push username/my-flask-app:1.0
```

```
sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker push sristee/my-flask-app:1.0
The push refers to repository [docker.io/sristee/my-flask-app]
f86e3273b98e: Pushed
79a6649c7642: Pushed
1540ffc5f3b7: Pushed
36125b464c36: Pushed
c8f6b54339a8: Pushed
298992e09a03: Pushed
4f237755fbae: Pushed
d7c97cb6f1fe: Pushed
1.0: digest: sha256:1a4606132226d0c4cb606942267b2b9c604b33d93840a6c4d897c0a18cb097cf size: 1990
```

Step-30:- Push Latest Image

```
docker push username/my-flask-app:latest
```

```

sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker push sristee/my-flask-app:latest
The push refers to repository [docker.io/sristee/my-flask-app]
f86e3273b98e: Layer already exists
79a6649c7642: Layer already exists
1540ffc5f3b7: Layer already exists
36125b464c36: Layer already exists
c8f6b54339a8: Layer already exists
298992e09a03: Layer already exists
4f237755fbae: Layer already exists
d7c97cb6f1fe: Layer already exists
latest: digest: sha256:1a4606132226d0c4cb606942267b2b9c604b33d93840a6c4d897c0a18cb097cf size: 1990

```

Step-31:- Pull from Docker Hub (on another machine)

```
docker pull username/my-flask-app:latest
```

```

sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker pull sristee/my-flask-app:latest
latest: Pulling from sristee/my-flask-app
Digest: sha256:1a4606132226d0c4cb606942267b2b9c604b33d93840a6c4d897c0a18cb097cf
Status: Image is up to date for sristee/my-flask-app:latest
docker.io/sristee/my-flask-app:latest

```

Step-32:- Run the pulled image

```
docker run -d -p 5000:5000 username/my-flask-app:latest
```

```

sristee@Sristee:/mnt/c/Users/srist/my-flask-app$ docker run -d -p 5000:5000 sristee/my-flask-app:latest
2aa865fd9ef8c3ad7e4fb3769662b2139c6bcd519b7c3137c05ff195687b9508

```

Node.js Application Example (Quick Version)**

Step-33:- Create a directory Node.js Application

```

mkdir my-node-app
cd my-node-app

```

```

sristee@Sristee:/mnt/c/Users/srist$ mkdir my-node-app
cd my-node-app

```

Step-34:- Create a ** app.js **

```

const express = require('express');
const app = express();
const port = 3000;

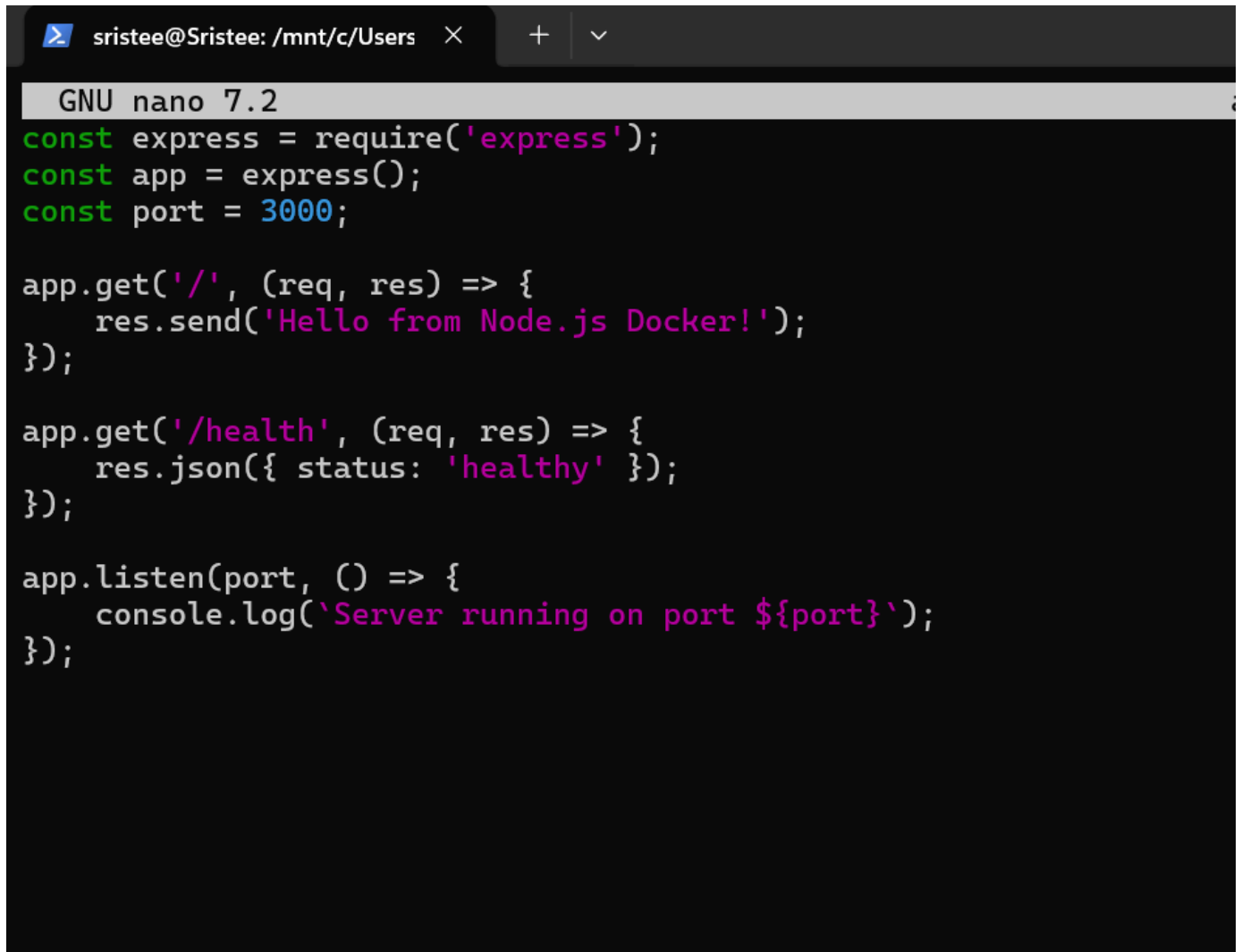
app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

```

```
});  
  
app.listen(port, () => {  
  console.log(`Server running on port ${port}`);  
});
```

```
sristee@Sristee:/mnt/c/Users/srist/my-node-app$ nano app.js
```

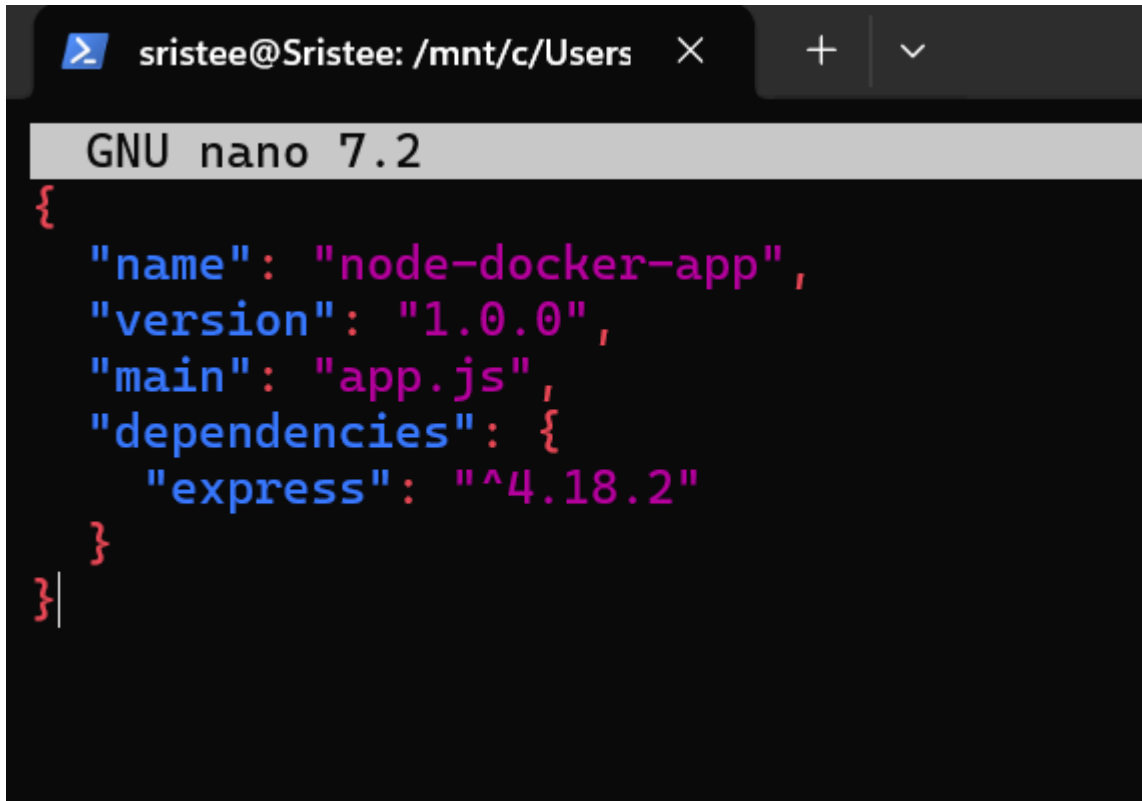


```
GNU nano 7.2  
const express = require('express');  
const app = express();  
const port = 3000;  
  
app.get('/', (req, res) => {  
  res.send('Hello from Node.js Docker!');  
});  
  
app.get('/health', (req, res) => {  
  res.json({ status: 'healthy' });  
});  
  
app.listen(port, () => {  
  console.log(`Server running on port ${port}`);  
});
```

Step-35:- Create a **package.json** :

```
{  
  "name": "node-docker-app",  
  "version": "1.0.0",  
  "main": "app.js",  
  "dependencies": {  
    "express": "^4.18.2"  
  }  
}
```

```
sristee@Sristee: /mnt/c/Users/srist/my-node-app$ nano package.json
```



```
GNU nano 7.2
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

Step-36:- Create a Node.js Dockerfile

```
FROM node:18-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

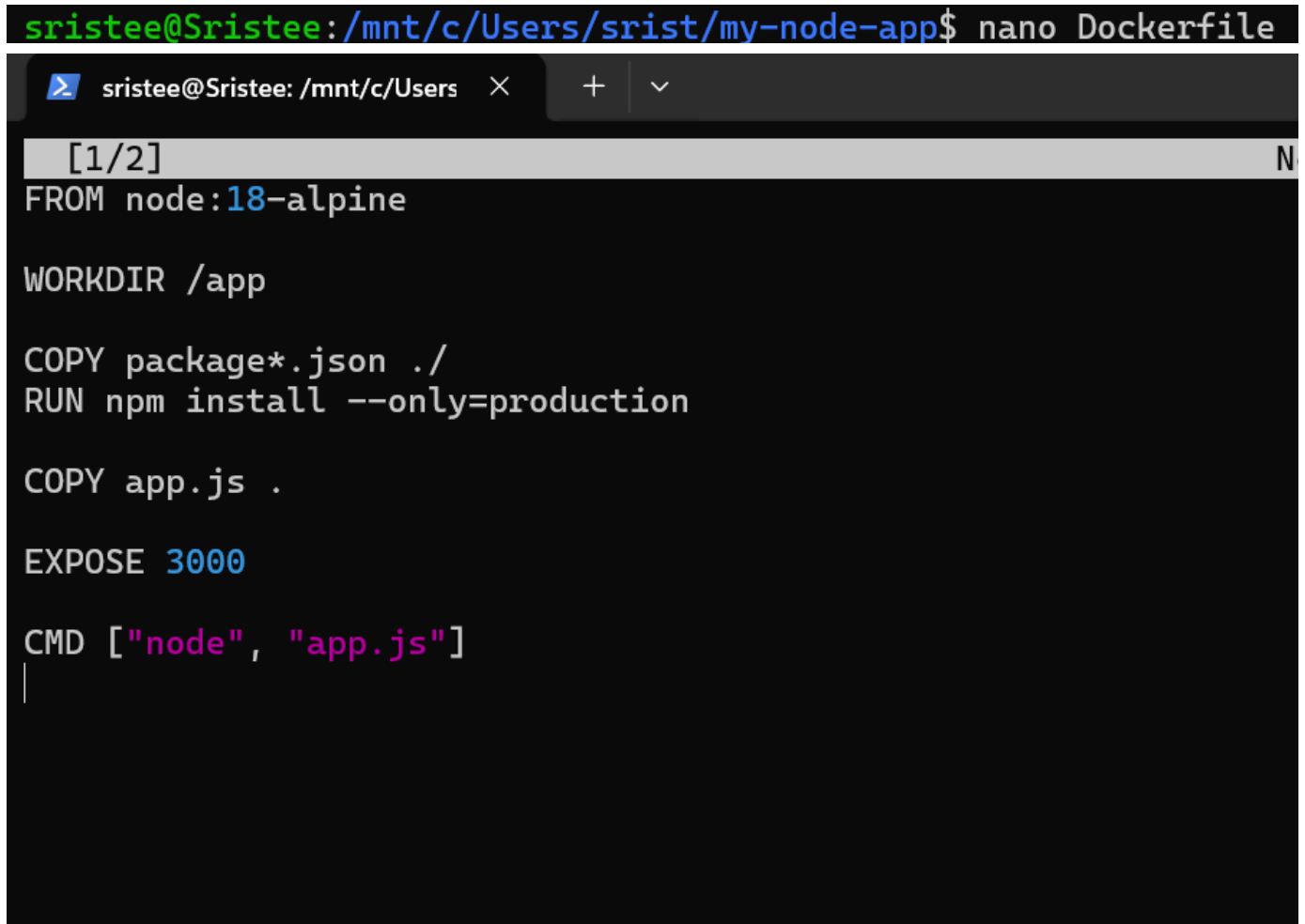
```
RUN npm install --only=production
```

```
COPY app.js .
```

```
EXPOSE 3000
```

```
CMD ["node", "app.js"]
```

```
sristee@Sristee:/mnt/c/Users/srist/my-node-app$ nano Dockerfile
```



```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --only=production

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

Step-37:- Build image

```
docker build -t my-node-app .
```

```

sristee@Sristee:/mnt/c/Users/srist/my-node-app$ docker build -t my-node-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   5.12kB
Step 1/7 : FROM node:18-alpine
18-alpine: Pulling from library/node
f18232174bc9: Pulling fs layer
dd71dde834b5: Pulling fs layer
1e5a4c89cee5: Pulling fs layer
25ff2da83641: Pulling fs layer
25ff2da83641: Waiting
1e5a4c89cee5: Verifying Checksum
1e5a4c89cee5: Download complete
f18232174bc9: Verifying Checksum
f18232174bc9: Download complete
f18232174bc9: Pull complete
25ff2da83641: Verifying Checksum
25ff2da83641: Download complete
dd71dde834b5: Verifying Checksum
dd71dde834b5: Pull complete
1e5a4c89cee5: Pull complete
25ff2da83641: Pull complete
Digest: sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
Status: Downloaded newer image for node:18-alpine
---> ee77c6cd7c18
Step 2/7 : WORKDIR /app
---> Running in bf32119f906e
---> Removed intermediate container bf32119f906e
---> 41c7f8b6d6fb8
Step 3/7 : COPY package*.json ./
---> 7113868d6da6

```

Step-38:- Run container

```
docker run -d -p 3000:3000 --name node-container my-node-app
```

```

Successfully tagged my-node-app:latest
sristee@Sristee:/mnt/c/Users/srist/my-node-app$ docker run -d -p 3000:3000 --name node-container my-node-app
f2cb352c9d8b12c29f69dfad12ac8278cce1f79d7fe4cf85806df98355eebafa

```

Step-39:- Test on Terminal

```
curl http://localhost:3000
```

```

sristee@Sristee:/mnt/c/Users/srist/my-node-app$ curl http://localhost:3000
Hello from Node.js Docker!sristee@Sristee:/mnt/c/Users/srist/my-node-app$ |

```

Conclusion

Essential Docker Commands Cheatsheet

Command	Purpose	Example
<code>docker build</code>	Build image	<code>docker build -t myapp .</code>
<code>docker run</code>	Run container	<code>docker run -p 3000:3000 myapp</code>
<code>docker ps</code>	List containers	<code>docker ps -a</code>
<code>docker images</code>	List images	<code>docker images</code>
<code>docker tag</code>	Tag image	<code>docker tag myapp:latest myapp:v1</code>
<code>docker login</code>	Login to Dockerhub using username and password or token	<code>echo "token" docker login -u username --password-stdin</code>
<code>docker push</code>	Push to registry	<code>docker push username/myapp</code>
<code>docker pull</code>	Pull from registry	<code>docker pull username/myapp</code>
<code>docker rm</code>	Remove container	<code>docker rm container-name</code>
<code>docker rmi</code>	Remove image	<code>docker rmi image-name</code>
<code>docker logs</code>	View logs	<code>docker logs container-name</code>
<code>docker exec</code>	Execute command	<code>docker exec -it container-name bash</code>

Common Workflow Summary

Development Workflow:

1. Create Dockerfile and .dockerignore

2. Build image

```
docker build -t myapp .
```

3. Test locally

```
docker run -p 8080:8080 myapp
```

4. Tag for production

```
docker tag myapp:latest myapp:v1.0
```

5. Push to registry

```
docker push myapp:v1.0
```

Production Workflow:

1. Pull from registry

```
docker pull myapp:v1.0
```

2. Run in production

```
docker run -d -p 80:8080 --name prod-app myapp:v1.0
```

3. Monitor

```
docker logs -f prod-app
```

Key Takeaways

1. **Dockerfile** defines how to build your image
2. **.dockerignore** excludes unnecessary files (important for security and performance)
3. **Tagging** helps version and organize images
4. **Multi-stage builds** create smaller, more secure production images
5. **Docker Hub** allows sharing and distributing images
6. **Always test** images locally before publishing

Cleanup

Remove all stopped containers

```
docker container prune
```

Remove unused images

```
docker image prune
```

Remove everything unused

```
docker system prune -a
```

[← Previous](#) [🏠 Home](#)

Containerization-and-DevOps

Experiment 5:- Docker - Volumes, Environment Variables, Monitoring & Networks

Part 1: Docker Volumes - Persistent Data Storage

Lab 1: Understanding Data Persistence

The Problem: Container Data is Ephemeral

```
docker run -it --name test-container ubuntu /bin/bash
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker run -it --name test-container ubuntu /bin/bash
```

```
echo "Hello World" > /data/message.txt  
cat /data/message.txt  
exit
```

```
root@d6e2d5c45aa2:/# mkdir /data  
echo "Hello World" > /data/message.txt  
cat /data/message.txt  
exit  
Hello World  
exit
```

```
docker start test-container  
docker exec test-container cat /data/message.txt
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker start test-container  
docker exec test-container cat /data/message.txt  
test-container  
Hello World
```

Solution: Docker Volumes

Lab 2: Volume Types

Anonymous Volume

```
docker run -d -v /app/data --name web1 nginx
docker volume ls
docker inspect web1
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker run -d -v /app/data --name web1 nginx
docker volume ls
docker inspect web1
d2980e79576e91a2faa4b343352ea23dd7da3a584b575f9d9c86215458649f9c
DRIVER      VOLUME NAME
local       1b6cc06836fe0de91e5453b606460a40ed3ff1ed2171e46bf8dcbcdcb45190217
local       35e0ad5bdfc326fc63153ca154c54cfb70b3b012674c3555676ca6f6b5f8299d
local       6573f937d22e216819ed0b2d87729eee10be67aa2243c63505eee740d1d37709
local       093875a53b50092f27a5578b887a478f92bfccf920791350349e4af17166b34a
local       2356594cf441c7d71bd0ce6e3ddb9f90931a2b3dc31fc93c02002ff64c7a6817b
local       container-app_pgdata
local       d58d4af4c7ec391077aa6bf7a1e0f8ef27cf8a0dad4fffc8277dd681cf8314b4
local       d1011413afe899081ffe520e23a770b7055acfe2d955e41b31c9a41afa90f631
local       dcaef17b4e3417bb72e652fbeddcc8c3faf13a62ac2541403a40d51c21e0f49
local       k3d-mycluster-images
local       myvolume
[
  {
    "Id": "d2980e79576e91a2faa4b343352ea23dd7da3a584b575f9d9c86215458649f9c",
    "Created": "2026-04-23T17:17:56.740360434Z",
    "Path": "/docker-entrypoint.sh",
    "Args": [
      "nginx",
      "-g",
      "daemon off;"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
```

Named Volume

```
docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
docker volume ls
docker volume inspect mydata
```

```

sristee@Sristee:/mnt/c/Users/srist$ docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
docker volume ls
docker volume inspect mydata
mydata
80db7a2ef486ad5371d4b967a5260d1b1c5db2d60b6d2918c99aae0512955666
DRIVER      VOLUME NAME
local       1b6cc06836fe0de91e5453b606460a40ed3ff1ed2171e46bf8dcbcd45190217
local       35e0ad5bdfc326fc63153ca154c54cfb70b3b012674c3555676ca6f6b5f8299d
local       6573f937d22e216819ed0b2d87729eee10be67aa2243c63505eee740d1d37709
local       093875a53b50092f27a5578b887a478f92bfccf920791350349e4af17166b34a
local       2356594cf441c7d71bd0ce6e3ddb90931a2b3dc31fc93c02002ff64c7a6817b
local       container-app_pgdata
local       d58d4af4c7ec391077aa6bf7a1e0f8ef27cf8a0dad4fffc8277dd681cf8314b4
local       d1011413afe899081ffe520e23a770b7055acfe2d955e41b31c9a41afa90f631
local       dcaef17b4e3417bb72e652fbeddcc8c3faf13a62ac2541403a40d51c21e0f49
local       k3d-mycluster-images
local       mydata
local       myvolume
[
  {
    "CreatedAt": "2026-04-23T17:18:02Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
    "Name": "mydata",
    "Options": null,
    "Scope": "local"
  }
]

```

Bind Mount

```

mkdir ~/myapp-data
docker run -d -v ~/myapp-data:/app/data --name web3 nginx
echo "From Host" > ~/myapp-data/host-file.txt
docker exec web3 cat /app/data/host-file.txt

```

```

sristee@Sristee:/mnt/c/Users/srist$ mkdir ~/myapp-data
docker run -d -v ~/myapp-data:/app/data --name web3 nginx

echo "From Host" > ~/myapp-data/host-file.txt
docker exec web3 cat /app/data/host-file.txt
72b8dc2c588001a05a4127fd693ea545da57e8e5dbb593fd7f7be15d10bd85fe1
From Host

```

Lab 3: Practical Examples

MySQL Persistent Storage

```
docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker run -d --name mysql-db -v mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=secret
mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
bb5107df7baa: Pull complete
62f1cc0a64ca: Pull complete
4f7cfd23b111: Pull complete
c689d1836928: Pull complete
caa784df8611: Pull complete
fe997c72f3e9: Pull complete
38fe0e0986de: Pull complete
09b552d72653: Pull complete
a0db31010df3: Pull complete
b723faee7585: Pull complete
139627f60883: Pull complete
Digest: sha256:d0304ed9fdb64a3f6c7ad11a5fb4f13abfc10e6dfa3f288d652e7320c34df7f9
Status: Downloaded newer image for mysql:8.0
93b53f7337441f99ccb25dffb58547ef0548a6cded7d3ec5f48b2336a5f386b5
```

```
docker stop mysql-db
docker rm mysql-db
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker stop mysql-db
docker rm mysql-db
mysql-db
mysql-db
```

```
docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
845966cdd965c995193adfeec337466549a3f7eea7b1c2c38088a68de311829b
```

NGINX Config Mount

```
mkdir ~/nginx-config
```

```
echo 'server {
  listen 80;
  location / {
```

```

        return 200 "Hello from mounted config!";
    }
}' > ~/nginx-config/nginx.conf

```

```

sristee@Sristee:/mnt/c/Users/srist$ mkdir ~/nginx-config
sristee@Sristee:/mnt/c/Users/srist$ echo 'server {
    listen 80;
    location / {
        return 200 "Hello from mounted config!";
    }
}' > ~/nginx-config/nginx.conf

```

```

docker run -d \
  --name nginx-custom \
  -p 8080:80 \
  -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
  nginx

```

```

sristee@Sristee:/mnt/c/Users/srist$ docker run -d \
  --name nginx-custom \
  -p 8080:80 \
  -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
  nginx
014d582d7b2a0b95beb02b9d25d02c989b4b631bc23bd2e34d0c6e12a0ef23fe

```

```
curl http://localhost:8080
```

```

sristee@Sristee:/mnt/c/Users/srist$ curl http://localhost:8080
Hello from mounted config!sristee@Sristee:/mnt/c/Users/srist$

```

Lab 4: Volume Commands

```
docker volume ls
```

```
^Csristee@Srirstee:/mnt/c/Users/srist$ docker volume ls
DRIVER      VOLUME NAME
local       1b6cc06836fe0de91e5453b606460a40ed3ff1ed2171e46bf8dcbcd45190217
local       35e0ad5bdfc326fc63153ca154c54cfb70b3b012674c3555676ca6f6b5f8299d
local       6573f937d22e216819ed0b2d87729eee10be67aa2243c63505eee740d1d37709
local       093875a53b50092f27a5578b887a478f92bfccf920791350349e4af17166b34a
local       2356594cf441c7d71bd0ce6e3ddb90931a2b3dc31fc93c02002ff64c7a6817b
local       container-app_pgdata
local       d58d4af4c7ec391077aa6bf7a1e0f8ef27cf8a0dad4fffc8277dd681cf8314b4
local       d1011413afe899081ffe520e23a770b7055acfe2d955e41b31c9a41afa90f631
local       dcaef17b4e3417bb72e652fbeddcc8c3faf13a62ac2541403a40d51c21e0f49
local       k3d-mycluster-images
local       mydata
local       mysql-data
local       myvolume
```

```
docker volume create app-volume
```

```
srirstee@Srirstee:/mnt/c/Users/srist$ docker volume create app-volume
app-volume
```

```
docker volume inspect app-volume
```

```
srirstee@Srirstee:/mnt/c/Users/srist$ docker volume inspect app-volume
[
  {
    "CreatedAt": "2026-04-23T19:21:41Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
    "Name": "app-volume",
    "Options": null,
    "Scope": "local"
  }
]
```

```
docker volume prune
```

```
srirstee@Srirstee:/mnt/c/Users/srist$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
```

```
docker volume rm volume-name
```

```
srirstee@Srirstee:/mnt/c/Users/srist$ docker volume rm app-volume
app-volume
```

```
docker cp local-file.txt container-name:/path/in/volume
```

```
Successfully copied 2.05kB to test-container:/data
```

Part 2: Environment Variables

Setting Variables

```
docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  my-node-app
```

```
srstee@srstee:/mnt/c/Users/srist$ docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  nginx
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
2d83921cb578	clever_colden	--	-- / --	--	--	--	--
efe79368a59d	elastic_nobel	--	-- / --	--	--	--	--
398899cd2568	app1	--	-- / --	--	--	--	--
014d582d7b2a	nginx-custom	--	-- / --	--	--	--	--
845966cdd965	new-mysql	--	-- / --	--	--	--	--
72b8dc2c5880	web3	--	-- / --	--	--	--	--
80db7a2ef486	web2	--	-- / --	--	--	--	--
d2980e79576e	web1	--	-- / --	--	--	--	--
d6e2d5c45aa2	test-container	--	-- / --	--	--	--	--
30562c803713	k3d-mycluster-serverlb	--	-- / --	--	--	--	--
6a25d6d965c4	k3d-mycluster-server-0	--	-- / --	--	--	--	--
61c715c02ee7	node_backend	--	-- / --	--	--	--	--
fe002d7b644e	postgres_db	--	-- / --	--	--	--	--

```
^C
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
--	--	--	-- / --	--	--	--	--
--	--	--	-- / --	--	--	--	--
--	--	--	-- / --	--	--	--	--
--	--	--	-- / --	--	--	--	--
--	--	--	-- / --	--	--	--	--
--	--	--	-- / --	--	--	--	--
--	--	--	-- / --	--	--	--	--
--	--	--	-- / --	--	--	--	--
--	--	--	-- / --	--	--	--	--
--	--	--	-- / --	--	--	--	--
61c715c02ee7	node_backend	0.00%	0B / 0B	0.00%	0B / 0B	0B / 0B	0

Using .env File

```
echo "DATABASE_HOST=localhost" > .env
echo "API_KEY=secret123" >> .env
```

```
docker run -d --env-file .env my-app
```

Dockerfile Example

```
ENV NODE_ENV=production
ENV PORT=3000
```

Flask Example

```
import os
from flask import Flask

app = Flask(__name__)

@app.route('/config')
def config():
    return {
        "db": os.environ.get("DATABASE_HOST"),
        "debug": os.environ.get("DEBUG")
    }

app.run(host="0.0.0.0", port=5000)
```

Test Variables

```
docker run -d \
  --name flask-app \
  -p 5000:5000 \
  -e DEBUG=true \
  flask-app

docker exec flask-app env
curl http://localhost:5000/config
```

Part 3: Docker Monitoring

```
docker stats
docker stats --no-stream
```

```
docker top container-name

docker logs -f container-name

docker inspect container-name

docker events
```

Simple Monitoring Script

```
#!/bin/bash
docker ps
docker stats --no-stream
docker system df
```

Part 4: Docker Networks

List Networks

```
docker network ls
```

Bridge Network

```
docker network create my-network

docker run -d --name web1 --network my-network nginx
docker run -d --name web2 --network my-network nginx

docker exec web1 curl http://web2
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker network create my-networkkrk
fd41f21543608733ffffef8aeb3180758c8b602e5b6e5e4052740436b2a67326b

sristee@Sristee:/mnt/c/Users/srist$ docker run -d --name web1 --network my-network nginx
docker run -d --name web2 --network my-network nginx
docker: Error response from daemon: Conflict. The container name "/web1" is already in use by container "d2980e79576e91a2faa4b343352e
a23dd7da3a584b575f9d9c86215458649f9c". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
docker: Error response from daemon: Conflict. The container name "/web2" is already in use by container "80db7a2ef486ad5371d4b967a526
0d1b1c5db2d60b6d2918c99aae0512955666". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
```

Host Network

```
sristee@Sristee:/mnt/c/Users/srist$ docker run -d --network host nginx
384d73e7f5abc5a91d1846c0d887751dadd829dcd20245fc5bb8e918f13dd036
```

None Network

```
sristee@Sristee:/mnt/c/Users/srist$ docker run -d --network none alpine sleep 3600
01ba8f0421139dbfc9a4eeb807603f36a3402f3c9776d4bffb74028efad3ba69
```

Network Commands

Multi-Container Example

```
docker run -d \
  --name web-app \
  --network app-network \
  -p 8080:3000 \
  -e DATABASE_HOST="postgres-db" \
  node-app
```

```

sristee@Sristee:/mnt/c/Users/srist$ docker network create app-network

docker run -d \
  --name postgres-db \
  --network app-network \
  -e POSTGRES_PASSWORD=secret \
  postgres:15

docker run -d \
  --name web-app \
  --network app-network \
  -p 8080:3000 \
  -e DATABASE_HOST="postgres-db" \
  nginx
e22429c3efbe0dad5d98d352a33bed8c80f8b745cc130141151082d63262ff43
Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
3531af2bc2a9: Pull complete
be67b9cdfb60: Pull complete
c375454b4ff0: Pull complete
ff8482cd03b7: Pull complete
ac28d232c7f3: Pull complete
fa77b44f96d2: Pull complete
dd1cc88a000a: Pull complete
58e6420a0782: Pull complete
4c7f2d7f4f80: Pull complete
3e0c43622abe: Pull complete
0892560966bb: Pull complete
9fa019823549: Pull complete
00da1877cea6: Pull complete
26803cb302a4: Pull complete
Digest: sha256:29342cb52157b098821961d2c14eec3c019071f56a5d559e990cf07cf541ea9b
Status: Downloaded newer image for postgres:15

```

Debugging

```

docker network inspect bridge
docker exec container-name ping google.com
docker port container-name

```

Part 5: Real-World Example

```
docker network create myapp-network
```

```

docker run -d \
  --name postgres \
  --network myapp-network \
  -e POSTGRES_PASSWORD=pass \
  postgres:15

```

```
docker run -d \
```

```
--name redis \  
--network myapp-network \  
redis:7-alpine
```

```
docker run -d \  
--name flask-app \  
--network myapp-network \  
-p 5000:5000 \  
flask-app:latest
```

Cheatsheet

```
docker volume create <name>  
docker run -v <volume>:/path
```

```
docker run -e VAR=value
```

```
docker stats  
docker logs -f <container>
```

```
docker network create <name>
```

Practice

- Create DB with volume
- Multi-service setup
- Analyze logs
- Network isolation

Cleanup

```
docker stop $(docker ps -aq)  
docker rm $(docker ps -aq)  
docker volume prune -f  
docker network prune -f  
docker image prune -f
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker stop $(docker ps -aq)
docker rm $(docker ps -aq)
docker volume prune -f
docker network prune -f
docker image prune -f
247f7cc41269
906fcc363eef
01ba8f042113
384d73e7f5ab
2d83921cb578
efe79368a59d
398899cd2568
014d582d7b2a
845966cdd965
72b8dc2c5880
80db7a2ef486
d2980e79576e
d6e2d5c45aa2
f2cb352c9d8b
2aa865fd9ef8
30562c803713
6a25d6d965c4
61c715c02ee7
fe002d7b644e
344384ffe887
735781f95690
eb521404c058
ea4aff8a2945
f9428a8d1a33
be9109e12c21
74abf512a92f
2f029def7772
f83fe34b7d69
```

```
deleted: sha256:07b1d12e9b2b1a8558bfffedebde0cb5745674fbf473138e02f0b2978eb6427b1
deleted: sha256:fcd2cd5e03dc217b7b1159b4bbb218a3e8f02e111308df2682230919c986f8d5
deleted: sha256:9e2ef2ea1e9f94cdd84999c6ba81e6dedf6e2f8fb4dca1e83d167727e437ec2d
deleted: sha256:12c9492b1dd99cbac4c877c8c351e07dca9418a8555ace353f81ead3bc259b02
deleted: sha256:c97346864f26b304464bcdab9a1cda52ad5d88eec4039f0264cc12088f83bb44
deleted: sha256:316fa65fb6cbf9ee17f01a0cd324bf54b96ac6df5240e77093d743f6885fb7ac
```

Total reclaimed space: 108.3MB

Key Takeaways

- Volumes store persistent data
- Env variables control config
- Monitoring helps debugging
- Networks connect containers
- Use .env for security

Containerization-and-DevOps

Experiment 6: Comparison of Docker Run and Docker Compose

Introduction

Docker Run (Imperative Approach)

The `docker run` command is used to create and start a container from an image. It requires explicit flags for:

- Port mapping (`-p`)
- Volume mounting (`-v`)
- Environment variables (`-e`)
- Network configuration (`--network`)
- Restart policies (`--restart`)
- Resource limits (`--memory` , `--cpus`)
- Container name (`--name`)

This approach is **imperative**, meaning you specify step-by-step instructions.

Docker Compose (Declarative Approach)

Docker Compose uses a YAML file (`docker-compose.yml`) to define services, networks, and volumes in a structured format.

Instead of multiple `docker run` commands, a single command is used:

```
docker compose up -d
```

Compose is **declarative**, meaning you define the desired state of the application.

Mapping: Docker Run vs Docker Compose

Docker Run Flag	Docker Compose Equivalent
<code>-p 8080:80</code>	<code>ports:</code>

Docker Run Flag	Docker Compose Equivalent
-v host:container	volumes:
-e KEY=value	environment:
--name	container_name:
--network	networks:
--restart	restart:
--memory	deploy.resources.limits.memory
--cpus	deploy.resources.limits.cpus
-d	docker compose up -d

Advantages of Docker Compose

1. Simplifies multi-container applications
2. Provides reproducibility
3. Version controllable configuration
4. Unified lifecycle management
5. Supports service scaling

Hands-On

Step-1:- Run nginx Container

```
docker run -d \
--name lab-nginx \
-p 8081:80 \
-v $(pwd)/html:/usr/share/nginx/html \
nginx:alpine
```

```
docker ps
```

```
sristee@sristee:/mnt/c/Users/srist$ docker run -d \
--name lab-nginx \
-p 8081:80 \
-v $(pwd)/html:/usr/share/nginx/html \
nginx:alpine

docker ps
f3a50d380f484f48fee7f92118e5cdfc2dc26e54c35609e658d487c32186215a
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS
f3a50d380f48   nginx:alpine   "/docker-entrypoint...." Less than a second ago Up Less than a second   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp, lab-nginx
```

Step-2:- Create a file index.html mounting on a container

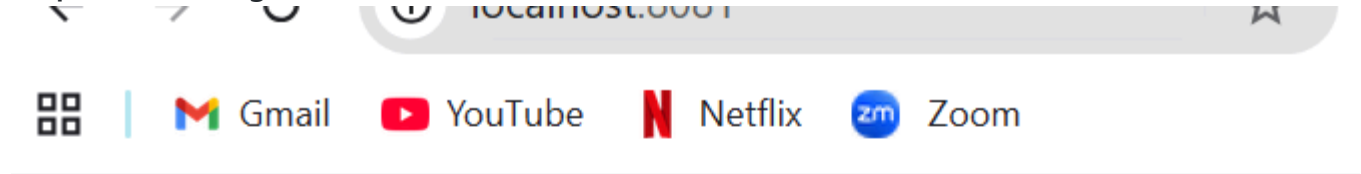
```
nano html/index.html
cat html/index.html
docker stop lab-nginx
docker rm lab-nginx
docker run -d \
--name lab-nginx \
-p 8081:80 \
-v $(pwd)/html:/usr/share/nginx/html \
nginx:alpine
```

index.html :-

```
<h1>Experiment 6 Working</h1>
<p>Nginx is running successfully</p>
```

```
sristee@Sristee:/mnt/c/Users/srist$ nano html/index.html
cat html/index.html
docker stop lab-nginx
docker rm lab-nginx
docker run -d \
--name lab-nginx \
-p 8081:80 \
-v $(pwd)/html:/usr/share/nginx/html \
nginx:alpine
<h1>Experiment 6 Working</h1>
<p>Nginx is running successfully</p>
lab-nginx
lab-nginx
7999e35ee53d128e1ef2de75fd5b5e2c2b7af0714761a9c67529899c38bdc931
```

Step-3:- View Page on Browser



Experiment 6 Working

Nginx is running successfully

Step-4:- Create same via compose

```
nano docker-compose.yml
cat docker-compose.yml
```

docker-compose.yml :-

```
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

```
sristee@Sristee:/mnt/c/Users/srist$ nano docker-compose.yml
cat docker-compose.yml
version: '3.8'

services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports:
      - "8081:80"
    volumes:
      - ./html:/usr/share/nginx/html
```

Step-5:- Create Containers via Compose

```
docker-compose up -d
docker-compose ps
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker-compose up -d
WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All containers will be scheduled on the current node
.

To deploy your application across the swarm, use 'docker stack deploy'.

Creating lab-nginx ... done
sristee@Sristee:/mnt/c/Users/srist$ docker-compose ps
```

Name	Command	State	Ports
lab-nginx	/docker-entrypoint.sh nginx ...	Up	0.0.0.0:8081->80/tcp,:::8081->80/tcp

Step-6:- Delete Compose Services

```
docker-compose down
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker-compose down
Stopping lab-nginx ... done
Removing lab-nginx ... done
Removing network srist_default
```

Step-7:- Create Network

```
docker network create wp-net
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker network create wp-net
9a8a10e4a0d086596f51a5fc1ef19e43b8f5f9005437f2465f61e208adef96cd
```

Step-8:- Run mysql container

```
docker run -d \  
--name mysql \  
--network wp-net \  
-e MYSQL_ROOT_PASSWORD=secret \  
-e MYSQL_DATABASE=wordpress \  
mysql:5.7
```

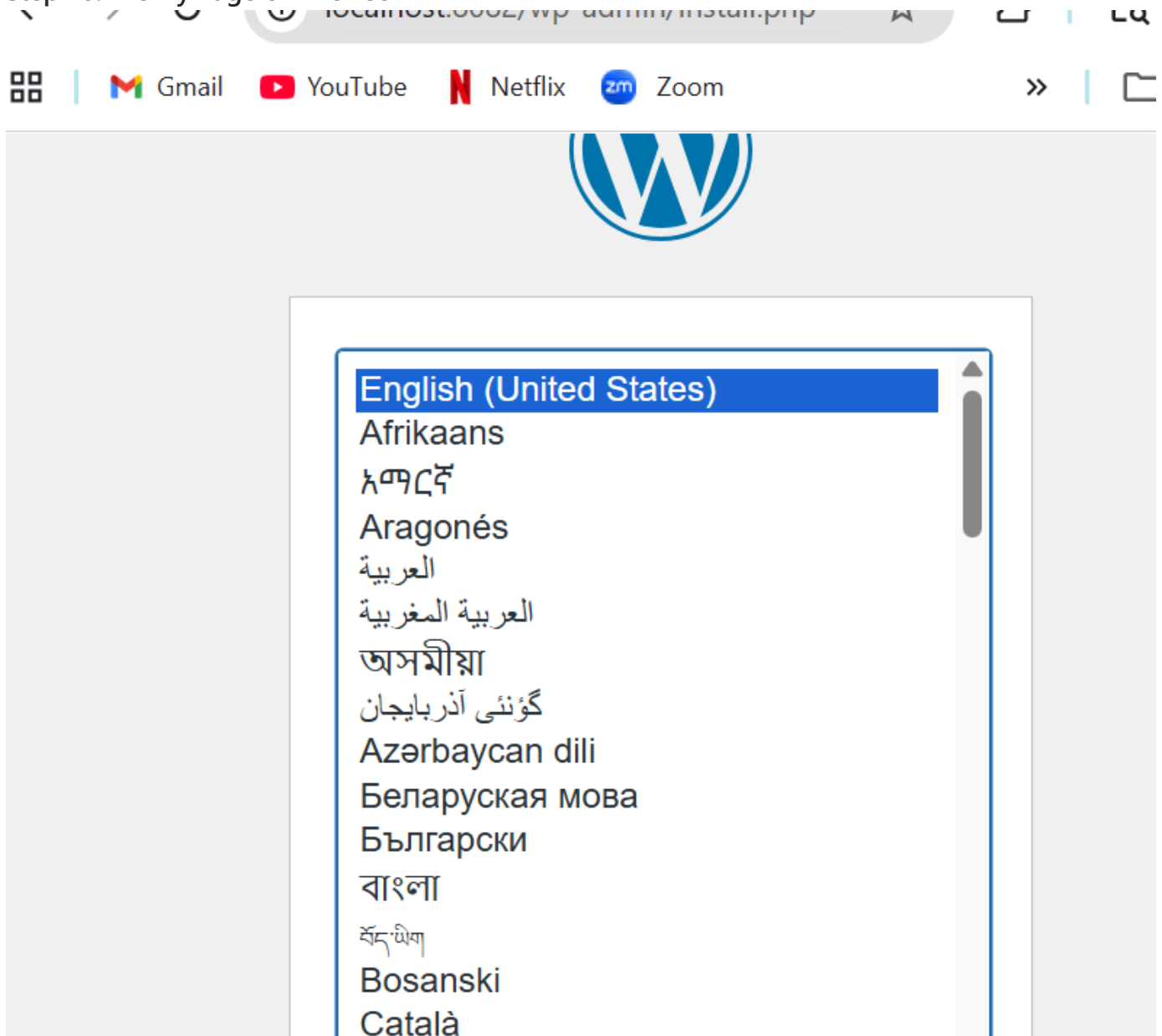
```
sristee@Sristee:/mnt/c/Users/srist$ docker run -d \  
--name mysql \  
--network wp-net \  
-e MYSQL_ROOT_PASSWORD=secret \  
-e MYSQL_DATABASE=wordpress \  
mysql:5.7  
cdc76750c0a8f83c57e30c2ebc75503636747d419eb866b9e4ba647c872bf305
```

Step-9:- Run wordpress container

```
docker run -d \  
--name wordpress \  
--network wp-net \  
-p 8082:80 \  
-e WORDPRESS_DB_HOST=mysql \  
-e WORDPRESS_DB_USER=root \  
-e WORDPRESS_DB_PASSWORD=secret \  
-e WORDPRESS_DB_NAME=wordpress \  
wordpress:latest
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker run -d \  
--name wordpress \  
--network wp-net \  
-p 8082:80 \  
-e WORDPRESS_DB_HOST=mysql \  
-e WORDPRESS_DB_USER=root \  
-e WORDPRESS_DB_PASSWORD=secret \  
-e WORDPRESS_DB_NAME=wordpress \  
wordpress:latest  
addc30464af4e890156479d7e6687e7041240cf4c8270308b03472e7c9cb26a7
```

Step-10:- Verify Page on Browser



Step-11:- Wordpress & MySql via compose

```
mkdir wp-compose
cd wp-compose
nano docker-compose.yml
cat docker-compose.yml
```

```
services:
  mysql:
    image: mysql:5.7
    container_name: mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: secret
```

```
volumes:  
  - mysql_data:/var/lib/mysql
```

```
wordpress:  
  image: wordpress:latest  
  container_name: wordpress  
  restart: always  
  ports:  
    - "8082:80"  
  environment:  
    WORDPRESS_DB_HOST: mysql  
    WORDPRESS_DB_USER: wpuser  
    WORDPRESS_DB_PASSWORD: secret  
  depends_on:  
    - mysql
```

```
volumes:  
  mysql_data:
```

```
sristee@Sristee:/mnt/c/Users/srist$ mkdir wp-compose
cd wp-compose
nano docker-compose.yml
cat docker-compose.yml
mkdir: cannot create directory 'wp-compose': File exists
services:
  mysql:
    image: mysql:5.7
    container_name: mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: secret
    volumes:
      - mysql_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress
    restart: always
    ports:
      - "8082:80"
    environment:
      WORDPRESS_DB_HOST: mysql
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: secret
    depends_on:
      - mysql

volumes:
  mysql_data:
```

Step-12:- Start Services

```
docker-compose up -d
docker-compose ps
```

```

sristee@Sristee:/mnt/c/Users/srist/wp-compose$ docker-compose up -d
WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All containers will be scheduled on the current node.

To deploy your application across the swarm, use 'docker stack deploy'.

Creating mysql ... done
Creating wordpress ... done
sristee@Sristee:/mnt/c/Users/srist/wp-compose$ docker-compose ps

```

Name	Command	State	Ports
mysql	docker-entrypoint.sh mysqld	Up	3306/tcp, 33060/tcp
wordpress	docker-entrypoint.sh apach ...	Up	0.0.0.0:8082->80/tcp, :::8082->80/tcp

Step-14:- Create container with Node Image

```

docker run -d \
--name webapp \
-p 5000:5000 \
-e APP_ENV=production \
-e DEBUG=false \
--restart unless-stopped \
node:18-alpine

```

```

sristee@Sristee:/mnt/c/Users/srist/wp-compose$ docker run -d \
--name webapp \
-p 5000:5000 \
-e APP_ENV=production \
-e DEBUG=false \
--restart unless-stopped \
node:18-alpine
84bee19ff5d897676fc5b99fb2bc82f50b8c5a6c95728877b5543d25dc1811e1

```

Step-15:- Create Node container via Compose

```

mkdir task3-webapp
cd task3-webapp
nano docker-compose.yml
cat docker-compose.yml

```

docker-compose.yml :-

```

services:
  webapp:
    image: node:18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production

```

```
DEBUG: "false"
restart: unless-stopped
```

```
sristee@Sristee:/mnt/c/Users/srist/wp-compose$ mkdir task3-webapp
cd task3-webapp
nano docker-compose.yml
cat docker-compose.yml
mkdir: cannot create directory 'task3-webapp': File exists
services:
  webapp:
    image: node:18-alpine
    container_name: webapp
    ports:
      - "5000:5000"
    environment:
      APP_ENV: production
      DEBUG: "false"
    restart: unless-stopped
```

Step-16:- Start Services

```
docker-compose up -d
docker-compose ps
```

```
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker-compose up -d
WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All containers will be scheduled on the current node
.

To deploy your application across the swarm, use 'docker stack deploy'.

Creating webapp ... done
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker-compose ps
   Name                  Command             State      Ports
-----
webapp  docker-entrypoint.sh node  Restarting
```

Step-18:- Stop Services

```
docker-compose down
```

```
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker-compose down
Stopping webapp ... done
Removing webapp ... done
Removing network task3-webapp_default
```

Step-19:- Create Network

```
docker network create app-net
```

```
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker network create app-net
b6012558fee73cdac0795db7ab1267b0fbd53c6e72e6eecd44458fc0d798e17e
```

Step-20:- Create Python & Postgres Conatiners via Terminal

```
docker run -d \
--name postgres-db \
--network app-net \
-e POSTGRES_USER=admin \
-e POSTGRES_PASSWORD=secret \
-v pgdata:/var/lib/postgresql/data \
postgres:15
```

```
docker run -d \
--name backend \
--network app-net \
-p 8000:8000 \
-e DB_HOST=postgres-db \
-e DB_USER=admin \
-e DB_PASS=secret \
python:3.11-slim
```

Step-21:- List Containers

```
docker ps
```

```
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker run -d \
--name postgres-db \
--network app-net \
-e POSTGRES_USER=admin \
-e POSTGRES_PASSWORD=secret \
-v pgdata:/var/lib/postgresql/data \
postgres:15

docker run -d \
--name backend \
--network app-net \
-p 8000:8000 \
-e DB_HOST=postgres-db \
-e DB_USER=admin \
-e DB_PASS=secret \
python:3.11-slim
6c4e46911b433e310b6a136f743f59578c73ebdd90a020dfdale71398aafc015
Unable to find image 'python:3.11-slim' locally
3.11-slim: Pulling from library/python
3531af2bc2a9: Already exists
91ff8760033c: Pull complete
f3ba2250c524: Pull complete
7ccd73948dde: Pull complete
Digest: sha256:6d85378d88a19cd4d76079817532d62232be95757cb45945a99fec8e8084b9c2
Status: Downloaded newer image for python:3.11-slim
9188d3583837ca6b761da24cee77924e132cbcba1729d6c9ece449bf32712ab0
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
6c4e46911b43	postgres:15	"docker-entrypoint.s..."	18 seconds ago	Up 18 seconds	5432/tcp
9bce4714d412	wordpress:latest	"docker-entrypoint.s..."	2 minutes ago	Up 2 minutes	0.0.0.0:8082->80/tcp, [::]:8082->80/tcp

Step-22:- List Networks

```
docker network ls
```

```
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
b6012558fee7	app-net	bridge	local
0683ee750a2c	bridge	bridge	local
9329abaf5075	docker_gwbridge	bridge	local
e6aa5e1497ad	host	host	local
l8dx8tqhvsf7	ingress	overlay	swarm
bb27b8c6ad33	none	null	local
9dca686b2520	wp-compose_default	bridge	local
9a8a10e4a0d0	wp-net	bridge	local

Step-23:- List Volumes

```
docker volumes ls
```

```
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker volume ls
```

DRIVER	VOLUME NAME
local	6aebc62e1838b938267634ef823236f3e5979915f7012536ca634f7e15a211ce
local	22c155c90b0e4bad2e014c4eb848bda37df50eb3be8c02f76c4b24fc6f05b776
local	74440ef86cd5f52f3d1559282a1adaf67dc03e641eb1f3d04b9dfae0b8514687
local	container-app_pgdata
local	k3d-mycluster-images
local	mydata
local	mysql-data
local	myvolume
local	pgdata
local	wp-compose_mysql_data

Step-24:- Stop Containers & Remove Network

```
docker stop backend postgres-db
docker rm backend postgres-db
docker network rm app-net
```

```
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker stop backend postgres-db
docker rm backend postgres-db
docker network rm app-net
backend
postgres-db
backend
postgres-db
app-net
```

Step-25:- Create same services via Compose

```
nano docker-compose.yml
cat docker-compose.yml
```

```
docker-compose.yml :-
```

```
services:
  postgres-db:
```

```
image: postgres:15
container_name: postgres-db
environment:
  POSTGRES_USER: admin
  POSTGRES_PASSWORD: secret
volumes:
  - pgdata:/var/lib/postgresql/data
networks:
  - app-net
```

```
backend:
  image: python:3.11-slim
  container_name: backend
  ports:
    - "8000:8000"
  environment:
    DB_HOST: postgres-db
    DB_USER: admin
    DB_PASS: secret
  depends_on:
    - postgres-db
  networks:
    - app-net
```

```
volumes:
  pgdata:
```

```
networks:
  app-net:
```

```

sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ nano docker-compose.yml
cat docker-compose.yml
services:
  postgres-db:
    image: postgres:15
    container_name: postgres-db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - app-net

  backend:
    image: python:3.11-slim
    container_name: backend
    ports:
      - "8000:8000"
    environment:
      DB_HOST: postgres-db
      DB_USER: admin
      DB_PASS: secret
    depends_on:
      - postgres-db
    networks:
      - app-net

volumes:
  pgdata:

networks:
  app-net:

```

Step-26:- Start Services

```

docker-compose up -d
docker-compose ps

```

```

sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker-compose up -d
docker-compose ps
WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All containers will be scheduled on the current node
.

To deploy your application across the swarm, use 'docker stack deploy'.

Creating network "task3-webapp_app-net" with the default driver
Creating volume "task3-webapp_pgdata" with default driver
Creating postgres-db ... done
Creating backend ... done

```

Name	Command	State	Ports
backend	python3	Exit 0	
postgres-db	docker-entrypoint.sh postgres	Up	5432/tcp

Step-27:- List Volumes

```

docker volume ls

```

```

sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker volume ls
DRIVER      VOLUME NAME
local       6aebc62e1838b938267634ef823236f3e5979915f7012536ca634f7e15a211ce
local       22c155c90b0e4bad2e014c4eb848bda37df50eb3be8c02f76c4b24fc6f05b776
local       74440ef86cd5f52f3d1559282a1adaf67dc03e641eb1f3d04b9dfae0b8514687
local       container-app_pgdata
local       k3d-mycluster-images
local       mydata
local       mysql-data
local       myvolume
local       pgdata
local       task3-webapp_pgdata
local       wp-compose_mysql_data

```

Step-28:- Stop Services

```
docker-compose down
```

```

sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker-compose down
Stopping postgres-db ... done
Removing backend      ... done
Removing postgres-db  ... done
Removing network task3-webapp_app-net

```

Step-29:- Create Node app

```

nano app.js
cat app.js

```

app.js :-

```

const http = require('http');

http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000);

```

```

sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ nano app.js
cat app.js
const http = require('http');

http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000);

```

Step-30:- Create Dockerfile

```

nano Dockerfile
cat Dockerfile

```

Dockerfile :-

```
FROM node:18-alpine

WORKDIR /app

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

```
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ nano Dockerfile
cat Dockerfile
FROM node:18-alpine

WORKDIR /app

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]
```

Step-31:- Create Compose File

```
nano docker-compose.yml
cat docker-compose.yml
```

docker-compose.yml :-

```
services:
  nodeapp:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: custom-node-app
    ports:
      - "3000:3000"
```

```
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ nano docker-compose.yml
cat docker-compose.yml
services:
  nodeapp:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: custom-node-app
    ports:
      - "3000:3000"
```

Step-32:- Start Compose Services

```
docker-compose up --build -d
docker-compose ps
```

```
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ docker-compose up --build -d
docker-compose ps
WARNING: The Docker Engine you're using is running in swarm mode.

Compose does not use swarm mode to deploy services to multiple nodes in a swarm. All containers will be scheduled on the current node
.

To deploy your application across the swarm, use 'docker stack deploy'.

Creating network "task3-webapp_default" with the default driver
Building nodeapp
failed to fetch metadata: fork/exec /usr/local/lib/docker/cli-plugins/docker-buildx: no such file or directory

DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
              Install the buildx component to build images with BuildKit:
              https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.096kB
Step 1/5 : FROM node:18-alpine
--> ee77c6cd7c18
Step 2/5 : WORKDIR /app
--> Using cache
--> 41c7f8b6dfb8
Step 3/5 : COPY app.js .
--> a1554c6f30d9
Step 4/5 : EXPOSE 3000
--> Running in 7469e03c1362
--> Removed intermediate container 7469e03c1362
--> 97937967b65f
Step 5/5 : CMD ["node", "app.js"]S
--> Running in 27d10de5b288
--> Removed intermediate container 27d10de5b288
--> 87009cb9c609
```

```
---> Removed intermediate container 27d10de5b288
---> 87009cb9c609
```

Successfully built 87009cb9c609

Successfully tagged task3-webapp_nodeapp:latest

Creating custom-node-app ... done

Name	Command	State	Ports
custom-node-app	docker-entrypoint.sh /bin/	Exit 127	

...

```
sristee@Sristee:/mnt/c/Users/srist/wp-compose/task3-webapp$ |
```

Conclusion

This experiment demonstrated:

- How to deploy **WordPress + MySQL** using Docker Compose.
- How containers communicate using internal networking.
- Importance of **volumes** for persistence.

Containerization-and-DevOps

Experiment 7: CI/CD using Jenkins, GitHub and Docker Hub

Introduction

What is Jenkins?

Jenkins is a **web-based GUI automation server** used to:

- Build applications
- Test code
- Deploy software

It provides:

- Dashboard (browser-based UI)
- Plugin ecosystem (GitHub, Docker, etc.)
- Pipeline as Code using `Jenkinsfile`

What is CI/CD ??

- **Continuous Integration (CI)**: Code is automatically built and tested after each commit
- **Continuous Deployment (CD)**: Built artifacts (Docker images) are automatically delivered/deployed

Workflow Overview

Developer → GitHub → Webhook → Jenkins → Build → Docker Hub

Prerequisites

Project Structure

```
my-app/  
├─ app.py  
└─ requirements.txt
```

- └─ Dockerfile
- └─ Jenkinsfile

Hands-On

Step-1:- Create a `app.py`

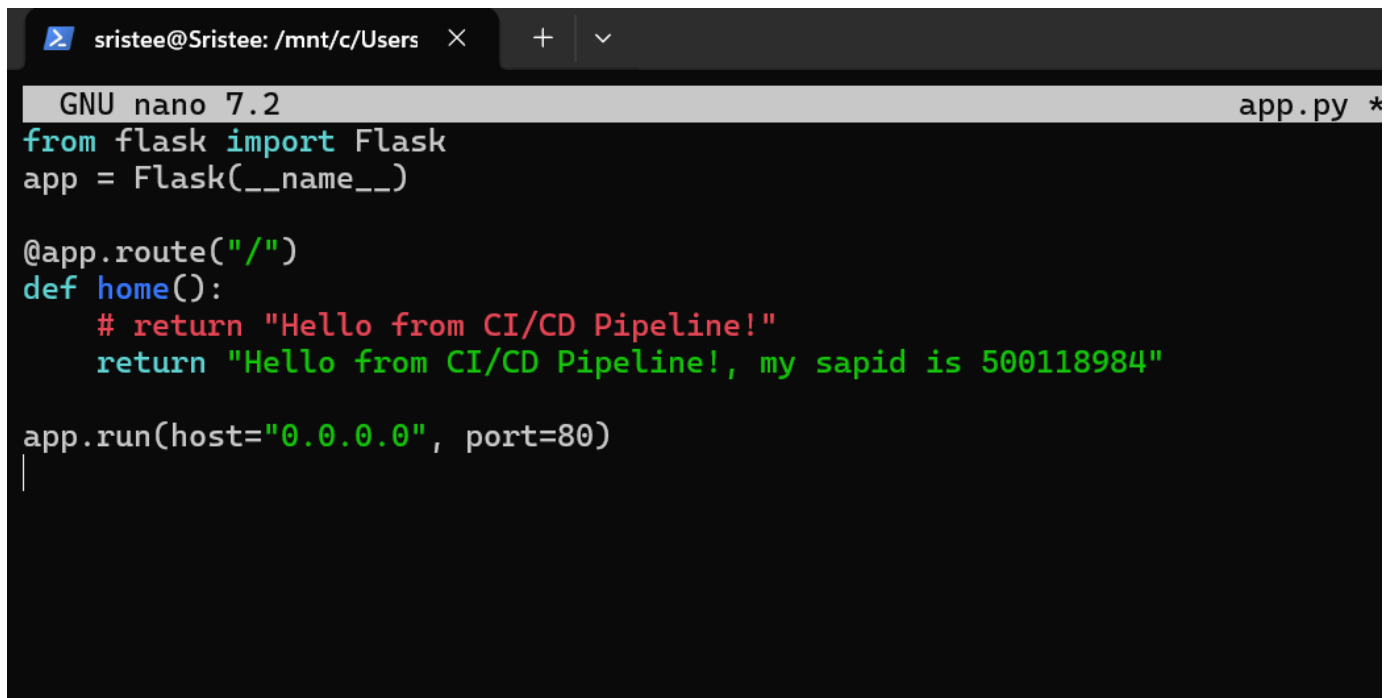
```
nano app.py
```

Paste This:

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    # return "Hello from CI/CD Pipeline!"
    return "Hello from CI/CD Pipeline!, my sapid is 500118984"

app.run(host="0.0.0.0", port=80)
```



```
sristee@Sristee: /mnt/c/Users  ×  +  v
GNU nano 7.2 app.py *
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    # return "Hello from CI/CD Pipeline!"
    return "Hello from CI/CD Pipeline!, my sapid is 500118984"

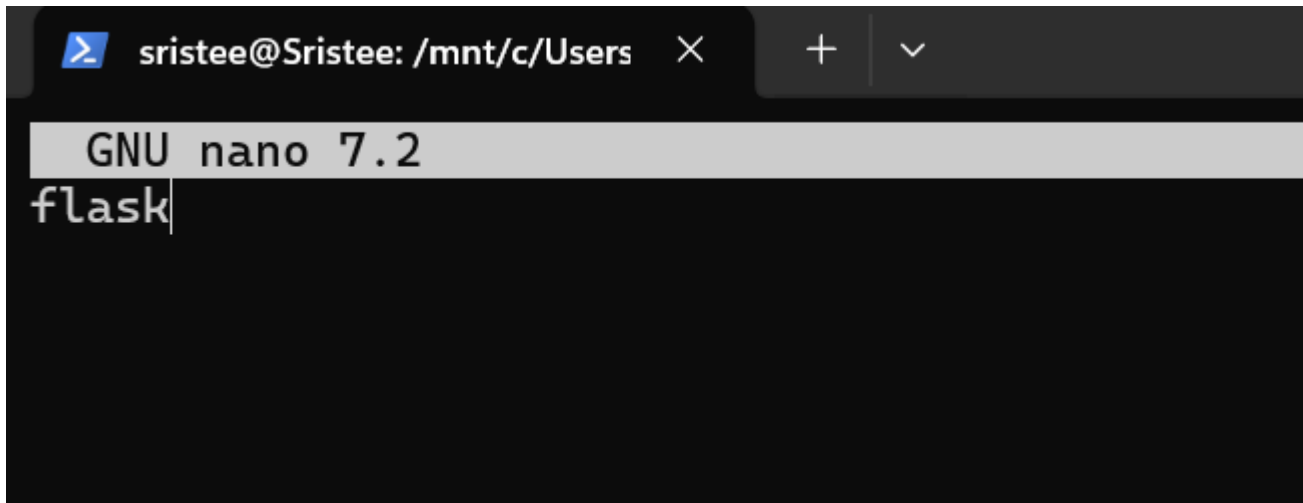
app.run(host="0.0.0.0", port=80)
|
```

Step-2:- Create `requirements.txt`

```
nano requirements.txt
```

Paste this:

```
flask
```



```
sristee@Sristee: /mnt/c/Users
GNU nano 7.2
flask
```

Step-3:- Create Dockerfile

```
nano Dockerfile
```

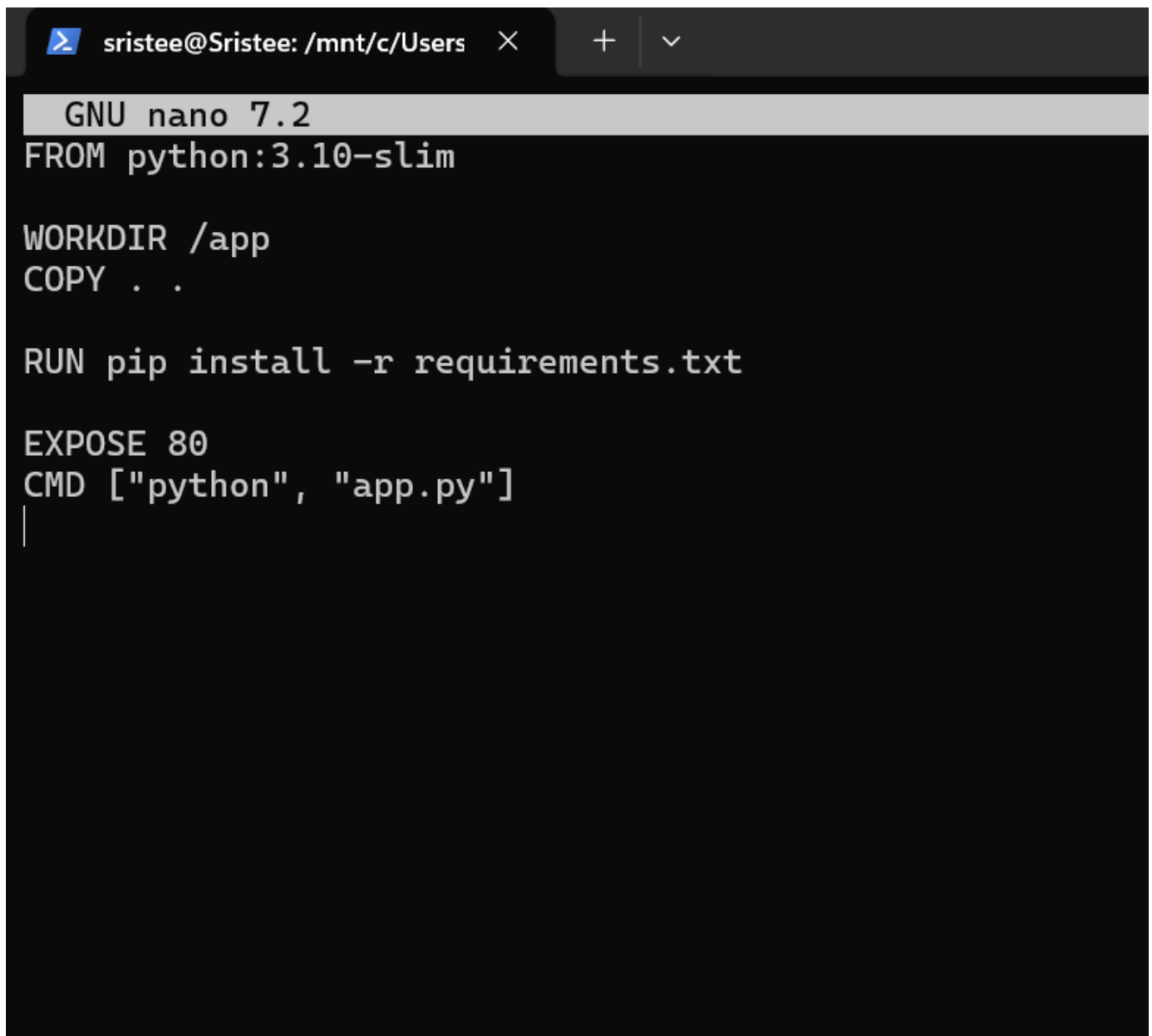
Paste this:

```
FROM python:3.10-slim

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80
CMD ["python", "app.py"]
```

A screenshot of a terminal window with a dark background. The title bar shows a terminal icon, the username 'sristee@Sristee', and the path '/mnt/c/Users'. Below the title bar, the terminal displays the content of a Dockerfile being edited in nano 7.2. The text is as follows:

```
GNU nano 7.2
FROM python:3.10-slim

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80
CMD ["python", "app.py"]
|
```

Step-4:- Create Jenkinsfile

```
nano Jenkinsfile
```

Paste this:

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "dhairyathareja/myapp"
    }

    stages {

        stage('Build Docker Image') {
            steps {
```

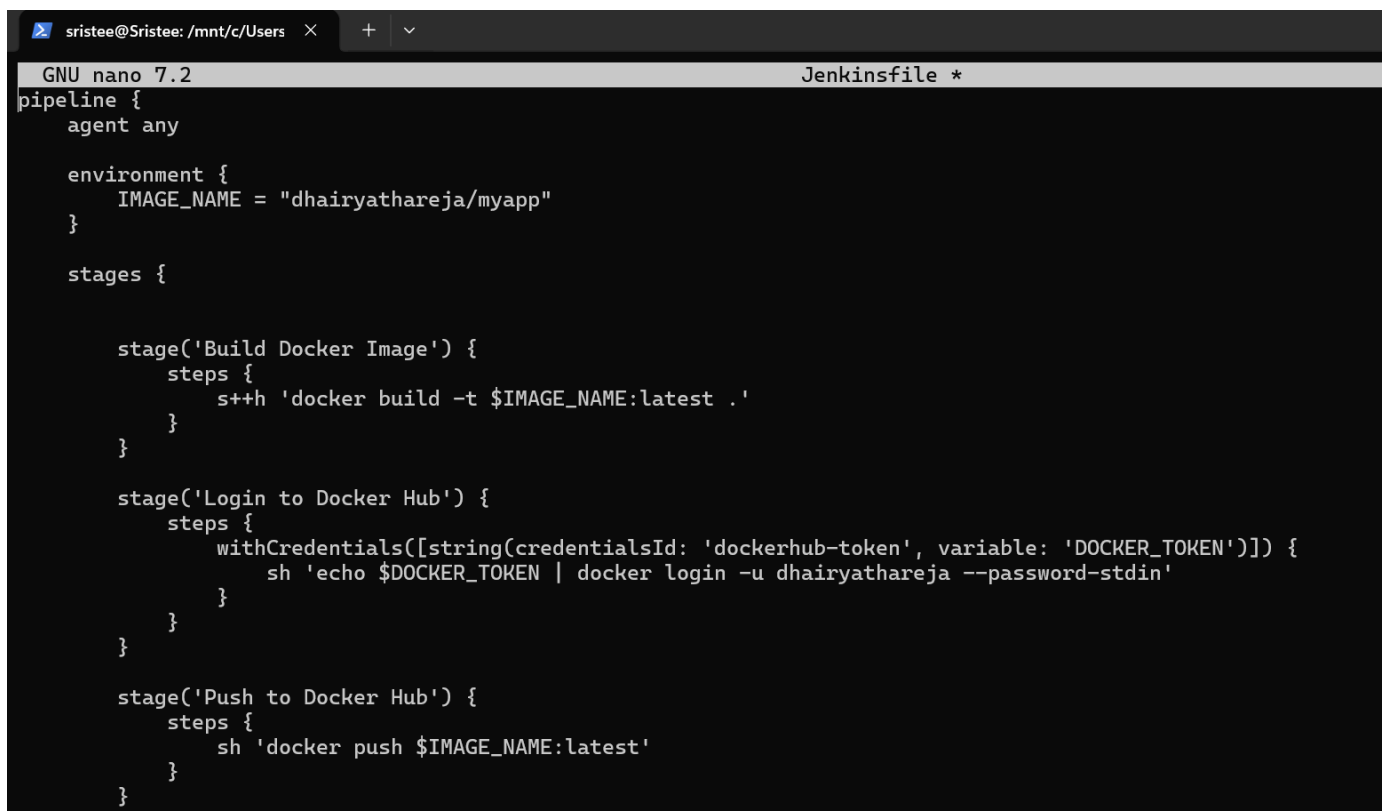
```

s++h 'docker build -t $IMAGE_NAME:latest .'
    }
}

stage('Login to Docker Hub') {
    steps {
        withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKE
            sh 'echo $DOCKER_TOKEN | docker login -u dhairyathareja --password-std
        }
    }
}

stage('Push to Docker Hub') {
    steps {
        sh 'docker push $IMAGE_NAME:latest'
    }
}
}
}
}

```



```

GNU nano 7.2 Jenkinsfile *
pipeline {
    agent any

    environment {
        IMAGE_NAME = "dhairyathareja/myapp"
    }

    stages {

        stage('Build Docker Image') {
            steps {
                s++h 'docker build -t $IMAGE_NAME:latest .'
            }
        }

        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
                    sh 'echo $DOCKER_TOKEN | docker login -u dhairyathareja --password-stdin'
                }
            }
        }

        stage('Push to Docker Hub') {
            steps {
                sh 'docker push $IMAGE_NAME:latest'
            }
        }
    }
}

```

Step-5:- Create docker-compose.yml

```

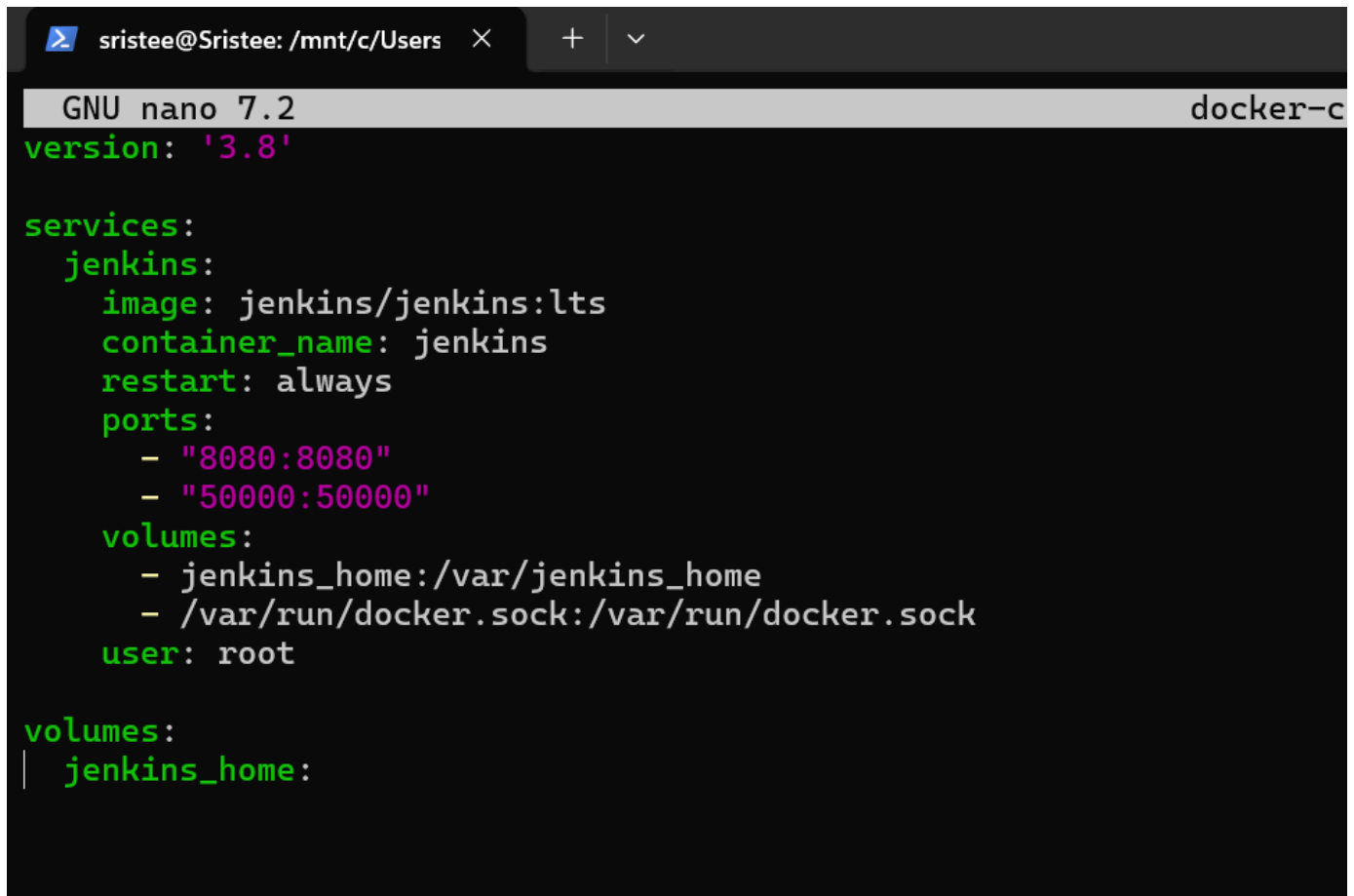
version: '3.8'

services:
    jenkins:
        image: jenkins/jenkins:lts
        container_name: jenkins
        restart: always

```

```
ports:
  - "8080:8080"
  - "50000:50000"
volumes:
  - jenkins_home:/var/jenkins_home
  - /var/run/docker.sock:/var/run/docker.sock
user: root
```

```
volumes:
  jenkins_home:
```



```
sristee@Sristee: /mnt/c/Users  GNU nano 7.2  docker-c
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

volumes:
  jenkins_home:
```

Step-6:- Start Up Compose

```
docker-compose up -d
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker-compose up -d
Creating network "srist_default" with the default driver
Creating volume "srist_jenkins_home" with default driver
Pulling jenkins (jenkins/jenkins:lts)...
lts: Pulling from jenkins/jenkins
e714309bcf67: Pull complete
a7730063fcfe: Pull complete
6e428cebe944: Pull complete
1aac2fe67fcc: Pull complete
32ad68c1bba8: Pull complete
f015f18f3c7a: Pull complete
6e001067f512: Pull complete
32ebd389acde: Pull complete
b850d1c40542: Pull complete
0454b3406328: Pull complete
a576649e9376: Pull complete
beb35d8d9493: Pull complete
Digest: sha256:7004d07dbcdc5439fdad8853acdb029c5e3ab7a3d8190184fbf89bec66786c02
Status: Downloaded newer image for jenkins/jenkins:lts
Creating jenkins ... done
```

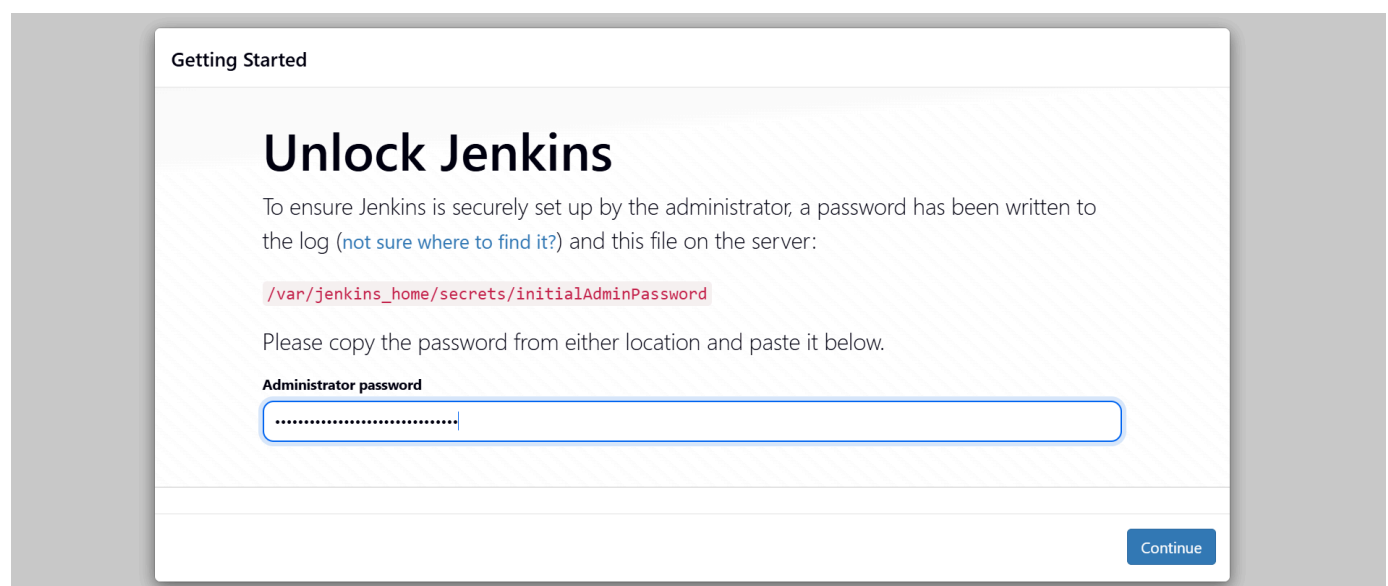
Step-7:- Exec Jenkins Contrainer to get Password

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

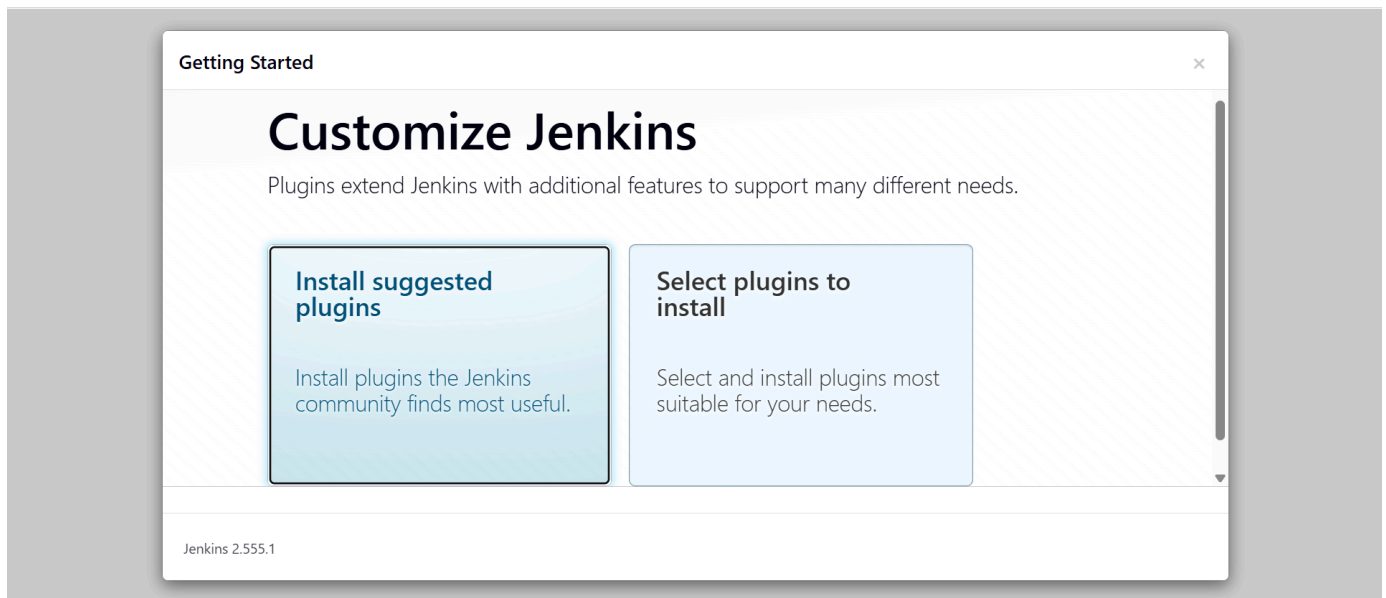
```
sristee@Sristee:/mnt/c/Users/srist$ docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
01d83812827b4bd7b7105c557aeee84e
```

Step-8:- Login to Jenkins Dashboard on Browser

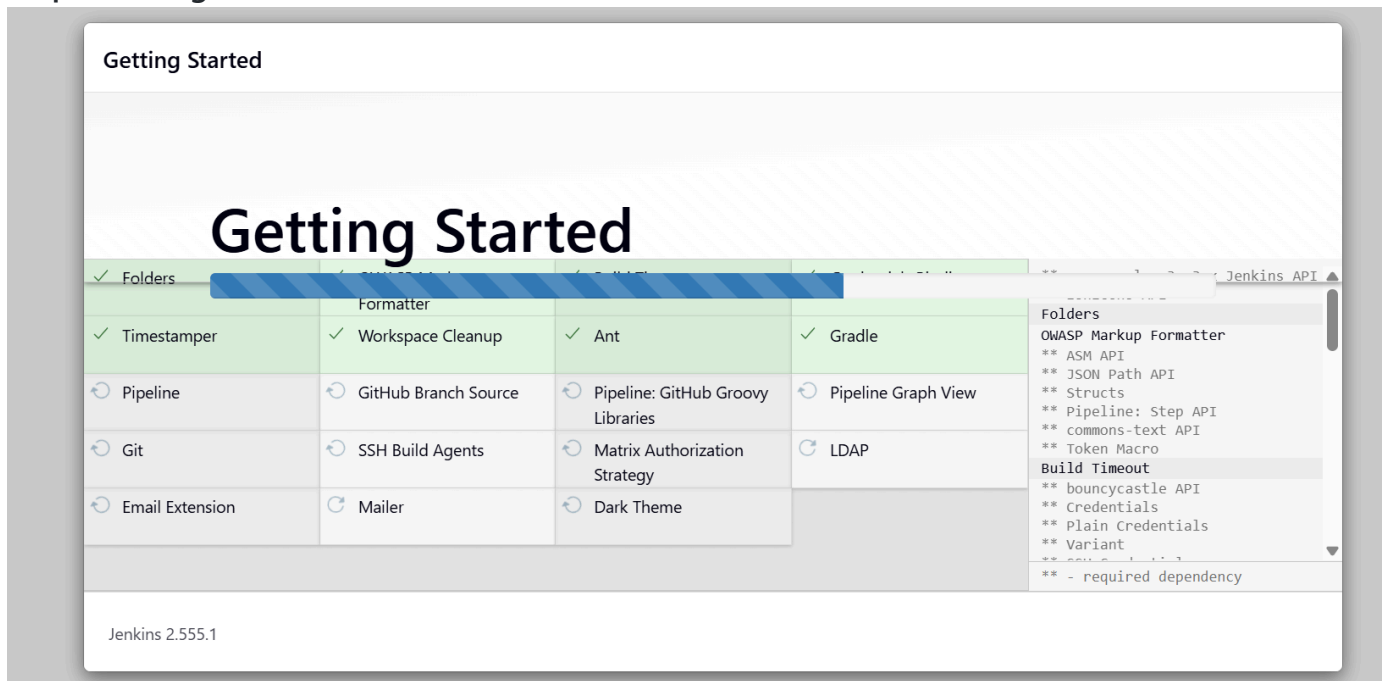
<http://localhost:8080>



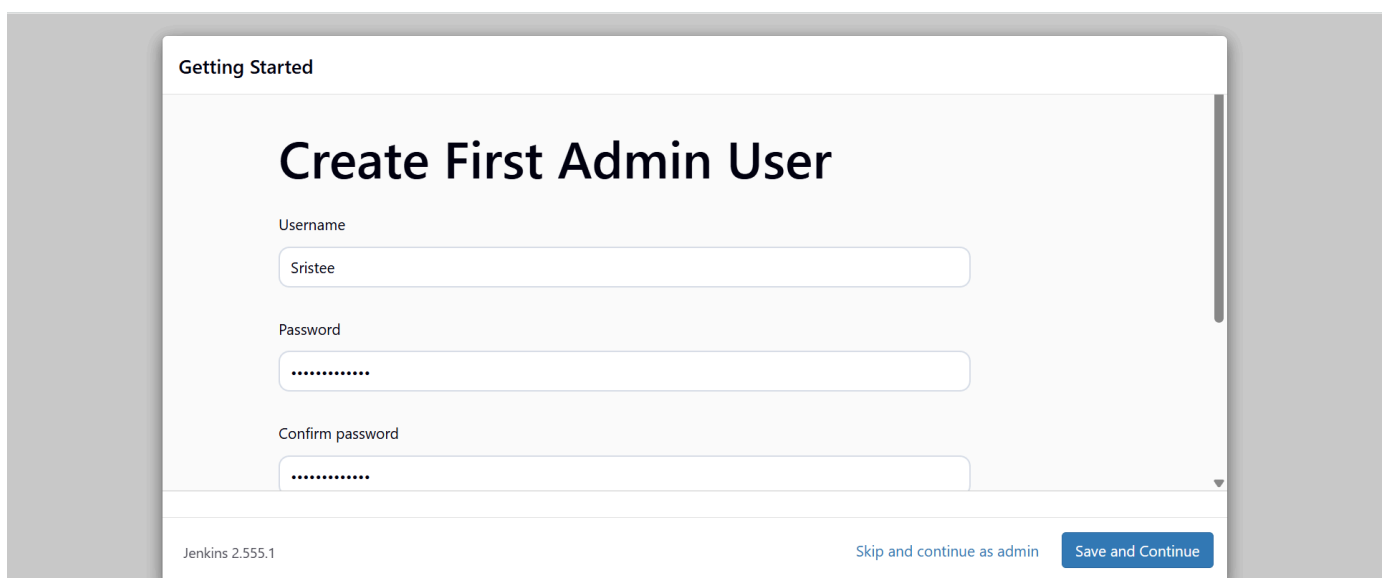
Step-9:- Install Sugeested Plugins



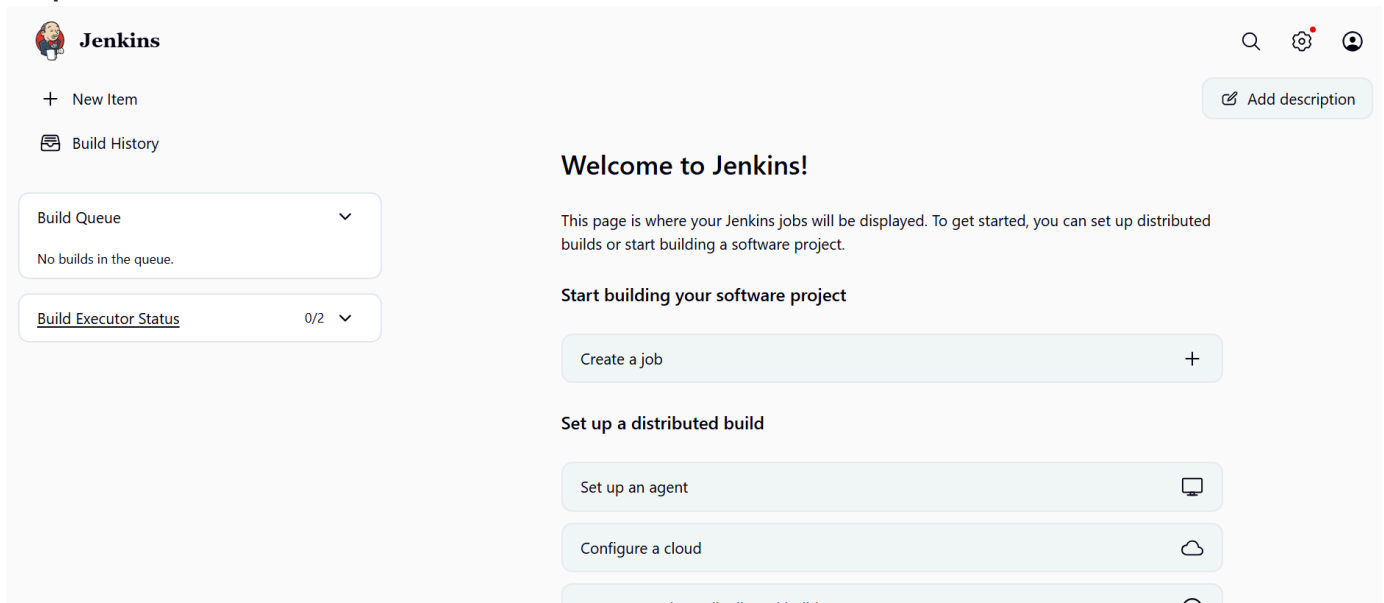
Step-10:- Plugins List



Step-11:- Create Admin User

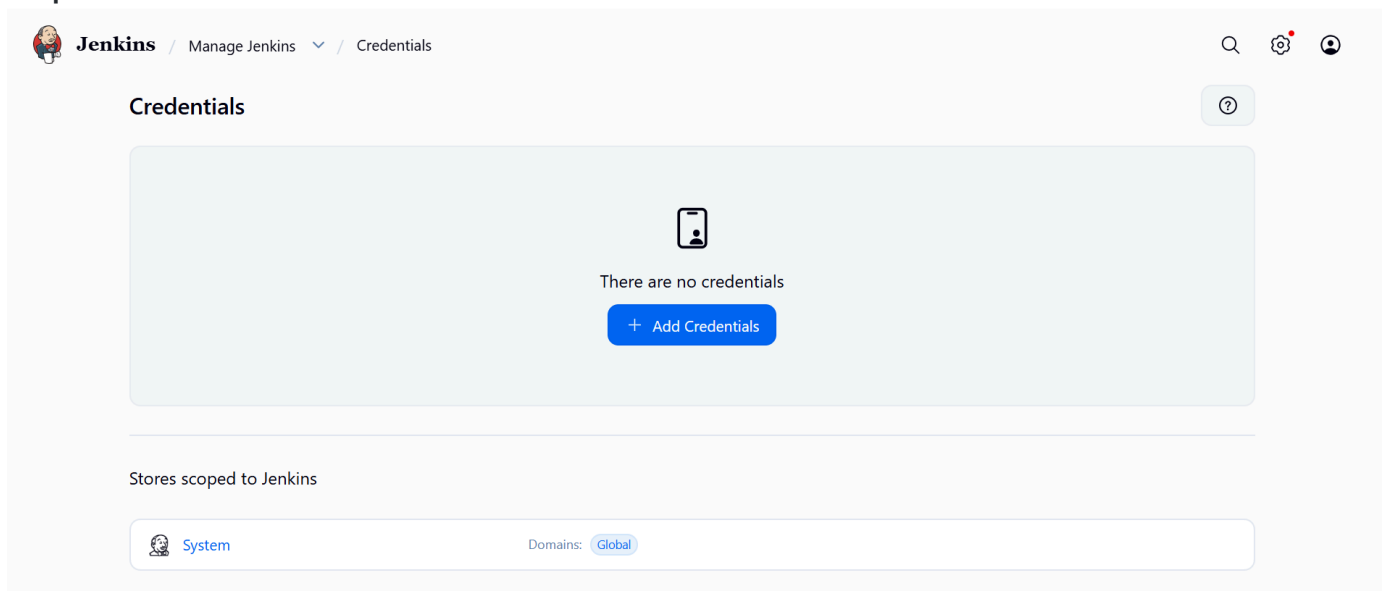


Step-12:- Admin Dashboard Interface



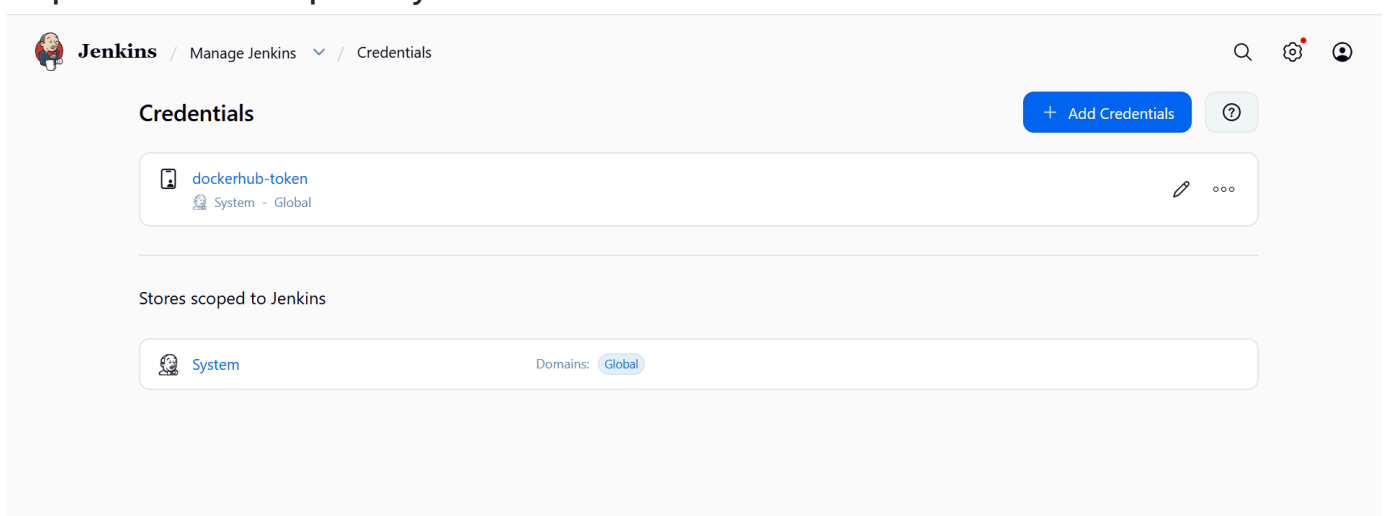
The Jenkins Admin Dashboard interface shows the Jenkins logo and name in the top left. Below it are links for '+ New Item' and 'Build History'. On the right, there are search, settings, and user icons, along with an 'Add description' button. The main content area is titled 'Welcome to Jenkins!' and includes a message: 'This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.' Below this is a section 'Start building your software project' with a 'Create a job' button. Another section 'Set up a distributed build' contains buttons for 'Set up an agent', 'Configure a cloud', and 'Learn more about distributed builds'.

Step-13:- Add DockerHub Token in Credentials



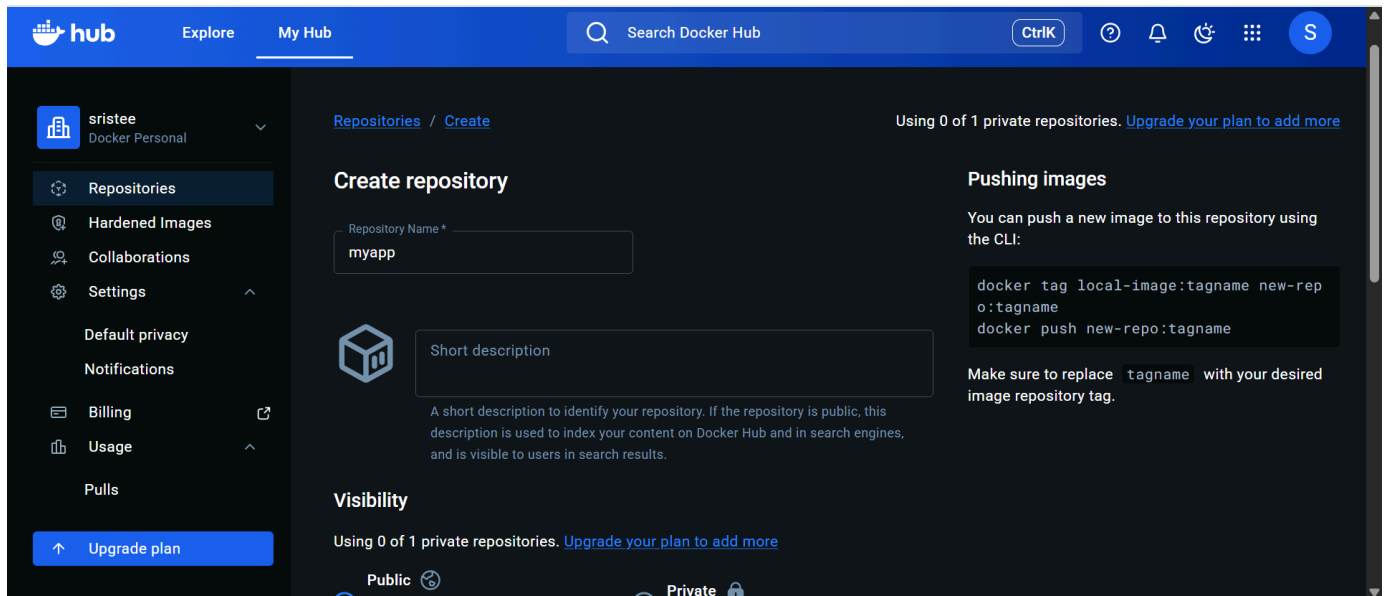
The Jenkins Credentials page is shown with the breadcrumb 'Manage Jenkins / Credentials'. The title 'Credentials' is at the top left, and a help icon is at the top right. The main area is a large light blue box with a person icon and the text 'There are no credentials'. Below this is a blue button labeled '+ Add Credentials'. At the bottom, there is a section 'Stores scoped to Jenkins' with a table showing the 'System' store and 'Global' domains.

Step-14:- Create a Repository in Dockerhub



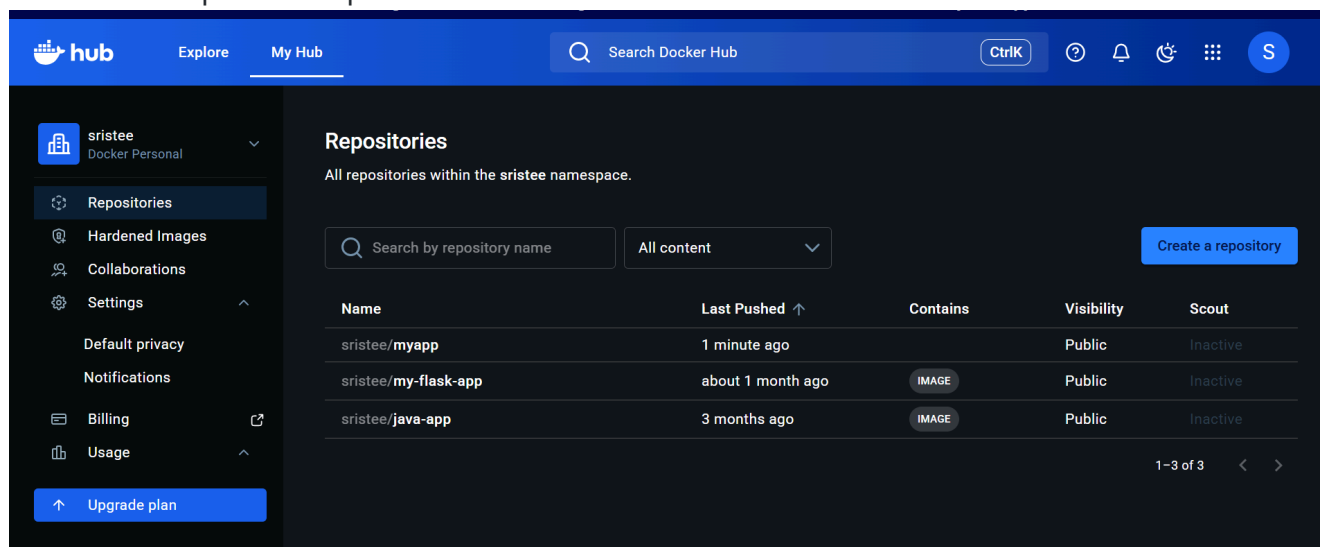
The Jenkins Credentials page is shown with the breadcrumb 'Manage Jenkins / Credentials'. The title 'Credentials' is at the top left, and a help icon is at the top right. A blue button labeled '+ Add Credentials' is at the top right. Below it, a credential entry is shown with the name 'dockerhub-token', the store 'System', and the domain 'Global'. To the right of the entry are edit and delete icons. At the bottom, there is a section 'Stores scoped to Jenkins' with a table showing the 'System' store and 'Global' domains.

Step-15:- Repository will be listed

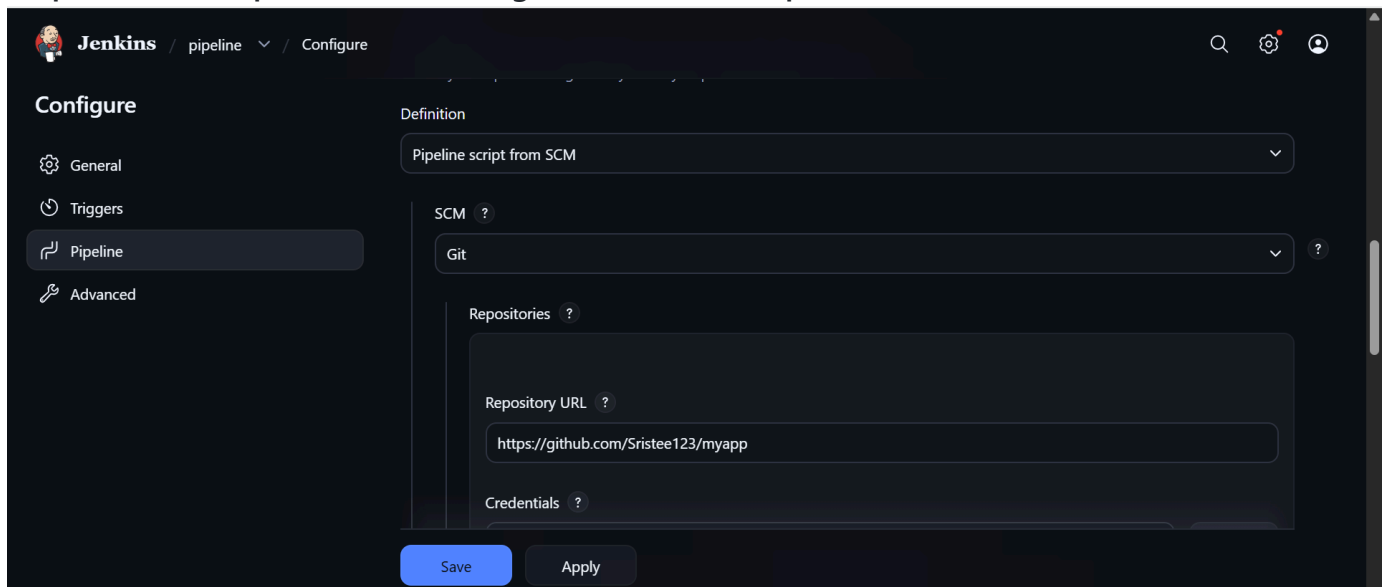


Step-16:- Create Pipeline Job

- New-Item:- Pipeline
- Name:- ci-cd-pipeline
- Definition:- Pipeline Script from SCM



Step-17:- Build Pipeline & Track Stages in Console Output



The screenshot shows the Jenkins 'Configure' page for a pipeline. The left sidebar has tabs for 'General', 'Triggers', 'Pipeline', and 'Advanced', with 'Pipeline' selected. The main area is titled 'Definition' and contains the following configuration:

- Definition:** Pipeline script from SCM
- SCM:** Git
- Repositories:**
 - Repository URL:** `https://github.com/Sristee123/myapp`
 - Credentials:** (empty field)

At the bottom, there are 'Save' and 'Apply' buttons.

Step-18:- Build Status Success will be shown with Green Tick



The screenshot shows the Jenkins pipeline status page for 'ci-cd-pipeline'. It features a green checkmark icon and the title 'ci-cd-pipeline'. Below the title is the heading 'Permalinks' followed by a list of build status links:

- Last build (#3), 2 min 24 sec ago
- Last stable build (#3), 2 min 24 sec ago
- Last successful build (#3), 2 min 24 sec ago
- Last failed build (#2), 13 min ago
- Last unsuccessful build (#2), 13 min ago
- Last completed build (#3), 2 min 24 sec ago

[< Previous](#) [Home](#) [Next >](#)

Containerization-and-DevOps

Experiment 9: Working with Ansible

Introduction

What is Ansible?

- Ansible is an open-source **automation tool** for **configuration management**, **application deployment**, and **orchestration**.
- It follows an **agentless** architecture, using SSH for Linux and WinRM for Windows.
- Uses YAML-based **playbooks** to define automation tasks.

Ansible is software that enables cross-platform automation and orchestration at scale and has become the standard choice among enterprise automation solutions.

Key Concepts

Component	Description
Control Node	Machine with Ansible installed
Managed Nodes	Target servers (no Ansible agent needed)
Inventory	Defines the list of managed nodes (EC2 instances, servers, etc.). inventory.ini
Playbooks	YAML files containing a sequence of automation steps.
Tasks	Individual actions in playbooks (e.g., installing a package).
Modules	Built-in functionality to perform tasks (e.g., <code>yum</code> , <code>apt</code> , <code>service</code>).
Roles	Pre-defined reusable automation scripts.

Benefits of using Ansible

- A free and open-source community project with a huge audience.
- Battle-tested over many years as the preferred tool of IT wizards.
- Easy to start and use from day one, without the need for any special coding skills.
- Simple deployment workflow without any extra agents.

- Includes some sophisticated features around modularity and reusability that come in handy as users become more proficient.
- Extensive and comprehensive official documentation that is complemented by a plethora of online material produced by its community.

To sum up, Ansible is simple yet powerful, agentless, community-powered, predictable, and secure.

Hands-On

Step-1:- Install Ansible in WSL

```
sudo apt install ansible -y
```

Step-2:- Verify Installation

```
ansible --version
```

Step-3:- Ping Localhost

```
ansible localhost -m ping
```

Step-4:- Generate ssh Keys

```
ssh-keygen -t rsa -b 4096
```

Step-5:- Copy keys in id_rsa

```
#Copy in current Directory  
cp ~/.ssh/id_rsa.pub .  
cp ~/.ssh/id_rsa .
```

Step-6:- Create Dockerfile

```
FROM ubuntu

RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server
RUN mkdir -p /var/run/sshd

# Configure SSH
RUN mkdir -p /run/sshd && \
    echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config
    sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config

# Create .ssh directory and set proper permissions
RUN mkdir -p /root/.ssh && \
    chmod 700 /root/.ssh

# Copy SSH keys (note: this is not secure for production!)
COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

# Set proper permissions for keys
RUN chmod 600 /root/.ssh/id_rsa && \
    chmod 644 /root/.ssh/authorized_keys

# Fix for SSH Login
RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/ssh/sshd_config

# Expose SSH port
EXPOSE 22

# Start SSH service when container starts
CMD ["/usr/sbin/sshd", "-D"]
```

Step-7:- Build Image

```
docker build -t ubuntu-server
```

Step-8:- Run container from build Image

```
docker run -d -p --rm 2222:22 -p 8221:8221 --name ssh-test-server ubuntu-server
```

Step-9:- Find Container IP Address

```
docker inspect -f '{{ .NetworkSettings.IPAddress }}' ssh-test-server
```

Step-10:- Test SSH connection via Conatiner IP

```
# with key
ssh -i ~/.ssh/id_rsa root@172.17.0.2
```

Step-11:- Start Multiple Conatiners

```
for i in {1..4}; do
    echo -e "\n Creating server${i}\n"
    docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
    echo -e "IP of server${i} is $(docker inspect -f '{{ .NetworkSettings.IPAddress }}' server${i})"
done
```

Step-12:- Create inventory.ini

```
# Get container IPs
echo "[servers]" > inventory.ini
for i in {1..4}; do
    docker inspect -f '{{ .NetworkSettings.IPAddress }}' server${i} >> inventory.ini
done
```

```
# Add inventory variables
cat << EOF >> inventory.ini
```

```
[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

```
ansible_python_interpreter=/usr/bin/python3
EOF
```

Step-13:- Verify inventory.ini

```
cat inventory.ini
```

Step-14:- Test Connectivity

```
ssh -i ~/.ssh/id_rsa root@172.17.0.3
```

**

Step-15:- Ansible Ping Test

```
ansible all -i inventory.ini -m ping
```

Step-16:- Create Playbook playbook1.yml

```
--- # it should start with three dash only
- name: Update and configure servers
  hosts: all
  become: yes

  tasks:
    - name: Update apt packages
      apt:
        update_cache: yes
        upgrade: dist

    - name: Install required packages
      apt:
        name: ["vim", "htop", "wget"]
        state: present

    - name: Create test file
      copy:
        dest: /root/ansible_test.txt
        content: "Configured by Ansible on "
```

Step-17:- Run Playbook

```
ansible-playbook -i inventory.ini playbook1.yml
```

Step-18:- Verify Changes

```
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
```

Step-19:- CleanUp

```
for i in {1..4}; do docker rm -f server${i}; done
```

[< Previous](#) [Home](#) [Next >](#)

Containerization-and-DevOps

Experiment 10: Working with SonarQube

Introduction

What is SonarQube?

SonarQube is an open-source platform for continuous inspection of code quality. It performs automatic reviews with static analysis to detect bugs, code smells &/ security vulnerabilities.

How SonarQube Solves the Problem:

- **Continuous Inspection:** Scans code with every commit, providing immediate feedback.
- **Quality Gates:** Defines pass/fail criteria for code quality.
- **Technical Debt Quantification:** Measures effort needed to fix issues.
- **Multi-language Support:** Supports 20+ programming languages.
- **Visual Analytics:** Dashboard showing code quality metrics and trends.

Hands-On

Step-1:- Create a `docker-compose.yml`

```
nano docker-compose.yml
```

Paste This:

```
version: '3.8'

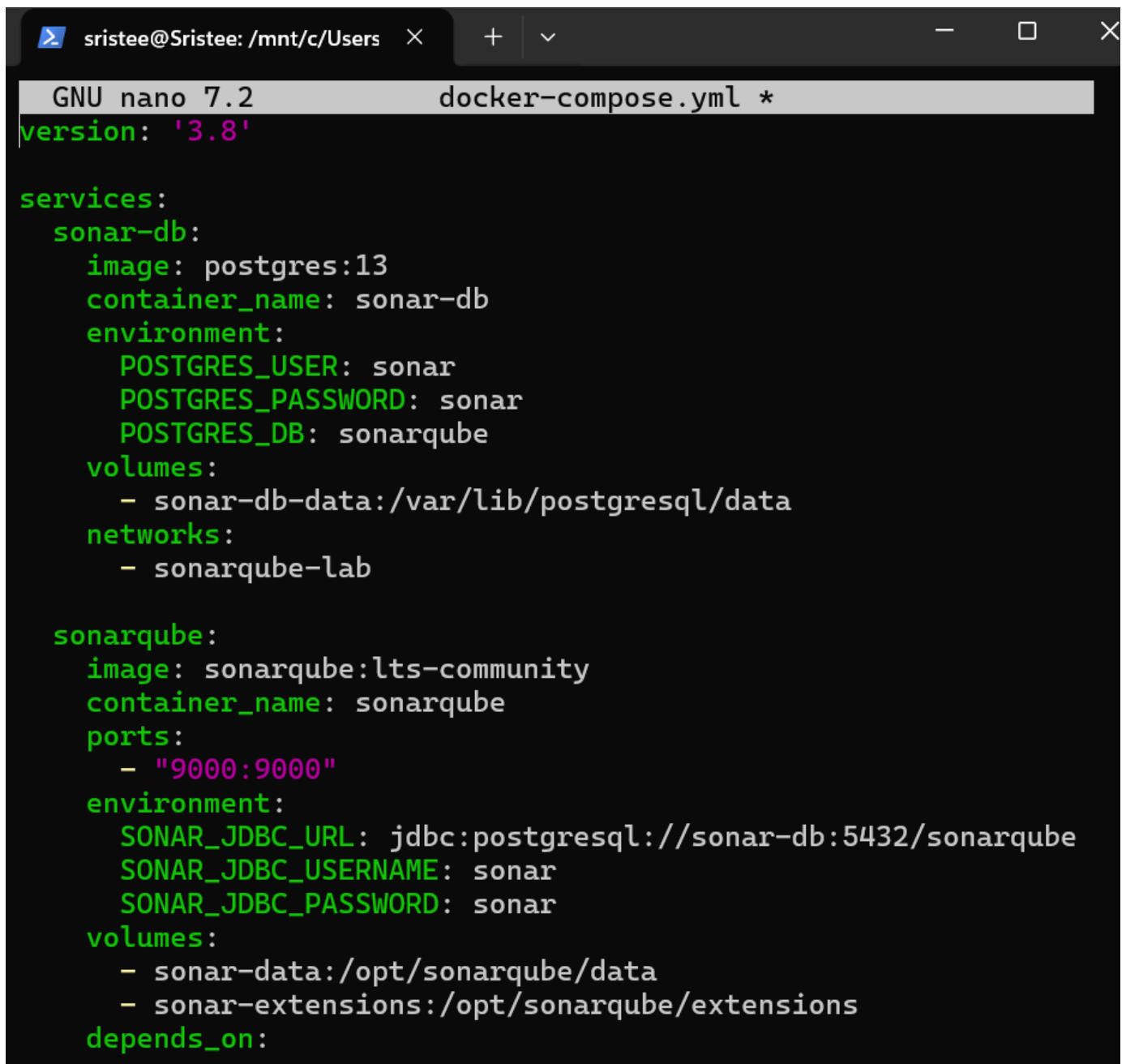
services:
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonarqube
```

```
volumes:  
  - sonar-db-data:/var/lib/postgresql/data  
networks:  
  - sonarqube-lab
```

```
sonarqube:  
  image: sonarqube:lts-community  
  container_name: sonarqube  
  ports:  
    - "9000:9000"  
  environment:  
    SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube  
    SONAR_JDBC_USERNAME: sonar  
    SONAR_JDBC_PASSWORD: sonar  
  volumes:  
    - sonar-data:/opt/sonarqube/data  
    - sonar-extensions:/opt/sonarqube/extensions  
  depends_on:  
    - sonar-db  
  networks:  
    - sonarqube-lab
```

```
volumes:  
  sonar-db-data:  
  sonar-data:  
  sonar-extensions:
```

```
networks:  
  sonarqube-lab:  
    driver: bridge
```

A screenshot of a terminal window with a dark background. The window title bar shows 'sristee@Sristee: /mnt/c/Users' and standard window controls. The terminal shows the GNU nano 7.2 editor editing 'docker-compose.yml'. The file content is a Docker Compose configuration for SonarQube and its database. The configuration includes a 'version' of '3.8', a 'services' section with 'sonar-db' (a PostgreSQL container) and 'sonarqube' (the SonarQube application), and a 'networks' section with 'sonarqube-lab'. The 'sonarqube' service has a 'depends_on' entry for 'sonar-db'.

```
GNU nano 7.2 docker-compose.yml *
version: '3.8'

services:
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonarqube
    volumes:
      - sonar-db-data:/var/lib/postgresql/data
  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000"
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db

networks:
  - sonarqube-lab
```

Step-2:- Start Compose

```
docker-compose up -d
```

```

sristee@Sristee:/mnt/c/Users/srist$ docker-compose up -d
Creating network "srist_sonarqube-lab" with driver "bridge"
Creating volume "srist_sonar-db-data" with default driver
Creating volume "srist_sonar-data" with default driver
Creating volume "srist_sonar-extensions" with default driver
WARNING: Found orphan containers (jenkins) for this project. If you removed or renamed this service in your compose file, you can run
this command with the --remove-orphans flag to clean it up.
Pulling sonar-db (postgres:13)...
13: Pulling from library/postgres
ae52c8ce1a35: Pull complete
bad194149a82: Pull complete
31efcb7: Pull complete
8cb000c698bc: Pull complete
23a39b1003e9: Pull complete
5f1cded65589: Pull complete
947bdcac0ea3: Pull complete
d7ecdcd7702a: Pull complete
fcfd49225910: Pull complete
ff08d42: Pull complete
Digest: sha256:4689940c683801b4ab839ab3b0a0a3555a5fe425371422310944e89eca7d8068
4f4fb700ef54: Pull complete
eb27e3a98da1: Pull complete
c7ad1fe61e07: Pull complete
4f4fb700ef54: Already exists
eb27e3a98da1: Downloading [=====] 100.7MB/300.4MB
b9a652f: Pull complete
f3929ce9ef98: Pull complete
e24a8
=====
1df735f481ad: Pull complete
5d5a1fad7028: Pull complete
Digest: sha256:f709975ab31d2d08f5a3ae2dc73a31ee011afc8cf28845082c17c55d45df9df5
Status: Downloaded newer image for sonarqube:lts-community
Creating sonar-db ... done
Creating sonarqube ... done

```

Step-3:- Verify SonarQube is running via Logs

docker-compose logs -f sonarqube

```

sristee@Sristee:/mnt/c/Users/srist$ docker-compose logs -f sonarqube
Attaching to sonarqube
sonarqube | 2026.05.01 13:46:42 INFO app[[o.s.a.AppFileSystem] Cleaning or creating temp directory /opt/sonarqube/temp
sonarqube | 2026.05.01 13:46:42 INFO app[[o.s.a.es.EsSettings] Elasticsearch listening on [HTTP: 127.0.0.1:9001, TCP: 127.0.0.1:38471]
sonarqube | 2026.05.01 13:46:42 INFO app[[o.s.a.ProcessLauncherImpl] Launch process[ELASTICSEARCH] from [/opt/sonarqube/elasticsearch]
sonarqube | 2026.05.01 13:46:42 INFO app[[o.s.a.SchedulerImpl] Waiting for Elasticsearch to be up and running
sonarqube | 2026.05.01 13:46:43 INFO es[[o.e.n.Node] version[7.17.15], pid[27], build[default/tar/0b8ecfb4378335f4689c4223d1f1115f16bef3ba/2023-11-10T22:03:46.987399016Z], OS[Linux/6.6.87.2-microsoft-standard-WSL2/amd64], JVM[Eclipse Adoptium/OpenJDK 64-Bit Server VM/17.0.16/17.0.16+8]
sonarqube | 2026.05.01 13:46:43 INFO es[[o.e.n.Node] JVM home [/opt/java/openjdk]
sonarqube | 2026.05.01 13:46:43 INFO es[[o.e.n.Node] JVM arguments [-XX:+UseG1GC, -Djava.io.tmpdir=/opt/sonarqube/temp, -XX:ErrorFile=/opt/sonarqube/logs/es_hs_err_pid%p.log, -Des.networkaddress.cache.ttl=60, -Des.networkaddress.cache.negative.ttl=10, -XX:+AlwaysPreTouch, -Xss1m, -Djava.awt.headless=true, -Dfile.encoding=UTF-8, -Djna.nosys=true, -Djna.tmpdir=/opt/sonarqube/temp, -XX:-OmitStackTraceInFastThrow, -Dio.netty.noUnsafe=true, -Dio.netty.noKeySetOptimization=true, -Dio.netty.recycler.maxCapacityPerThread=0, -Dio.netty allocator.numDirectArenas=0, -Dlog4j.shutdownHookEnabled=false, -Dlog4j2.disable.jmx=true, -Dlog4j2.formatMsgNoLookups=true, -Djava.locale.providers=COMPAT, -Dcom.redhat.fips=false, -Des.enforce.bootstrap.checks=true, -Xmx512m, -Xms512m, -XX:MaxDirectMemorySize=256m, -XX:+HeapDumpOnOutOfMemoryError, -Des.path.home=/opt/sonarqube/elasticsearch, -Des.path.conf=/opt/sonarqube/temp/conf/es, -Des.distribution.flavor=default, -Des.distribution.type=tar, -Des.bundled_jdk=false]
sonarqube | 2026.05.01 13:46:43 INFO es[[o.e.p.PluginsService] loaded module [analysis-common]
sonarqube | 2026.05.01 13:46:43 INFO es[[o.e.p.PluginsService] loaded module [lang-painless]
sonarqube | 2026.05.01 13:46:43 INFO es[[o.e.p.PluginsService] loaded module [parent-join]
sonarqube | 2026.05.01 13:46:43 INFO es[[o.e.p.PluginsService] loaded module [reindex]
sonarqube | 2026.05.01 13:46:43 INFO es[[o.e.p.PluginsService] loaded module [transport-netty4]
sonarqube | 2026.05.01 13:46:43 INFO es[[o.e.p.PluginsService] no plugins loaded
sonarqube | 2026.05.01 13:46:43 INFO es[[o.e.e.NodeEnvironment] using [1] data paths, mounts [[/opt/sonarqube/data (/dev/sde)], net usable_space [949.5gb], net total_space [1006.8gb], types [ext4]
sonarqube | 2026.05.01 13:46:43 INFO es[[o.e.e.NodeEnvironment] heap size [512mb], compressed ordinary object pointers [true]
sonarqube | 2026.05.01 13:46:43 INFO es[[o.e.n.Node] node name [sonarqube], node ID [AwW_7vW1R4mvW7_xVjhZpA], cluster name [sonarqube], roles [data_frozen, master, remote_cluster_client, data, data_content, data_hot, data_warm, data_cold, ingest]
sonarqube | 2026.05.01 13:46:46 INFO es[[o.e.t.NettyAllocator] creating NettyAllocator with the following configs: [name=unpooled, suggested_max_allocation_size=256kb, factors={es.unsafe.use_unpooled_allocator=null, g1gc_enabled=true, g1gc_region_size=1mb, heap

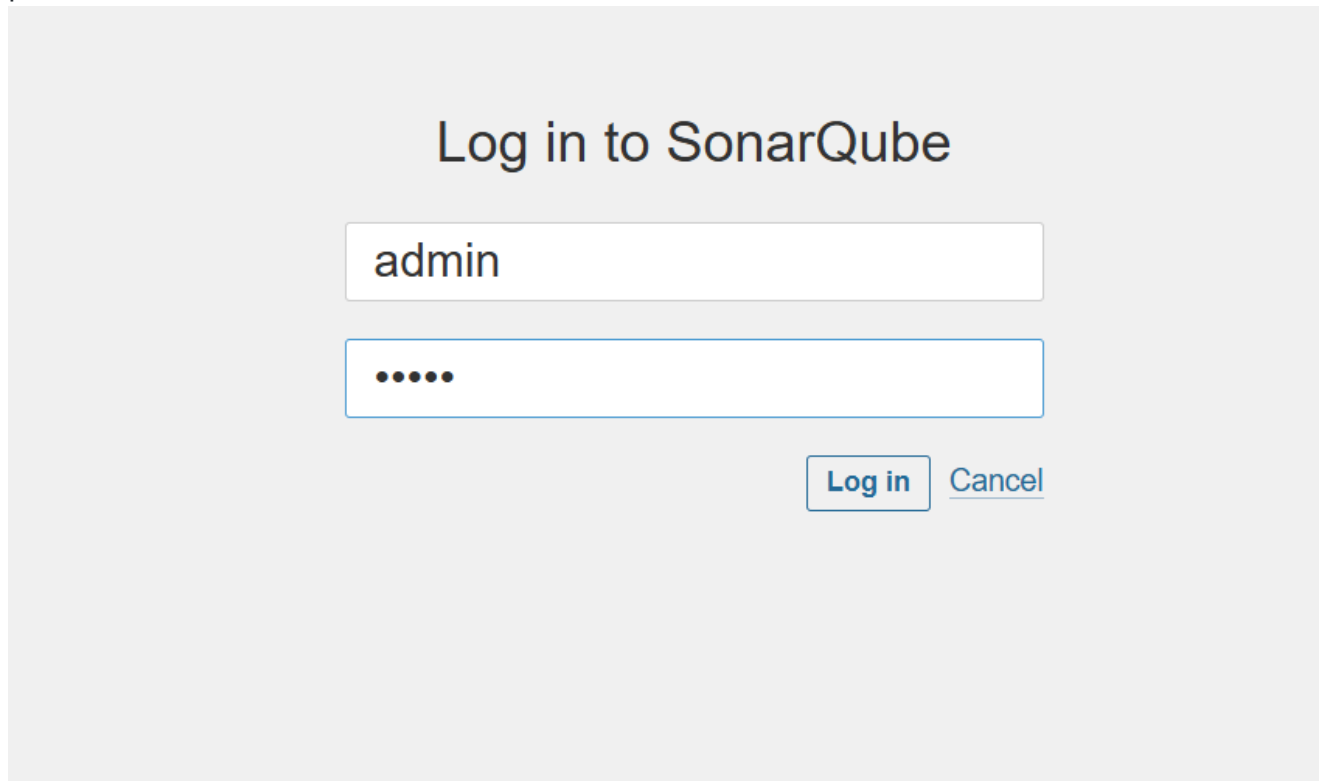
```

Step-4:- Login SonarQube

http://localhost:9000

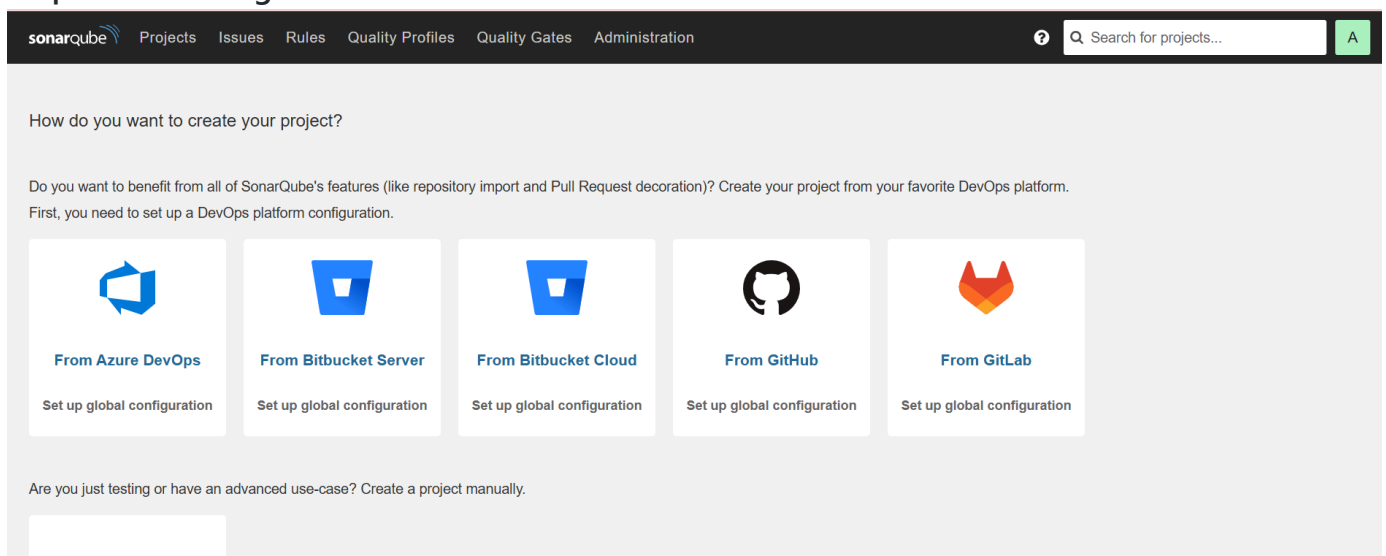
- username: admin

- password: admin



The image shows the SonarQube login interface. At the top, it says "Log in to SonarQube". Below this, there are two input fields: the first contains the username "admin", and the second contains masked characters (dots). To the right of the password field are two buttons: "Log in" and "Cancel".

Step-5:- HomePage



The image shows the SonarQube home page. At the top is a navigation bar with links: sonarqube, Projects, Issues, Rules, Quality Profiles, Quality Gates, and Administration. There is also a search bar and a user icon. Below the navigation bar, the main content area asks "How do you want to create your project?". It provides instructions on creating a project from a DevOps platform and lists five options: From Azure DevOps, From Bitbucket Server, From Bitbucket Cloud, From GitHub, and From GitLab. Each option includes a "Set up global configuration" link. At the bottom, there is a link to "Create a project manually".

Step-6:- Create Sample Java App

```
mkdir -p sample-java-app/src/main/java/com/example
cd sample-java-app
```

```
sristee@Sristee:/mnt/c/Users/srist$ mkdir -p sample-java-app/src/main/java/com/example
cd sample-java-app
```

Step-7:- Create Calculator.java

```
nano src/main/java/com/example/Calculator.java
```

```
package com.example;

public class Calculator {

    public int divide(int a, int b) {
        return a / b;
    }

    public int add(int a, int b) {
        int result = a + b;
        int unused = 100;
        return result;
    }

    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId;
        return query;
    }

    public int multiply(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    public int multiplyAlt(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    public String getName(String name) {
        return name.toUpperCase();
    }

    public void riskyOperation() {
        try {
            int x = 10 / 0;
        } catch (Exception e) {
        }
    }
}
```

```

sristee@Sristee: /mnt/c/Users  ×  +  ▾
GNU nano 7.2 src/main/java/com/example/Calculator.java *
package com.example;

public class Calculator {

    public int divide(int a, int b) {
        return a / b;
    }

    public int add(int a, int b) {
        int result = a + b;
        int unused = 100;
        return result;
    }

    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId;
        return query;
    }

    public int multiply(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    public int multiplyAlt(int a, int b) {
        int result = 0;

```

Step-8:- Create pom.xml

nano pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

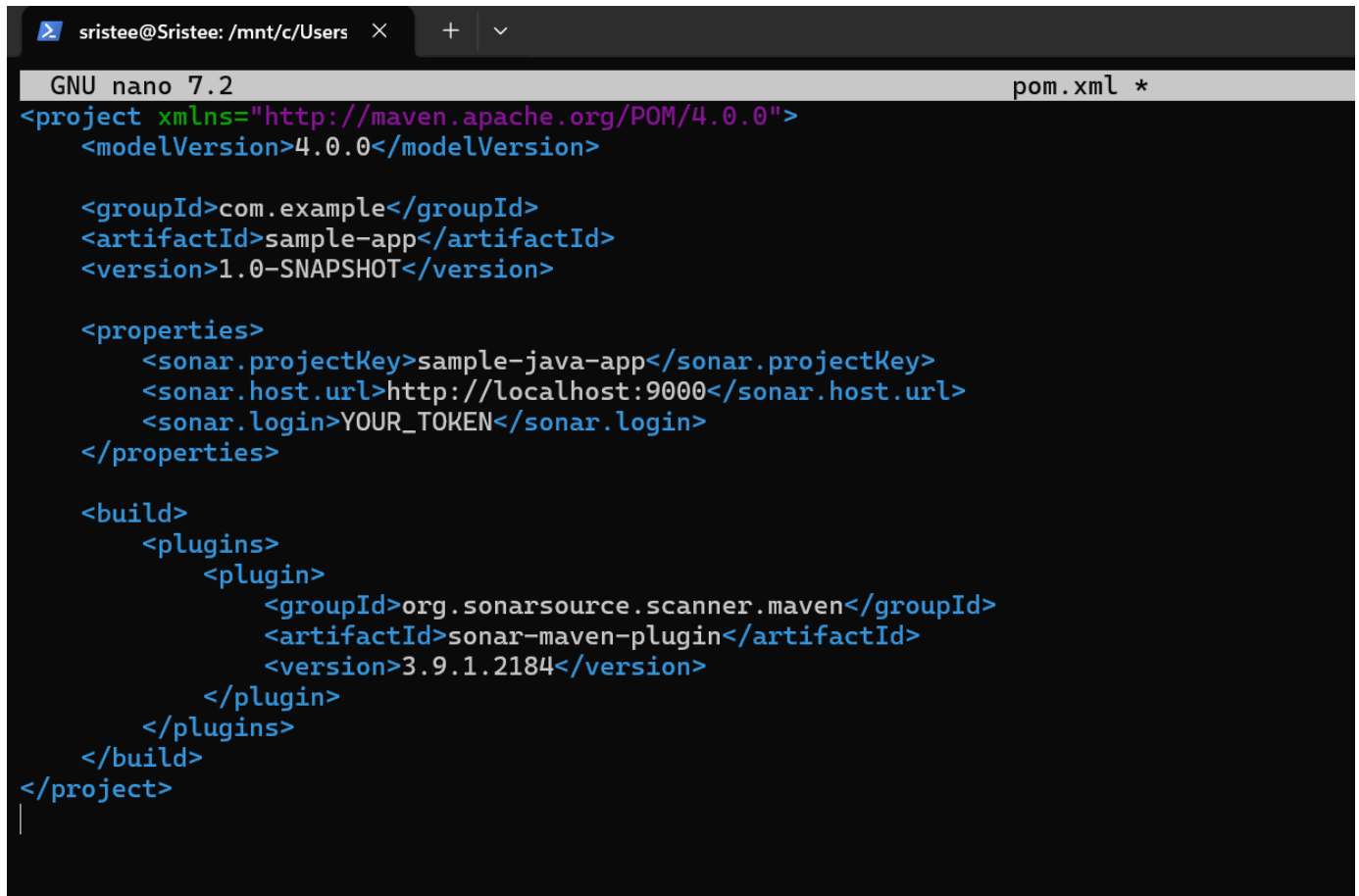
    <groupId>com.example</groupId>
    <artifactId>sample-app</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <sonar.projectKey>sample-java-app</sonar.projectKey>
        <sonar.host.url>http://localhost:9000</sonar.host.url>
        <sonar.login>YOUR_TOKEN</sonar.login>
    </properties>

    <build>
        <plugins>
            <plugin>
                <groupId>org.sonarsource.scanner.maven</groupId>
                <artifactId>sonar-maven-plugin</artifactId>
                <version>3.9.1.2184</version>
            </plugin>
        </plugins>
    </build>

```

```
</build>
</project>
```



```
GNU nano 7.2 pom.xml *
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>sample-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <sonar.login>YOUR_TOKEN</sonar.login>
  </properties>

  <build>
    <plugins>
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
        <version>3.9.1.2184</version>
      </plugin>
    </plugins>
  </build>
</project>
|
```

Step-9:- Create Tokens Profile -> My Account -> Security -> TokenName: scanner-token -> Generate

Tokens

If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan as a replacement of the user login. This will increase the security of your installation by not letting your anal

Generate Tokens

Name	Type	Expires in	
Enter Token Name	Select Token Type	30 days	Generate



New token "scanner-token" has been created. Make sure you copy it now, you won't be able to s



Copy

squ_cb22ef63348c9f31b602b46d067c133ccaf5ccbc

Name	Type	Project	Last use	
scanner-token	User		Never	M

Step-10:- Run Scanner

```
mvn sonar:sonar -Dsonar.login=YOUR_TOKEN
```

```
sristee@Sristee:/mnt/c/Users/srist/sample-java-app$ mvn sonar:sonar -Dsonar.login=squ_cb22ef63348c9f31b602b46d067c133ccaf5ccbc
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-mave
n-plugin-3.9.1.2184.pom
Downloading from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-maven
-plugin-3.9.1.2184.pom (12 kB at 16 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/sonarsource/parent/parent/59.0.29/parent-59.0.29.pom
Downloading from central: https://repo.maven.apache.org/maven2/org/sonarsource/parent/parent/59.0.29/parent-59.0.29.pom (32 kB at 148
kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-mave
n-plugin-3.9.1.2184.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/sonarsource/scanner/maven/sonar-maven-plugin/3.9.1.2184/sonar-maven
-plugin-3.9.1.2184.jar (50 kB at 122 kB/s)
[INFO]
[INFO] -----< com.example:sample-app >-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- sonar-maven-plugin:3.9.1.2184:sonar (default-cli) @ sample-app ---
Downloading from central: https://repo.maven.apache.org/maven2/org/sonatype/plexus/plexus-sec-dispatcher/1.4/plexus-sec-dispatcher-1.
4.pom
Downloading from central: https://repo.maven.apache.org/maven2/org/sonatype/plexus/plexus-sec-dispatcher/1.4/plexus-sec-dispatcher-1.4
.pom (3.0 kB at 55 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/sonatype/spice/spice-parent/12/spice-parent-12.pom
Downloading from central: https://repo.maven.apache.org/maven2/org/sonatype/spice/spice-parent/12/spice-parent-12.pom (6.8 kB at 126 k
B/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/sonatype/forge/forge-parent/4/forge-parent-4.pom
Downloading from central: https://repo.maven.apache.org/maven2/org/sonatype/forge/forge-parent/4/forge-parent-4.pom (8.4 kB at 123 kB/
s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/1.5.5/plexus-utils-1.5.5.pom
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/1.5.5/plexus-utils-1.5.5.pom (5.1 kB a
t 92 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/1.0.11/plexus-1.0.11.pom
```

Step-11:- Scroll down to view Build Success

```
[INFO] 0 source files to be analyzed
[INFO] 0/0 source files have been analyzed
[INFO] Sensor IaC Docker Sensor [iac] (done) | time=43ms
[INFO] ----- Run sensors on project
[INFO] Sensor Analysis Warnings import [csharp]
[INFO] Sensor Analysis Warnings import [csharp] (done) | time=0ms
[INFO] Sensor Zero Coverage Sensor
[INFO] Sensor Zero Coverage Sensor (done) | time=14ms
[INFO] Sensor Java CPD Block Indexer
[INFO] Sensor Java CPD Block Indexer (done) | time=22ms
[INFO] SCM Publisher No SCM system was detected. You can use the 'sonar.scm.provider' property to explicitly specify it.
[INFO] CPD Executor Calculating CPD for 1 file
[INFO] CPD Executor CPD calculation finished (done) | time=11ms
[INFO] Analysis report generated in 141ms, dir size=127.4 kB
[INFO] Analysis report compressed in 743ms, zip size=19.2 kB
[INFO] Analysis report uploaded in 305ms
[INFO] ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=sample-java-app
[INFO] Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
[INFO] More about the report processing at http://localhost:9000/api/ce/task?id=AZ3j1Q2ThbXjnvJdHX98
[INFO] Analysis total time: 6.217 s
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 10.144 s
[INFO] Finished at: 2026-05-01T13:58:04Z
[INFO] -----
sristee@Sristee:/mnt/c/Users/srist/sample-java-app$ |
```

Step-12:- On Browser test will be Listed

A Administrator

Profile Security Notifications Projects

Those projects are the ones you are administering.

sample-app

sample-java-app

Analyzed 1 minute ago

Passed

1 of 1 shown

Step-13:- View Report

sample-app ☆ 🔗 main +

Last analysis of this Branch had [1 warning](#)

Overview Issues Security Hotspots Measures Code Activity

To benefit from more of SonarQube's features, [set up analysis in your favorite CI](#).

QUALITY GATE STATUS ?

Passed

All conditions passed.

MEASURES

New Code

Overall Code

1 🐛 Bugs

0 🔒 Vulnerabilities

← Previous

🏠 Home

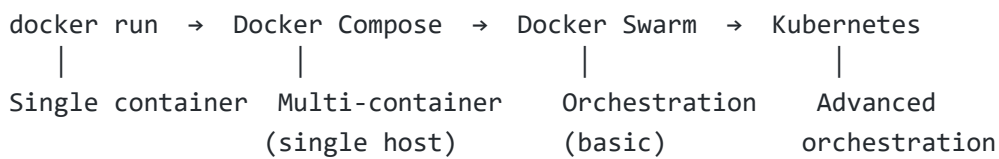
Next →

Containerization-and-DevOps

Experiment 11: Orchestration via Docker Swarm

Introduction

The Progression Path



This experiment focuses on: Moving from Compose → Swarm

Hands-On

Step-1:- Create a `docker-compose.yml`

```
nano docker-compose.yml
```

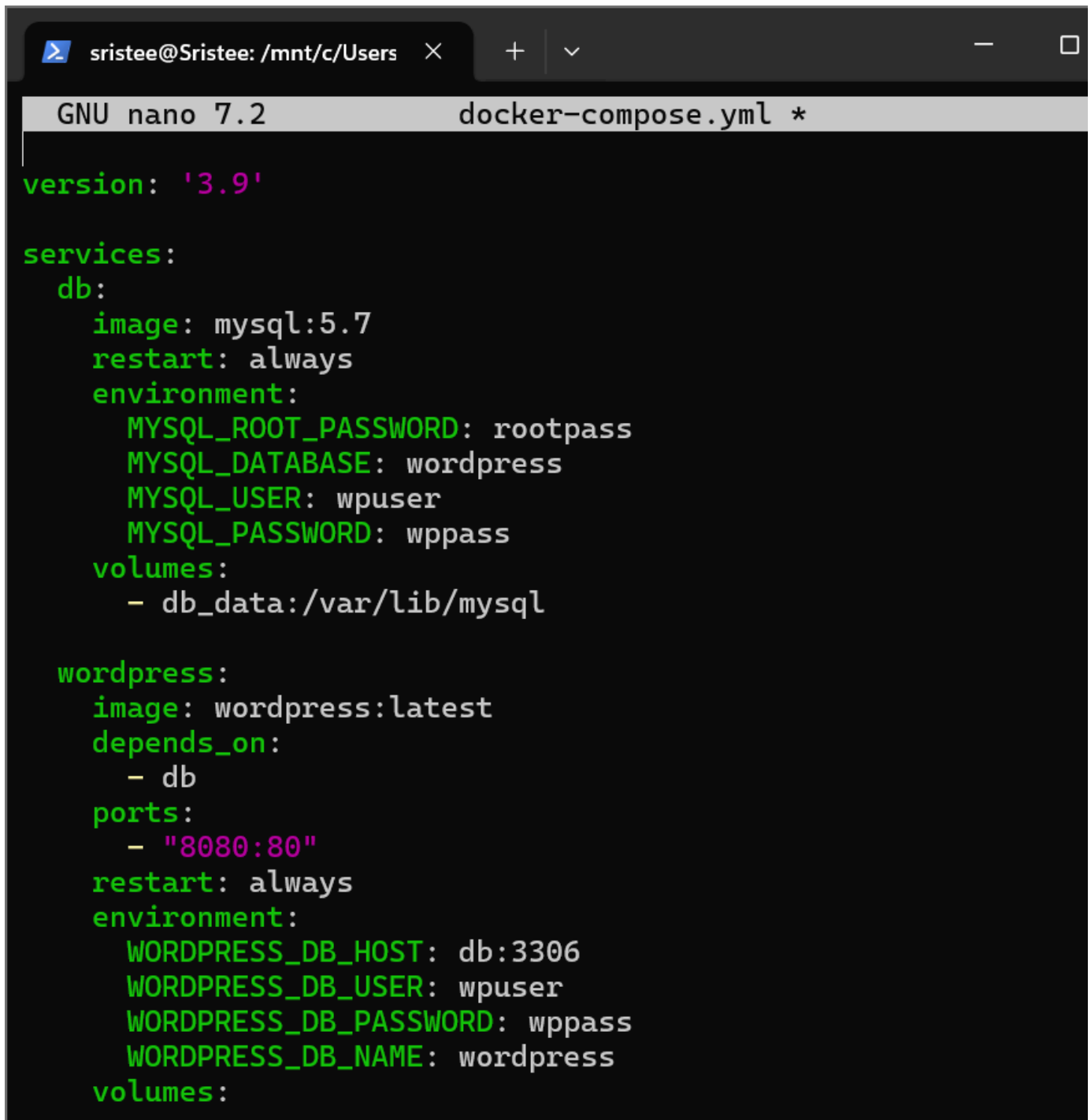
Paste This:

```
version: '3.9'

services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql
```

```
wordpress:
  image: wordpress:latest
  depends_on:
    - db
  ports:
    - "8080:80"
  restart: always
  environment:
    WORDPRESS_DB_HOST: db:3306
    WORDPRESS_DB_USER: wpuser
    WORDPRESS_DB_PASSWORD: wppass
    WORDPRESS_DB_NAME: wordpress
  volumes:
    - wp_data:/var/www/html
```

```
volumes:
  db_data:
  wp_data:
```



```

> sristee@Sristee: /mnt/c/Users  ×  +  ▾
GNU nano 7.2 docker-compose.yml *
version: '3.9'

services:
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    depends_on:
      - db
    ports:
      - "8080:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:

```

Step-2:- Initialize Swarm

```
docker swarm init
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker swarm init
Swarm initialized: current node (q3csz6ymrj72glarz8s9t7ya1) is now a manager.
```

To add a worker to this swarm, run the following command:

```
docker swarm join --token SWMTKN-1-49wisfectycuemewlvautm7e0ff4rx3tth9zcgct6zzugk6p0-6w7582ptvp31g8kas4b0a7g7v 192.168.65.3:2377
```

Step-3:- Verify Swarm is Active

```
docker node ls
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILAB
q3csz6ymrj72glarz8s9t7ya1	* docker-desktop	Ready	Active
Leader	28.4.0		

Step-4:- Deploy as a stack

```
docker stack deploy -c docker-compose.yml wpstack
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker stack deploy -c docke
r-compose.yml wpstack
Ignoring unsupported options: restart

Since --detach=false was not specified, tasks will be created in
the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress
```

Step-5:- Verify by listing Services

```
sristee@Sristee:/mnt/c/Users/srist$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE
k2msptowdv5x	wpstack_db	replicated	1/1	mysql
s0l6diouv9h8	wpstack_wordpress	replicated	1/1	wordp

Step-6:- Details for container service

```
docker service ps wpstack_wordpress
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker service ps wpstack_wo
rddress
```

ID	NAME	IMAGE	NODE
ruhtqouunusx	wpstack_wordpress.1	wordpress:latest	docker-d
esktop	Running	Running 25 seconds ago	

Step-7:- List Containers

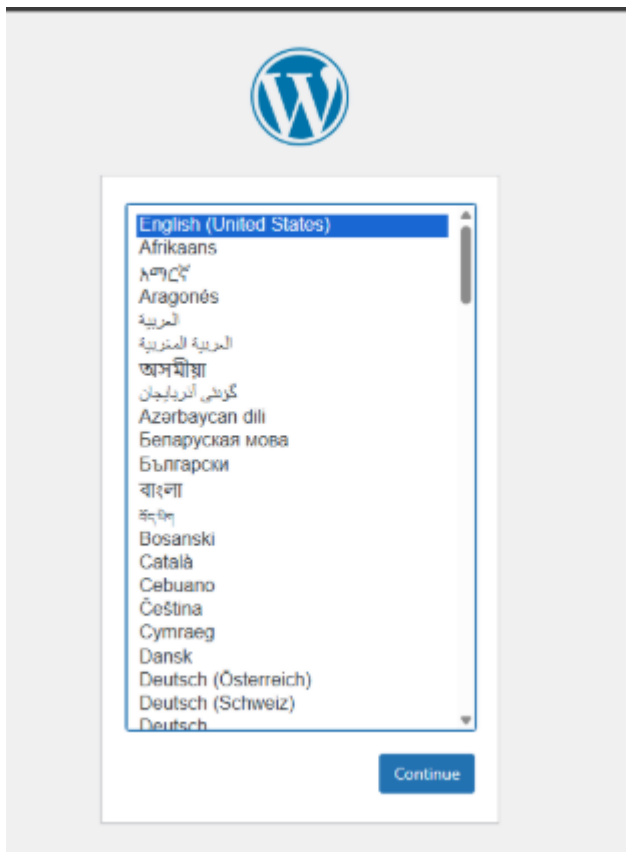
```
docker ps
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	PORTS
b960912c9742	wordpress:latest	"docker-entrypoint.s..."	80/tcp
9fcab7ab091b	mysql:5.7	"docker-entrypoint.s..."	3306/tcp, 33060/tcp
3a6ec2eecb2d	postgres:13	"docker-entrypoint.s..."	5432/tcp
7284201b6163	sonarqube:lts-community	"/opt/sonarqube/dock..."	0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
f25c36a0b318	ubuntu-server	"/usr/sbin/sshd -D"	0.0.0.0:8221->8221/tcp, [::]:8221->8221/tcp, 0.0.0.0:2222->22/tcp, [::]:2222->22/tcp
69c176800b45	jenkins/jenkins:lts	"/usr/bin/tini -- /u..."	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp
762697afd308	container-app-backend	"docker-entrypoint.s..."	

Step-8:- Verify application on Containers

```
http://localhost:8080/
```



Step-9:- Scale application Containers

```
docker service scale wpstack_wordpress=3
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker service scale wpstack
_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service wpstack_wordpress converged
```

Step-10:- Verify Scalling

```
docker service ls
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE
k2msptowdv5x:5.7	wpstack_db	replicated	1/1	mysql
s0l6diouv9h8ress:latest	wpstack_wordpress	replicated	3/3	wordp

Step-11:- List Wordpress Containers

```
docker ps | grep wordpress
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker ps | grep wordpress
713ae8d37b41    wordpress:latest    "docker-entrypoint.s..."
    18 seconds ago    Up 18 seconds        80/tcp

    wpstack_wordpress.3.kmu14mkwjzwtcrzdti8uxz
9fw
6799136ffe14    wordpress:latest    "docker-entrypoint.s..."
    18 seconds ago    Up 18 seconds        80/tcp

    wpstack_wordpress.2.zitfnb152cqfkveveocvgv
ajp
b960912c9742    wordpress:latest    "docker-entrypoint.s..."
    About a minute ago    Up About a minute    80/tcp

    wpstack_wordpress.1.ruhtqouunusxzano0pvi2t
yj6
```

Step-12:- Kill any Container

```
docker kill <container-id>
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker kill '713ae
713ae
```

Step-13:- Watch Swarm Recreating Contrainer

```
docker service ps wpstack_wordpress
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker stack rm wpstack
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
```

Step-14:- Verify Container running

```
docker ps | grep wordpress
```

Step-15:- Remove Stack

```
docker stack rm wpstack
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker stack rm wpstack
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
```

Step-16:- Verify Service Removal

```
docker service ls
docker ps
```

```
sristee@Sristee:/mnt/c/Users/srist$ docker service ls
docker ps
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
CONTAINER ID	IMAGE	COMMAND	PORTS		
3a6ec2eecb2d	postgres:13	"docker-entrypoint.s..."	5432/tcp		
7284201b6163	sonarqube:lts-community	"/opt/sonarqube/dock..."	0.0.0		
f25c36a0b318	ubuntu-server	"/usr/sbin/sshd -D"	0.0.0		
69c176800b45	jenkins/jenkins:lts	"/usr/bin/tini -- /u..."	0.0.0		
762697afd308	container-app-backend	"docker-entrypoint.s..."			
f1f291734380	container-app-database	"docker-entrypoint.s..."			
	postgres_db				

Conclusion

Summary

You started with	You can now do
Single container (<code>docker run</code>)	Multi-container (Compose)
Manual scaling	One-command scaling (<code>scale</code>)
Manual recovery	Automatic self-healing
Single host	Multi-host cluster ready

Final Takeaway

Compose defines the application. Swarm runs it reliably.

Quick Reference Card

Initialize Swarm

```
docker swarm init
```

Deploy stack

```
docker stack deploy -c docker-compose.yml <stack-name>
```

List services

```
docker service ls
```

Scale service

```
docker service scale <stack-name_service-name>=<replicas>
```

See service tasks

```
docker service ps <service-name>
```

Remove stack

```
docker stack rm <stack-name>
```

Leave Swarm (if needed)

```
docker swarm leave --force
```

[< Previous](#) [Home](#) [Next >](#)

Containerization-and-DevOps

Experiment 12: Kubernetes

Introduction

Why Kubernetes over Docker Swarm?

Reason	Explanation
Industry standard	Most companies use Kubernetes
Powerful scheduling	Automatically decides where to run your app
Large ecosystem	Many tools and plugins available
Cloud-native support	Works on AWS, Google Cloud, Azure, etc.

Core Kubernetes Concepts (Simple Explanation)

Docker Concept	Kubernetes Equivalent	What it means
Container	Pod	A pod is a group of one or more containers. Smallest unit in K8s.
Compose service	Deployment	Describes how your app should run (e.g., 2 copies, which image to use)
Load balancing	Service	Exposes your app to the outside world or other pods
Scaling	ReplicaSet	Ensures a certain number of pod copies are always running

Hands-On

Step-1:- Create a `wordpress-deployment.yaml`

```
nano wordpress-deployment.yaml
```

Paste This:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
        - name: wordpress
          image: wordpress:latest
          ports:
            - containerPort: 80
```

```
apiVersion: apps/v1|
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 2
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
      - name: wordpress
        image: wordpress:latest
        ports:
        - containerPort: 80
```

Step-2:- Apply Deployment

```
kubectl apply -f wordpress-deployment.yaml
```

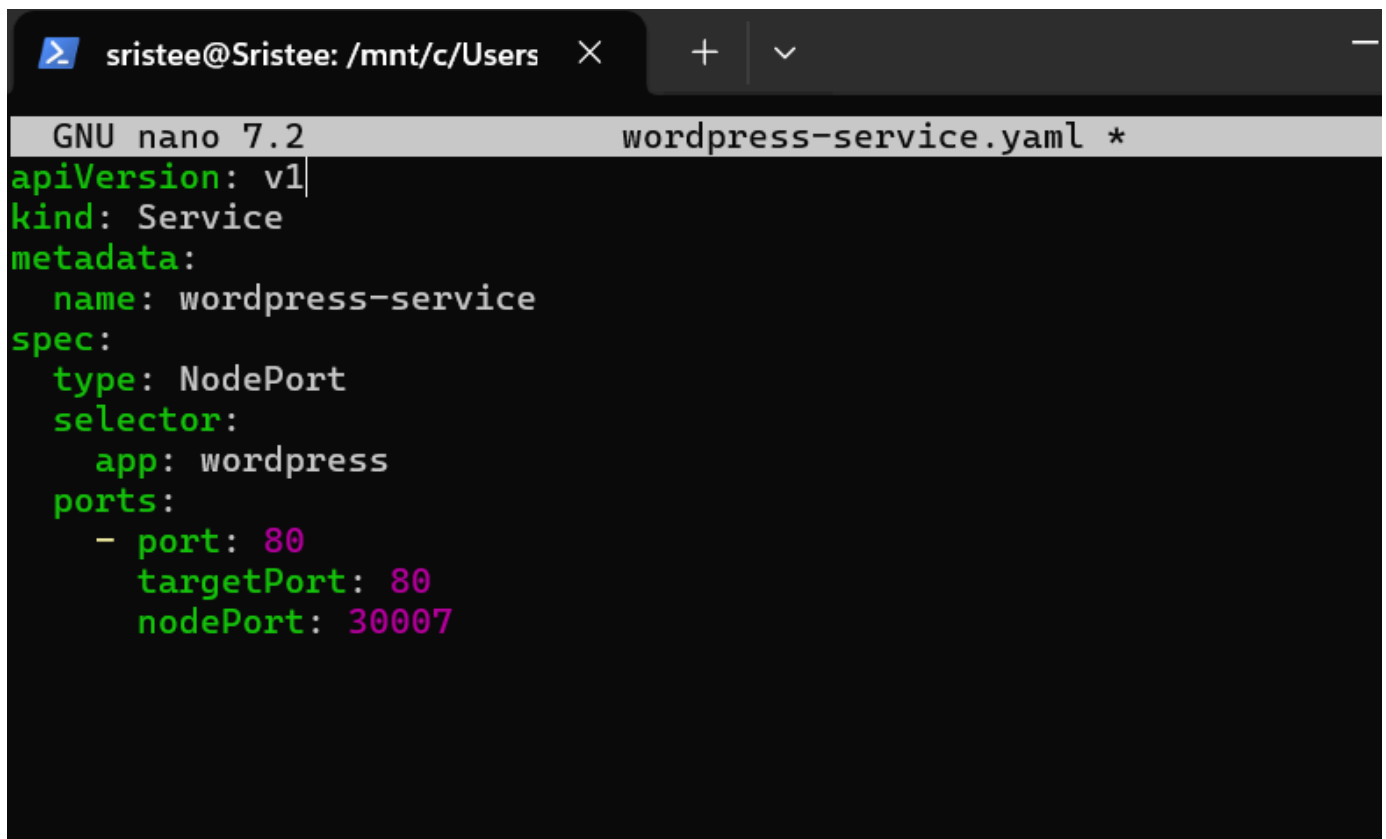
```
sristee@Sristee:/mnt/c/Users/srist$ kubectl apply -f wordpress-deployment.y
aml
deployment.apps/wordpress created
```

Step-3:- Create wordpress-service.yaml

```
nano wordpress-service.yaml
```

Paste this:-

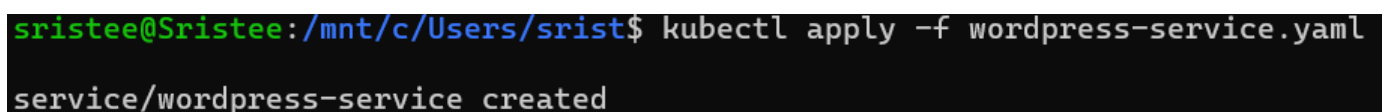
```
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```



```
srstee@Sristee: /mnt/c/Users
GNU nano 7.2 wordpress-service.yaml *
apiVersion: v1
kind: Service
metadata:
  name: wordpress-service
spec:
  type: NodePort
  selector:
    app: wordpress
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

Step-4:- Deploy Service

```
kubectl apply -f wordpress-service.yaml
```



```
srstee@Sristee: /mnt/c/Users/sristee$ kubectl apply -f wordpress-service.yaml
service/wordpress-service created
```

Step-5:- Get pods

```
kubectl get pods
```

```
sristee@Sristee:/mnt/c/Users/srist$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-7d6f6db8d8-62ttm	1/1	Running	0	2m
wordpress-7d6f6db8d8-cn197	1/1	Running	0	2m

Step-6:- Check service

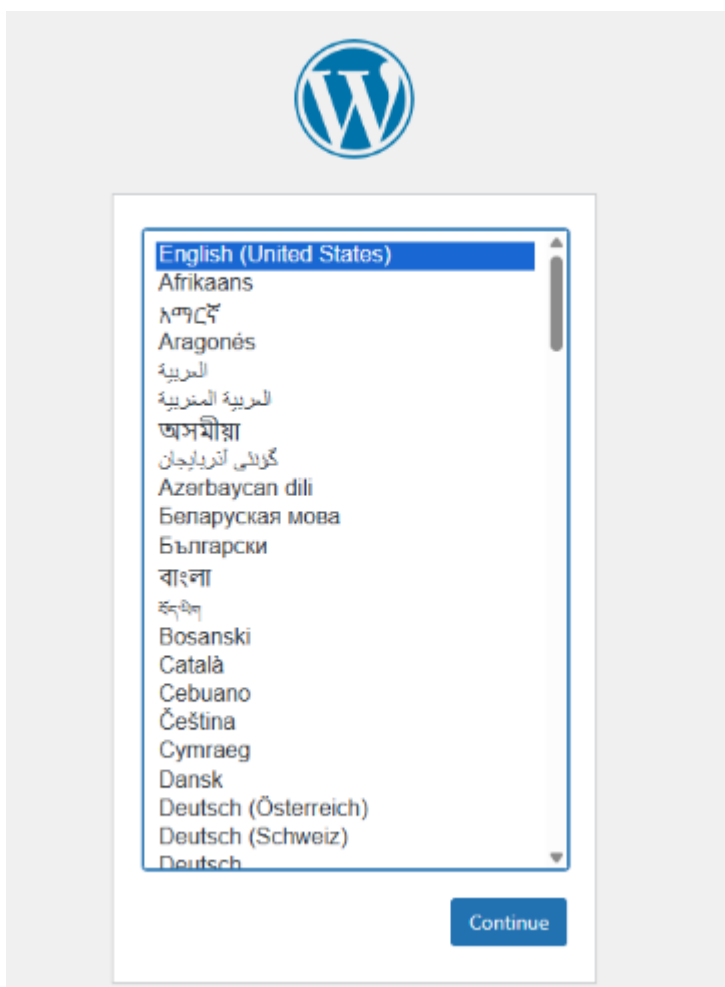
```
kubectl get svc
```

```
sristee@Sristee:/mnt/c/Users/srist$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP
wordpress-service	NodePort	10.43.207.46	<none>	80:30007/TCP

Step-7:- Verify On Browser

```
http://localhost:8080/
```



Step-8:- Scale Deployment

```
kubectl scale deployment wordpress --replicas=4
```

```
sristee@Sristee:/mnt/c/Users/srist$ kubectl scale deployment wordpress --re  
plicas=4  
deployment.apps/wordpress scaled
```

Step-9:- Get Pods

```
kubectl get pods
```

```
sristee@Sristee:/mnt/c/Users/srist$ kubectl get pods  
NAME                                READY   STATUS    RESTARTS   AGE  
wordpress-7d6f6db8d8-62ttm         1/1     Running   0           3m37s  
wordpress-7d6f6db8d8-b8hlx         1/1     Running   0           19s  
wordpress-7d6f6db8d8-cn197         1/1     Running   0           3m37s  
wordpress-7d6f6db8d8-rzsgr         1/1     Running   0           19s
```

Step-10:- Delete Pods to check Self-healing

```
kubectl delete pod <pod-name>  
kubectl get pods
```

```
sristee@Sristee:/mnt/c/Users/srist$ kubectl delete pod wordpress-7d6f6db8d8  
-62ttm  
pod "wordpress-7d6f6db8d8-62ttm" deleted from default namespace
```

Insights

When to Use Which Tool

Tool	Best for
k3d	Quick learning on your laptop
Minikube	Single-node cluster testing
kubeadm	Real, production-style cluster

Summary of Commands (Cheat Sheet)

Goal	Command
Apply a YAML file	<code>kubectl apply -f file.yaml</code>

Goal	Command
See all pods	<code>kubectl get pods</code>
See all services	<code>kubectl get svc</code>
Scale a deployment	<code>kubectl scale deployment <name> --replicas=N</code>
Delete a pod	<code>kubectl delete pod <pod-name></code>
See all nodes	<code>kubectl get nodes</code>

[!\[\]\(98ece4d13f19770642db27ae956e1502_img.jpg\) Previous](#) [!\[\]\(47a33ccab9dd90762cc6a9d597eb24f4_img.jpg\) Home](#)